

The Representational Grammar

A Finite and Local Mapping of Formal Symbols

Bill Cochran
wkcochran@gmail.com
<https://github.com/wkcochran123/measurement>

July 12, 2026

Preface: The Freedom to Answer

A question and an answer are not the same freedom.

You may choose where to put the apparatus, which dial to turn, which basis to use, which symbol to write at the top of the page. None of that entitles you to choose what the apparatus says back. The first freedom belongs to the reader of the world: the freedom to ask in one way rather than another. The second belongs, if it belongs anywhere, to the thing being read: the freedom to answer from its own state rather than echo the name placed upon it. Confuse those two and calibration becomes ventriloquism. The instrument appears to speak, but the voice coming out of it is yours.

This volume is about the distance between those freedoms. It is the fourth reading of one machine: a finite and local grammar for asking what licenses the names assigned to formal reports. The first three volumes build and turn the instrument in the languages of construction, physics, and computation. This one follows representations, transitions, slices, and receipts. Its order is the order in which an instrument needs them: first a question, then a selectable way to pose it, then an answer, then a record that can survive being read again. The companion technical treatment retains the title *Scientific Computation*; this volume governs how its symbols may be mapped and named.

Four words stand at that threshold: *free will, the axiom of choice, witness,*

and *observation*. They are often made to blur into one another. Choice is called freedom; seeing is called witnessing; an answer is treated as though it had been selected; a selected name is treated as though it were already an observed fact. The machine needs the four kept apart, and it needs them arranged on two axes. One axis separates a *requirement* from an *action*: what must be possible from what is actually done. The other separates the contravariant from the covariant: what comes back against a reading from what travels forward with the thing read. The whole preface fits in one small table:

	requirement	action
contravariant	axiom of choice	witness
covariant	free will	observation

The table is not a demonstration about human freedom, and it is not a new axiom added to the construction. It is the reading this volume lays over the calibration. Its value is disciplinary: it tells us who owes what at every stage, and prevents a requirement from masquerading as its own fulfillment.

Begin with the contravariant requirement. Before an experiment can return a value, a requested reading determines a family of setups under which a value would count as its answer. The requirement is that this family contain at least one admissible member: a frame must be choosable, a representative nameable, a detector orientable, a coordinate available to be fixed. Change the object one way and the coordinate description must change against it so that their pairing continues to mean the same thing. That backward demand — from the claim we would like to state to the admissible setup that could justify it — is contravariant.

Promote that demand far enough and it becomes the axiom of choice. Given a family of nonempty collections, choice says that a representative may be selected from each even when no common rule tells us which representative to take. Zermelo stated the axiom while establishing the well-ordering

result [11]; Gödel showed its relative consistency [4]; Cohen made forcing into a working method for independence demonstrations [3]. The controversy was never over whether a person can point at one apple. It was over whether the existence of a selector may be granted globally when the selecting act has not been exhibited. In the language of this instrument, the axiom of choice is therefore a *requirement*: let there be a way to choose the representatives the reading demands. It is an open invoice, not a receipt.

The receipt is the witness. A witness is the contravariant action: not the promise that a representative exists, but the representative produced; not “there is a path,” but this path; not “some value satisfies the condition,” but this value together with the check that it does. Constructive mathematics makes the distinction unusually hard to evade, because evidence for an existential claim carries a particular witness [7]. A requirement points back from a proposed conclusion toward what would justify it. A witness walks that route and comes back carrying the justification.

This is why a witness bears a cost. The cost need not be mystical and it need not be money. In this device it is the finite work of constructing, evaluating, and validating the record that fills the required place. An axiom can say, in one stroke, that a selector exists. A witness must perform the selection in the case at hand. The former licenses; the latter acts. The pull-back of the requirement is the literal contravariant operation. The witness is its contravariant action because it travels against the observation arrow to fill that pulled-back demand with a particular handle.

Now turn the table over. Free will, as the phrase is used here, is not the observer’s privilege of picking a coordinate. That would put it back in the contravariant column and make it merely another name for choice. Free will is the *covariant requirement*: after the question has been chosen, the system must still be able to answer. Its response must travel with its state and with the admissible change of circumstances; it must not have been filled in by the label chosen to solicit it.

This is a narrower claim than the old quarrel between necessity and liberty, though it touches that quarrel. Hume asked how freedom could be made intelligible in a world of causes [5]; Kant placed freedom at the fault line between the world as described and the agency by which a description is taken up [6]. The operational answer offered here is modest. Freedom does not mean lawlessness, randomness, or exemption from cause. A compass needle is not free *from* the field; it is free to answer the field rather than the word painted beneath it. Covariance is the form of that lawful freedom. Change the admissible frame and the response changes with it according to a rule. What must remain free is not the response from law, but the response from having been decided in advance by the questioner's notation.

The covariant action is observation. Observation is not a private glance and not the passive discovery of a value that was already printed somewhere behind the apparatus. It is the act by which a system's response is gathered into a record. Bohr insisted that a quantum phenomenon includes the conditions under which it is observed [1]; von Neumann made the chain from system through apparatus to record mathematically explicit [8]. This instrument keeps the same useful caution without asking those theories to prove its construction. The chosen setup may condition or disturb the response, but it must not predetermine or copy the requested report. Observation carries the answer forward from the system into the ledger. That forward carriage is why the action is covariant.

The four terms can be written more compactly. Let X be the typed space of world-apparatus states $x = (N, s)$, let R be the reading space, and let G be the interaction-trace space. Observation is a covariant action

$$O : X \longrightarrow X \times R \times G, \quad O(N, s) = (N', s', y, \Gamma).$$

Let $\pi_y : X \times R \times G \rightarrow R$ project the observation component. Here $y \in R$ is the observation and $\Gamma \in G$ is the interaction trace. The receipt retains prior state and history, intervention, actuation, geometry, sensor loading,

back-action, energy and material exchange, latency, hysteresis, drift, saturation, calibration, environment, rejected outcomes, uncertainty, and alternative causal models. The world answers the intervention in N' , not an untouched N .

A requested reading pulls backward to the world-apparatus states that could witness it. Write $\approx_\varepsilon$ for a declared comparison relation, and define

$$F_r^\varepsilon := \{x \in X : (\pi_y \circ O)(x) \approx_\varepsilon r\}, \quad R_{\text{adm}} := \{r \in R : F_r^\varepsilon \neq \emptyset\}.$$

Choice cannot make an empty fiber nonempty. Its requirement is only that representatives can be selected from the nonempty fibers. A global selector — an approximate section — would be a map $W : R_{\text{adm}} \rightarrow X$ with $W(r) \in F_r^\varepsilon$. The contravariant action in a particular case is the local witness $w_r := W(r)$. Calibration then asks, point by point, for

$$(\pi_y \circ O)(w_r) \approx_\varepsilon r, \quad r \in R_{\text{adm}}.$$

Choose a representative, run the observation, and recover the claimed reading within the instrument's owned floor. In a finite case the device may construct the needed local witness without invoking a global choice principle. Define

$$X_{\text{adm}} := \{x \in X : (\pi_y \circ O)(x) \in R_{\text{adm}}\}.$$

The induced subtype map is

$$\overline{O} : X_{\text{adm}} \rightarrow R_{\text{adm}}, \quad \overline{O}(x) = ((\pi_y \circ O)(x), \text{ its defining membership evidence}).$$

Let $\iota : X_{\text{adm}} \hookrightarrow X$ be inclusion. Calibration does not require the typed reconstruction identity of maps $X_{\text{adm}} \rightarrow X$,

$$W \circ \overline{O} = \iota.$$

A successful name need not reconstruct the whole world. The label is a usable handle, not the object exhausted. That asymmetry is the space in which both measurement and humility live.

One pass, however, is not enough. On one pass the selected representative and the returned reading may agree merely because the selection planted the answer. The proposed protocol therefore has two measurement passes and one role exchange. Its finite grammar can be replayed on computational records; physical passes are auditable only when two covariant intervention receipts exist. At either pass, two of three typed roles are returned or verified directly while the third is marked unresolved. The unresolved marker is neither zero nor absence.

Write the roles as $a \in A, b \in B, c \in C$, a composite domain Z , and declare

$$\Sigma : A \times B \times C \rightarrow Z, \quad R_c : A \times B \times Z \rightarrow C, \quad R_a : B \times C \times Z \rightarrow A.$$

The word *superposition* means only this declared composite relation. It does not, by itself, mean ordinary addition, a linear or quantum state, or a mechanism behind a physical process. The two passes have the schematic form

$$\begin{aligned} t_1 : \quad & a_1, b_1 \text{ returned directly,} \quad c \text{ unresolved,} \quad z_1 \text{ observed,} \\ t_2 : \quad & b_2, c_2 \text{ returned directly,} \quad a \text{ unresolved,} \quad z_2 \text{ observed.} \end{aligned}$$

The maps and comparison rules are fixed before either target is read. The report retains separately:

$$b_1 \sim b_2, \quad R_c(a_1, b_1, z_1) \sim c_2, \quad R_a(b_2, c_2, z_2) \sim a_1.$$

Each $\sim$ expands into separate exact-equality, resolution-equality, and uncertainty-compatibility rows. The round-trip row uses both reconstruction maps and the carried-role overlap; no combined agreement bit erases

which relation passed. The unresolved place must change hands, and preregistered maps must survive the exchange. If the same slot remains unresolved twice, nothing new has been learned. If a map changes to preserve a favored label, the name has been fitted rather than inferred.

The role scheduler $a \mapsto b \mapsto c \mapsto a$ has algebraic period three. Three scheduler applications returning every role is not a requirement for three calibration passes; the calibration still has two passes and one exchange.

This is what allows the machine to use names. A name does not assert that all three values were simultaneously sharp. It names the role that persists when sharpness circulates among them. On the first pass the witness fixes a frame and the observation returns two direct values plus one relation. On the second pass a new witness changes which value is held relationally, and a new observation tests whether the old relation survives. What earns the name is the invariant across the two passes: the same transition law, recovered when the direct and superposed roles exchange. The name is contravariant — ours, frame-bound, a handle chosen against the change. What licenses the name is covariant — the world's answer, carried through the change and returned repeatably.

The two-pass calibration therefore closes the table into a loop. The contravariant requirement asks for representatives from the stipulated nonempty fibers; the axiom grants a selector globally, while the witness supplies a representative locally. Free will asks whether the selected state can have a consequence computed from that state rather than stipulated by its name. Observation supplies that consequence. The second pass then turns the consequence into a new requirement and asks the loop to survive once more. Requirement becomes action; action becomes the next requirement. Contravariant question and covariant answer meet in a report stable up to $\approx_\varepsilon$ — for this instrument, the repeatable bracket it owns.

That loop also explains how the four volumes belong together. They are not four independent votes cast for the same conclusion. They partition one

serial construction and place different requirements upon it. Construction asks what must exist; physics asks what could be observed; computation asks what can be witnessed and at what governed toolchain-counter cost; this volume audits finite reports and checks how the translations are calibrated. Agreement among the gauges is useful only where the same relation survives the change of register. The second pass tests a declared translation; it does not, by itself, turn that translation into an established physical interpretation.

One final rule of honesty belongs at the door. The project distinguishes what is *built*, what is *measured*, and what is *interpreted*. Finite definitions, demonstrated relations, evaluations, and logical-dependency footprints are built. Governed counter reports are measured only within their recorded toolchain, dependency, snapshot, configuration, and run conditions. The language of free will, choice, witness, observation, and physical naming is interpretation: the reading this volume proposes, not a result supplied by internal validation. The owned result is the open bracket the model reports. Its safest name is the compiler structure constant, α_c ; comparison with a physical constant is a calibrated apparatus interpretation performed by another register, never substitution of an outside target for the model's report.

That distinction is why the finite reports remain open to audit beside the prose. Where this volume leans on established mathematics or experimental science, it names the people whose work supplies the ground. Where the citations thin, the claim is ours and the reader should slow down. Generosity is part of the calibration: it lets inherited knowledge and new interpretation be told apart. But no citation and no paragraph replaces the second pass. The reader is handed the instrument because replay is the final witness of computational repeatability; physical naming additionally requires the covariant apparatus receipts described above.

You may choose the question. You may not choose the answer. The world must be allowed to answer, the instrument must make a record, and

the record must survive the exchange of roles before its names are earned.

Choose the frame. Produce the witness. Let the world answer. Observe it twice. Then — and only then — use the name.

Contents

Preface: The Freedom to Answer	i
I A Difference	1
1 A Difference Appears	2
1.1 Difference is a predicate, not a particle	2
1.2 The symbol and the receiver	10
1.3 From a decision to a record	14
2 Difference Becomes Countable	19
2.1 Counting is an interface	19
2.2 The update and the alleged endpoint	24
2.3 A finite trace is not an infinite object	28
3 The Residue Refuses To Disappear	35
3.1 The branch that cannot be silently discarded	35
3.2 Iterative error versus structural residue	40
3.3 Observation becomes history	45
II The Turn	51
4 The Tower Turns Around	52

4.1	Requirements pull backward	52
4.2	Witnesses run forward	58
4.3	Two-pass calibration	62
5	The Slip Becomes A Number	69
5.1	A crossing predicate	69
5.2	Bisection is iteration by slicing	75
5.3	A report is not a certificate	79
6	The Bracketed Number	85
6.1	Dyadic data structure	85
6.2	Resolution as an explicit parameter	89
6.3	Stop rules and budgets	94
III	The Faces	99
7	The Number Splits Into Faces	100
7.1	Computation names	100
7.2	Physics names	105
7.3	Same number or same process	109
8	The Electron Is Named	113
8.1	Partition and collision	113
8.2	A collision is not a cycle	117
8.3	When a physical name is licensed	120
9	The Corridor Rotates	125
9.1	Computational rotation	125
9.2	Physical phase	129
9.3	Resonance audit	133

IV	The Closing	139
10	The Native Apparatus Presents the Residue	140
10.1	Apparatus ratio	140
10.2	The physical balance	144
10.3	Slicing the same ledger	149
11	The Meter	154
11.1	What a heartbeat counts	154
11.2	Warmup, read, subtract	158
11.3	Self-application without mystification	163
12	The Field Closes	168
12.1	Second differences	168
12.2	Physics field equations	172
12.3	Iteration and slicing in the field report	178
V	The Bound	183
13	Alpha As A Bracket	184
13.1	The internal formula	184
13.2	The count-to-three bracket	188
13.3	Nodes, quadrature, and root finding	193
14	The Machine Measures Its Own Measurement	199
14.1	The two propositions	199
14.2	The cost readings	203
14.3	The proposed second pass	207
15	What The Machine Can And Cannot Claim	212
15.1	What is built	212
15.2	What is measured	217

CONTENTS

xiii

15.3 What is interpreted	221
------------------------------------	-----

Part I

A Difference

Chapter 1

A Difference Appears

1.1 Difference is a predicate, not a particle

Scientific computation begins by fixing a grammar before assigning meaning to its names. Let M be a finite alphabet of marks:

$$\begin{aligned} \textit{mark} &::= m_0 \mid \cdots \mid m_{q-1}, \\ \textit{question} &::= \text{diff}(s, \sigma), \\ \textit{answer} &::= \text{yes}(w) \mid \text{no}(r). \end{aligned}$$

The payload w certifies that two marks differ; r certifies that they do not. The grammar does not make a mark a material object, a question a detector, or an answer an observation. Those names require separate provenance.

For a reference $\sigma \in M$, the governing question is

$$D_\sigma(s) := (s \neq \sigma).$$

A resolver Δ must terminate with an evidenced judgment,

$$\frac{w : s \neq \sigma}{\Delta(s, \sigma) \Downarrow \text{yes}(w)} \quad \frac{r : \neg(s \neq \sigma)}{\Delta(s, \sigma) \Downarrow \text{no}(r)}.$$

A later routine may erase the evidence:

$$E(\text{yes}(w)) = 1, \quad E(\text{no}(r)) = 0.$$

This external Boolean encoding is lossy. Its bit records the selected branch but not the certificate that licensed it.

Reading has a direction. Let X be a finite collection of records and let $e : X \rightarrow M$ represent each record by a mark. Construction sends records forward through e . A question $q : M \rightarrow \{0, 1\}$, or its evidence-bearing refinement, is read contravariantly by pullback:

$$e^*q := q \circ e : X \rightarrow \{0, 1\}.$$

For a second representation $g : M \rightarrow N$,

$$(g \circ e)^* = e^* \circ g^*.$$

The order reverses: first pull the question on N through g , then through e . This is an external representation schema, not a claim that a particular map e has already been constructed. It separates the covariant forward construction of a representation from the contravariant act of reading it.

An audit record is

$$\mathcal{R} = (X, M, e, \sigma, D, \Delta, E, (A, R), J, F, P),$$

where J states invariants, F states failures, and P tags computation, physics, or interpretation. For $q = E \circ \Delta(-, \sigma)$,

$$A = \{x \in X \mid (e^*q)(x) = 1\}, \quad R = \{x \in X \mid (e^*q)(x) = 0\}.$$

The words accepted and rejected are conventional branch labels, not value judgments.

The finite algorithm is explicit. Given $L = [x_0, \dots, x_{n-1}]$ and an independent decision budget N , initialize two empty output lists and $i \leftarrow 0$. While $i < n$ and $i < N$, compute $m_i = e(x_i)$, resolve $\Delta(m_i, \sigma)$, append the record and its evidence to the corresponding slice, and update $i \leftarrow i + 1$. Stop with reason COMPLETE when $i = n$, BUDGET when $i = N < n$, or RESOLVER-DOMAIN when the declared resolver does not apply to (m_i, σ) . If k records are visited, its cost is

$$C(L, N) = \sum_{i=0}^{k-1} (C_e(x_i) + C_\Delta(e(x_i), \sigma) + C_{\text{append}}(i)), \quad k \leq \min(n, N).$$

Its certificate contains the visited prefix $[x_0, \dots, x_{k-1}]$, both evidence-bearing slices, the stop reason, and every declared Boolean erasure. Coverage and exclusivity apply only to that prefix: $|A| + |R| = k$. A complete run has $k = n$; a budget or resolver-domain stop has $k = i$ and says nothing about the unvisited suffix. Stable append preserves relative order *within* each branch, but separating the branches loses their original interleaving. Classification error is zero relative to the declared question when every recorded certificate checks. That internal guarantee is not an empirical error rate.

Worked example 1: printed marks (computation). Take $X = M = \{\mathbf{x}, \mathbf{o}\}$, e the identity, $\sigma = \mathbf{x}$, and $L = [\mathbf{x}, \mathbf{o}, \mathbf{o}, \mathbf{x}]$. Four decisions give

$$A = [\mathbf{o}, \mathbf{o}], \quad R = [\mathbf{x}, \mathbf{x}], \quad |A| + |R| = 4.$$

With $N \geq 4$, the stopping certificate is COMPLETE with $i = 4 = |L|$. With $N = 2$, it instead certifies only the prefix $[\mathbf{x}, \mathbf{o}]$, one entry in each slice, and stop reason BUDGET; it asserts nothing about the two remaining marks. Exchanging the two printed shapes everywhere preserves slice sizes while swapping their branch names. No metric follows: matching shapes does not define a distance between them.

Worked example 2: an integer comparator (computation). Let $M = \{0, \dots, 9\}$, $\sigma = 7$, and process $L = [7, 8, 2, 7, 9]$. The result is

$$A = [8, 2, 9], \quad R = [7, 7].$$

The separated slices lose the original interleaving between branches and discard magnitude, while stable append preserves relative order within each branch. Values 8 and 2 take the same branch although a separately supplied arithmetic map gives $|8-7| = 1$ and $|2-7| = 5$. The name difference does not supply subtraction. To obtain it, the representation must add $d : M \times M \rightarrow \mathbb{N}$ and publish its cost and assumptions.

Worked example 3: a finite packet alphabet (communication model). Declare a discrete source on $M = \{\mathbf{A}, \mathbf{B}, \mathbf{C}\}$ with

$$p(\mathbf{A}) = \frac{1}{2}, \quad p(\mathbf{B}) = p(\mathbf{C}) = \frac{1}{4}.$$

For $p(x) > 0$, Shannon self-information is

$$I(x) = -\log_2 p(x),$$

so the three marks carry one, two, and two bits respectively [9]. With $\sigma = \mathbf{A}$, both $\mathbf{B}$ and $\mathbf{C}$ enter the same slice. The erased decision therefore cannot recover the source mark. Shannon's channel language belongs here to a separately declared computational model, not to a physical signal.

Worked example 4: a sampled switch (physics and interpretation). Suppose an experiment samples a switch at $t_k = k\Delta t$ and records $v_k \in \{0, 1\}$. Physics owns the switch, sampling interval, noise, and acquisition apparatus. Post-processing may represent a record by $e(x_k) = v_k$ and pull back the question D_0 . If the apparatus has false-positive probability ε_+ , false-negative

probability ε_- , and on-state prior π , its empirical error is

$$P_e = (1 - \pi)\varepsilon_+ + \pi\varepsilon_-.$$

This belongs to the physical route from switch state to recorded mark, not to the internal comparator. The routes meet only through an explicit comparison map and observations; shared names prove no identity.

Worked example 5: convergence demonstrated (the instrument calibration). Here *convergence* means agreement of two completed finite numerical routes at a declared resolution, not an asymptotic claim. Both routes begin from the same calibration data

$$C = 18, \quad \text{target} = 5, \quad d_* = \sqrt{C/\text{target}} = \sqrt{18/5}.$$

The direct route $1 \rightarrow 3$ is a one-pass forward evaluation at d_* . In the declared scaled-integer representation it reports

$$d = 137011290548979455469 \quad \text{at scale } 10^{18}.$$

The stepped route $1 \rightarrow 2 \rightarrow 3$ carries the residue through intermediate quasi-Newton updates. Its finite trace is

count	r_k	reported inverse value
0	2/1	129.6
1	17/9	137.7
2	≈ 1.897309	≈ 137.016
3	≈ 1.897366	$s/10^{18}$

and it stops at count three with

$$s = 137011290753751157155 \quad \text{at scale } 10^{18}.$$

These are distinct instrument algorithms. The stepped route has mediant/continued-fraction, Newton/BFGS, and residue-carrying secant realizations; these are numerical constructions whose update and stopping rules must be declared separately. Both endpoint computations are covariant forward constructions over the shared calibration data. The contravariant action enters when the comparison question is pulled back to those data; it is not a second name for either endpoint algorithm.

The finite comparator certificate is not integer equality. It records

$$|d-s| = 204771701686, \quad \frac{|d-s|}{10^{18}} = 0.000000204771701686 \approx 2.04771701686 \times 10^{-7}.$$

Thus the reported decimal values agree to nine digits. To state the reading precisely, let X be the shared calibration-input space, D the scaled-integer report space, and

$$F = (F_{\text{direct}}, F_{\text{stepped}}) : X \rightarrow D \times D.$$

At count-three resolution define the comparison question

$$q_3(a, b) := \left[\frac{|a-b|}{10^{18}} < 8.1 \right], \quad F^*q_3 = q_3 \circ F.$$

The pullback F^*q_3 is the contravariant reading of the two covariantly constructed reports. Write $a \equiv_3 b$ when this declared-resolution question answers yes. For the shared input above, $d \equiv_3 s$. The house slogan $1 \rightarrow 3 = 1 \rightarrow 2 \rightarrow 3$ abbreviates this relation; it is not integer equality and establishes path-independence only for these two routes on this calibration input.

The two-pass calibration therefore licenses a path-independent shared numerical result at the declared resolution, which is what earns the name α_c . It does not identify the methods, meanings, traces, costs, or physics/computation provenance of the routes. Nor does it promote either scaled integer to the owned output. The honest result remains the ordered walls written in the

repository's house notation $[129.6, 137.7]$, not a point. The word *open* is the repository's metaphor for an unresolved jar, not topological interval notation; the displayed bracket records the two owned ordered walls. The values near 137.011290 are finer reports obtained past the floor.

The roles of the two passes are now exact. Pass one directly certifies $C = 18$ and $\text{target} = 5$, then infers the crossing $\sqrt{C/\text{target}}$; the crossing is not claimed as an independent measurement. Pass two traverses the stepped trace and compares its endpoint with the direct endpoint. Any comparison with a physical quantity requires a separate provenance chain and empirical observations.

Conditional Richardson audit. The direct and stepped reports are *not* a valid Richardson pair: they come from different methods, and no common discretization family $A(h)$, refinement ratio r , or error order p has been established. Only under the counterfactual assumptions $D = A(h)$, $S = A(h/2)$, and leading error proportional to h^p would the extrapolate be

$$A_R = S + \frac{S - D}{2^p - 1}.$$

Applying that formula conditionally gives the sensitivity ledger

p	A_R	correction added to S
1	137.011290958522858841	$2.04771701686 \times 10^{-7}$
2	137.011290822008391050333 ...	$6.8257233895333 \times 10^{-8}$
4	137.011290767402603934066 ...	$1.3651446779067 \times 10^{-8}$

Treating the stepped report as the finer value is itself arbitrary and counterfactual here. Reversing the orientation reverses the correction, and no common refinement family selects either orientation. These are sensitivity values, not measurements, uncertainty bounds, or owned results. Their variation demonstrates dependence on an unestablished order assumption. Li-

censing Richardson extrapolation would require one declared family $A(h)$, a declared ratio r , at least three refinement levels, an observed-order estimate

$$p_{\text{obs}} = \frac{\log(|A_h - A_{h/r}|/|A_{h/r} - A_{h/r^2}|)}{\log r},$$

and stability under further refinement. Until those conditions are met, the ordered-wall jar [129.6, 137.7] remains the owned result.

Two finite consequences and losses are immediate. If $|M| > 2$, the erased map $E \circ \Delta(-, \sigma) : M \rightarrow \{0, 1\}$ cannot be injective, so a binary slice cannot recover every mark. Also, σ does not determine the question: equality, order, membership, and threshold rules can share a reference while producing different slices. A report must publish (σ, D) , the representation e , update rule, stopping rule, cost, and certificate.

Symmetry supplies only

$$D_\sigma(s) \iff D_s(\sigma).$$

It preserves a relation under exchange of arguments; it proves no equality of procedures, traces, costs, or physical protocols. A two-pass calibration may test those stronger claims, but it must record both passes and state its comparison map.

The result is finite and reusable. Marks support explicit questions; terminating decisions carry evidence; evidence may be erased to bits; and pulled-back questions can slice finite records. Physics enters only when an experiment supplies an apparatus, units, uncertainty, and observations. A resonant name belongs to one provenance chain at a time. If computation and physics later report the same number or process, each route must be shown separately before comparison.

1.2 The symbol and the receiver

A receiver is not a second object placed beside a symbol. It is a declared action that projects a finite alphabet into the distinctions a reading can retain. Fix a mark domain M , a reference $\sigma \in M$, and an answer alphabet $B = \{0, 1\}$. The read grammar is

$$\begin{aligned} \textit{projection} &::= \pi_\sigma : M \rightarrow B, \\ \textit{accepted} &::= \text{yes}(m, w), \\ \textit{rejected} &::= \text{no}(m, r), \end{aligned}$$

where w certifies $m \neq \sigma$ and r certifies its negation. The projection acts by

$$\pi_\sigma(m) = \begin{cases} 1, & m \neq \sigma, \\ 0, & m = \sigma. \end{cases}$$

This is one declared receiver grammar. Another grammar may replace its question and resolver; all slices below concern the displayed inequality rule. Its zero fiber is $N_\sigma = \pi_\sigma^{-1}(\{0\})$. The accepted-bit view suppresses distinctions among marks in that fiber by mapping them all to zero, while the rejected evidence ledger retains the visited occurrences. More generally, information may be lost in either many-to-one fiber of π_σ . This is a finite projection, not a claim about a linear kernel.

Direction remains part of the grammar. A forward representation $e : X \rightarrow M$ carries records into marks, and $\pi_\sigma : M \rightarrow B$ carries marks forward into bits. Let $q : B \rightarrow \mathbf{Answer}$ be a question asked of a projected bit. Reading pulls q back:

$$\pi_\sigma^* q := q \circ \pi_\sigma, \quad e^*(\pi_\sigma^* q) = q \circ \pi_\sigma \circ e.$$

Equivalently,

$$(\pi_\sigma \circ e)^* q = e^*(\pi_\sigma^* q).$$

The order of reading reverses the order of forward maps. The receiver there-

fore consists of a finite forward projection followed by contravariant reading of the projected value; the projection itself is not contravariant.

The numerical method is a budgeted stable partition. Given $L = [x_0, \dots, x_{n-1}]$, a budget N , representation e , reference σ , and resolver Δ , initialize $A \leftarrow []$, $R \leftarrow []$, $i \leftarrow 0$. While $i < n$ and $i < N$:

1. $m_i \leftarrow e(x_i)$,
2. $a_i \leftarrow \Delta(m_i, \sigma)$,
3. append (x_i, m_i, a_i) to A for yes, otherwise to R ,
4. $i \leftarrow i + 1$.

The run stops with COMPLETE at $i = n$, BUDGET at $i = N < n$, or RESOLVER-DOMAIN when the declared decision does not cover the encountered pair. Its certificate contains the visited prefix, accepted and rejected evidence, erased projection bits, and stop reason. On the visited prefix,

$$|A| + |R| = i, \quad A \cap R = \emptyset.$$

Stable append preserves relative order within each slice but loses the interleaving between slices. A partial certificate says nothing about the unvisited suffix.

The audit schema is

$$\mathcal{Q} = (X, M, e, \sigma, \pi_\sigma, \Delta, N, A, R, J, F, P).$$

Here J records invariants, F records failure and information loss, and P records provenance. The schema is external scientific bookkeeping. The computational route owns finite marks, decisions, projection, traversal, and slices. Physics owns apparatus variables, units, noise, and observations. Interpretation owns any proposed bridge.

Worked example 1: repeated finite marks (computation). Let $M = \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$, $\sigma = \mathbf{a}$, and

$$L = [\mathbf{a}, \mathbf{c}, \mathbf{b}, \mathbf{a}, \mathbf{c}].$$

With $N \geq 5$, the complete run returns

$$A = [\mathbf{c}, \mathbf{b}, \mathbf{c}], \quad R = [\mathbf{a}, \mathbf{a}].$$

The projected trace is $[0, 1, 1, 0, 1]$. It cannot distinguish $\mathbf{b}$ from $\mathbf{c}$, although the evidence-bearing accepted slice can retain their marks. If only the projected trace is stored, the original input order has several compatible preimages. In this example the loss occurs in the accepted fiber $\{\mathbf{b}, \mathbf{c}\}$, not in the zero fiber $\{\mathbf{a}\}$; either fiber can lose distinctions when it has more than one member.

Worked example 2: budgeted packet reading (computation). Take packet marks $M = \{\text{idle}, \text{data}, \text{check}\}$, reference $\sigma = \text{idle}$, six records, and $N = 4$. Suppose their first four marks are

$$[\text{idle}, \text{data}, \text{check}, \text{data}].$$

The run stops for budget with one rejected and three accepted records. Its coverage equation is $1 + 3 = 4$, not six. Neither remaining packet may be classified from this certificate. If each decision costs c_i , the certified work is $\sum_{i=0}^3 c_i$; a claim about six decisions would exceed the ledger.

Worked example 3: Shannon loss under projection (communication model). Let a discrete source satisfy

$$p(\mathbf{a}) = \frac{1}{2}, \quad p(\mathbf{b}) = p(\mathbf{c}) = \frac{1}{4}.$$

For $p(m) > 0$, self-information is $I(m) = -\log_2 p(m)$ [9]. The source entropy is

$$H(M) = -\sum_m p(m) \log_2 p(m) = \frac{3}{2} \text{ bits.}$$

Projection against $\sigma = \mathbf{a}$ yields an equiprobable bit, hence $H(B) = 1$ bit. The lost $\frac{1}{2}$ bit is the average distinction between $\mathbf{b}$ and $\mathbf{c}$ erased by the projection. This calculation requires the displayed discrete distribution; the receiver grammar alone supplies no probabilities.

Worked example 4: threshold readout (physics, then interpretation). Suppose a finite experiment records integer converter readings $z_k \in \{0, \dots, 1023\}$. A declared threshold τ gives

$$\pi_\tau(z) = \mathbf{1}\{z > \tau\}.$$

Physics owns the sensor, converter, units relating the converter reading to the measured quantity, calibration of τ , and observed false-positive and false-negative frequencies. Computation owns the finite comparison and resulting slices. Reading the projection as a detector action is licensed only by the experimental protocol. Equal output bits do not identify two sensor states, and a perfect internal comparison does not imply zero empirical error.

The receiver therefore earns a precise but limited name. It is a finite projection followed by contravariant reading of the projected value, not a physical detector. Its accepted and rejected slices certify a finite partition of the visited prefix. Its many-to-one fibers state exactly what the erased bit cannot recover. Any later claim that a symbol has physical meaning must supply a separate apparatus route and comparison map; resemblance of names is not a calibration.

1.3 From a decision to a record

A durable record begins with a receipt, not with a familiar arithmetic interpretation. The neutral grammar is

$$\begin{aligned}
 \textit{numeral-record} &::= \text{anchor}(\textit{receipt}) \mid \text{numeral-step}(\textit{receipt}, \textit{numeral-record}), \\
 \textit{tally} &::= \text{halt}(\textit{receipt}) \mid \text{tally-step}(\textit{receipt}, \textit{numeral-record}, \textit{tally}), \\
 \textit{piece} &::= \text{origin}(\textit{receipt}) \mid \text{piece-step}(\textit{receipt}, \textit{tally}, \textit{piece}), \\
 \textit{record} &::= \text{end}(\textit{receipt}) \mid \text{record-step}(\textit{receipt}, \textit{piece}, \textit{record}).
 \end{aligned}$$

The numeral record is the earlier finite recursive payload, given here only by its anchor and extension forms. Its arithmetic meaning is reserved rather than supplied by the name. The grammar posits finite forms and nothing else. A receipt accompanies every form, and each recursive argument stores the remainder. No magnitude, chronology, or physical event is supplied by these productions.

Forward construction prepends one step:

$$\begin{aligned}
 T' &= \text{tally-step}(\rho, \nu, T), \\
 P' &= \text{piece-step}(\rho, T, P), \\
 S' &= \text{record-step}(\rho, P, S).
 \end{aligned}$$

Repeated prepending places the newest addition at the head. Tail traversal proceeds by matching the head, recording its receipt and payload, and continuing with the recursive remainder. Given budget N , initialize the visit count $j \leftarrow 1$. Continue while a step remains and $j \leq N$; stop with END, BUDGET, or RESOLVER-DOMAIN. The certificate contains the visited prefix in traversal orientation, the final unvisited tail when present, all encountered receipts, and the stop reason.

Traversal indices are analysis-ledger entries, not members of the grammar. The ledger assigns $j = 1, 2, \dots$ as a run visits heads. A later run may assign the same indices only when it declares the same starting form and traversal

orientation. Prepend construction and tail traversal point in opposite list orientations: construction adds newest first; complete traversal reads newest to oldest. Arrival orientation is recovered only by an explicit reversal.

Three finite slices are available after traversal. A shape slice separates terminal forms from step forms. An index slice selects ledger positions satisfying a declared question, such as j odd. Given a declared comparison receipt ρ_* , a receipt slice selects one value of the derived relation $\text{parity}(\rho, \rho_*)$. Each slice must retain evidence, visited occurrences, and orientation. Stable append preserves relative traversal order within each output but loses interleaving between outputs. For v visited occurrences,

$$|A| + |R| = v, \quad A \cap R = \emptyset.$$

A budget stop certifies nothing about the unvisited tail.

Worked example 1: prepend and budgeted traversal (computation). Let $E = \text{end}(\rho_e)$, and prepend three record steps carrying pieces P_1, P_2, P_3 in that arrival order:

$$S_1 = \text{record-step}(\rho_1, P_1, E), \quad S_2 = \text{record-step}(\rho_2, P_2, S_1), \quad S_3 = \text{record-step}(\rho_3, P_3, S_2).$$

Complete tail traversal reports P_3, P_2, P_1 , followed by the terminal receipt. With $N = 2$, it certifies only ledger entries $(1, \rho_3, P_3)$ and $(2, \rho_2, P_2)$, then returns the remaining tail S_1 with BUDGET. It may not report anything about P_1 without another visit.

A head projection deliberately ignores recursive tails. For step forms define

$$\pi_{\text{head}}(\text{record-step}(\rho, P, S)) = (\rho, P).$$

For a terminal form it returns its terminal receipt and a terminal tag. This is a many-to-one forward map. Its fibers contain every form with the same head summary, regardless of tail.

Worked example 2: same head, different tail (computation). Let

$$S_A = \text{record-step}(\rho, P, S_1), \quad S_B = \text{record-step}(\rho, P, S_2), \quad S_1 \neq S_2.$$

Then

$$\pi_{\text{head}}(S_A) = \pi_{\text{head}}(S_B) = (\rho, P).$$

The head read cannot reconstruct which tail was present. The pair lies in one fiber of π_{head} , and the missing distinction is exactly the tail information suppressed by projection. Retaining the original forms in an evidence ledger preserves them; retaining only the projected head does not.

The relation between two receipts also directs a finite comparison. Declare a current receipt ρ , a comparison receipt ρ_* , a derived relation $\text{parity}(\rho, \rho_*) \in \{\text{agree}, \text{disagree}\}$, current payload a , comparison payload b , and a binary question C . The action is

$$\text{compare}_{\rho, \rho_*}(a, b) = \begin{cases} C(a, b), & \text{parity}(\rho, \rho_*) = \text{agree}, \\ C(b, a), & \text{parity}(\rho, \rho_*) = \text{disagree}. \end{cases}$$

Derived agreement sends the current payload into the forward comparison; derived disagreement reverses the arguments. This rule establishes no order law. If C is asymmetric, reversal may change the answer; if C is symmetric, it may not. The relation between receipts chooses argument orientation but does not prove anything about the comparison relation.

Worked example 3: receipt-directed comparison (computation).

Let two numeral-record payloads be denoted ν_L, ν_R , and declare $C(a, b)$ to answer yes exactly for the ordered pair (ν_L, ν_R) . Then

$$\text{compare}_{\rho_1, \rho_*}(\nu_L, \nu_R) = \text{yes} \quad \text{when} \quad \text{parity}(\rho_1, \rho_*) = \text{agree}, \quad \text{compare}_{\rho_2, \rho_*}(\nu_L, \nu_R) = \text{no} \quad \text{when} \quad \text{pa}$$

The outputs differ solely because the second receipt relation reverses the arguments. Neither output assigns an arithmetic or physical meaning to ν_L, ν_R . Those resonances remain reserved.

Contravariance belongs to questions pulled through forward projections. If $q : H \rightarrow \mathbf{Answer}$ asks about a head summary and $\pi_{\text{head}} : S \rightarrow H$, then

$$\pi_{\text{head}}^* q = q \circ \pi_{\text{head}}.$$

For an index analysis, let π_{idx} project each visited ledger entry to its assigned index. An index question q_I is read as $\pi_{\text{idx}}^* q_I$. The projections move summaries forward; reading pulls questions backward. The receipt-directed argument reversal above is a separate finite action and is not identified with this pullback law.

Worked example 4: shape, index, and receipt slices (computation).

Relative to declared comparison receipt ρ_* , suppose a complete traversal yields four ledger rows whose middle column is the derived parity

index	derived receipt relation	payload
1	$\text{parity}(\rho_4, \rho_*) = \mathbf{agree}$	P_4
2	$\text{parity}(\rho_3, \rho_*) = \mathbf{disagree}$	P_3
3	$\text{parity}(\rho_2, \rho_*) = \mathbf{agree}$	P_2
4	$\text{parity}(\rho_1, \rho_*) = \mathbf{agree}$	P_1

and then a terminal row. The odd-index slice selects rows one and three. The derived-agree receipt slice selects rows one, three, and four. The shape slice separates the four step rows from the terminal row. These are three questions over one traversal, not three reconstructions of the original arrival orientation.

A computation-side resonance with Cohen may now be stated carefully.

From a completed traversal S , define the finite partial assignment

$$p_S(k) = \text{payload at traversal index } k$$

for visited indices k . Extending the budget can extend the assignment when earlier rows remain unchanged. This resembles Cohen’s finite-condition discipline: compatible finite commitments may be extended [3]. The resemblance concerns finite partial assignments only. Because traversal reads prepend-built forms newest first, $p_S(k)$ is indexed in traversal orientation, not arrival orientation. No generic extension, forcing relation, independence result, or claim about every possible continuation follows.

The action ledger closes the section. The receipt grammar and prepend rules are *posited*. A finite construction trace, visited-prefix certificate, projection equality, or checked slice is *demonstrated*. Head and index questions are *read* contravariantly through declared projections. Arithmetic meanings for numeral records and physical meanings for receipt relations, sides, or arrival events remain *reserved*. A head projection does not reconstruct a tail; a finite slice does not certify an unread remainder; argument reversal does not select which side is which; and a compatible partial assignment does not produce a generic extension.

Chapter 2

Difference Becomes Countable

2.1 Counting is an interface

Counting begins with four finite capabilities, not with a completed enumeration. A neutral grammar is

$$\begin{aligned} \textit{counting-capability} &::= \text{counting}(\textit{carrier}, \textit{tally}, U), \\ \textit{admission-capability} &::= \text{admission}(\textit{counting-capability}, A), \\ \textit{indexing-capability} &::= \text{indexing}(\textit{counting-capability}, \textit{origin}, V), \\ \textit{bounding-capability} &::= \text{bounding}(\textit{indexing-capability}, B). \end{aligned}$$

Here U updates a tally-shaped record, A asks an admission question, V updates an indexed record from a declared origin, and B asks a bound question. The capabilities provide places for finite data and questions. They do not guarantee that repeated updates are distinct, that every candidate appears, or that an index assignment is one-to-one or onto.

Use the neutral forms

$$\text{halt}(\rho), \quad \text{step}(\rho, \nu, T),$$

with receipt ρ , numeral record ν , and recursive tail T . Arithmetic meaning

for ν remains reserved. The default counting update has a head-update/tail-reset shape:

$$\begin{aligned} U(\text{halt}(\rho)) &= \text{halt}(\rho_*), \\ U(\text{step}(\rho, \nu, T)) &= \text{step}(\rho_*, \nu_*, T_*), \end{aligned}$$

where ρ_*, ν_*, T_* are the current receipt, carrier payload, and stored tally supplied by the capability. The input tail does not survive the second branch. The default indexing update likewise uses a fixed origin O :

$$\begin{aligned} V(\text{halt}(\rho)) &= \text{step}(\rho, c, O), \\ V(\text{step}(\rho, \nu, T)) &= \text{step}(\rho, U(\nu), O). \end{aligned}$$

It retains the input head receipt, updates the head payload, and resets the tail to O .

Worked example 1: same head, different tail (computation). Let

$$R_1 = \text{step}(\rho, \nu, T_1), \quad R_2 = \text{step}(\rho, \nu, T_2), \quad T_1 \neq T_2.$$

The default recurrence gives

$$V(R_1) = V(R_2) = \text{step}(\rho, U(\nu), O).$$

Thus the update is many-to-one: records with the same head and different tails share an output fiber. Iteration cannot reconstruct the discarded tail, so this recurrence alone cannot certify enumeration of its input records.

The default admission question is also only a question. On neutral numeral forms it has three branches:

$$A(s, t) = \begin{cases} \text{yes}, & s \text{ is an anchor,} \\ \text{no}, & s \text{ is a step and } t \text{ is an anchor,} \\ R(\rho_t) \vee Q(\nu_s, \nu_t), & s, t \text{ are steps.} \end{cases}$$

Here R is the declared question carried by the threshold receipt; an affirmative receipt answer admits the pair without consulting Q . The default bound question has a related orientation rule:

$$B(s, t) = \begin{cases} \text{yes}, & s \text{ is an anchor,} \\ \text{no}, & s \text{ is a step and } t \text{ is an anchor,} \\ [\text{parity}(\rho_s, \rho_t) = \text{agree} \wedge Q(i_s, i_t)] \\ \vee [\text{parity}(\rho_s, \rho_t) = \text{disagree} \wedge Q(i_t, i_s)], & s, t \text{ are steps.} \end{cases}$$

Agreement preserves the comparison arguments; disagreement reverses them. Neither capability supplies a resolver for A , B , or Q , and none proves reflexivity, transitivity, antisymmetry, totality, or any other order law. The questions and defaults may be replaced in another declared capability; subsequent slices then concern the replacements.

The actual iteration is the forward orbit

$$r_0 = O, \quad r_{k+1} = V(r_k).$$

For an actual finite run with transition budget N , record $r_0 = O$, then apply V for $k = 0, \dots, N - 1$. Stop with BUDGET after the N -th transition. The certificate contains the declared origin, budget N , all N transition pairs (r_k, r_{k+1}) , the receipts retained inside the recorded values, the prefix $[r_0, \dots, r_N]$ of $N + 1$ values, every declared erasure, and the stop reason. It certifies only those N transitions and says nothing about a further update.

This is a covariant construction. Given a question q on orbit values, reading is contravariant pullback:

$$V^*q = q \circ V.$$

For composable forward maps W and V ,

$$(V \circ W)^*q = W^*(V^*q).$$

Receipt-directed reversal inside B is not categorical variance; it is merely a branch that exchanges two arguments.

Worked example 2: an orbit need not enumerate (computation).

Let a permitted update satisfy $V(a) = b$ and $V(b) = a$. Starting at a , the orbit is

$$[r_0, r_1, r_2, r_3, r_4, r_5, r_6] = [a, b, a, b, a, b, a] \quad (N = 6).$$

A six-transition run has seven recorded values but only two distinct values. It demonstrates iteration and a two-cycle, not seven distinct indexed objects. Therefore an update interface, even with a budget and origin, is not an enumeration certificate.

A separate finite indexing experiment can be posited, but it must be named as external. Given $L = [x_1, \dots, x_n]$, capacity K , budget N , and an explicitly declared resolver Δ_A for the chosen admission question, visit while $j \leq n, K, N$. Resolve $\Delta_A(x_j)$, append admitted rows to a ledger and rejected rows to a complement slice, then increment j . Stop with COMPLETE, CAPACITY, BUDGET, or RESOLVER-DOMAIN. Its certificate contains the visited prefix, both slices, assigned and unused indices, unvisited suffix, and stop reason. For v visits,

$$|\text{admitted}| + |\text{rejected}| = v.$$

This generic $L/K/N$ traversal, its assignment ι , and its resolver are *posited* analysis machinery; they are not contained in the four capability names.

A genuine finite enumeration claim needs an encoder and decoder. For

finite sets X and I , require both round trips:

$$\text{dec}(\text{enc}(x)) = x \quad (x \in X), \quad \text{enc}(\text{dec}(i)) = i \quad (i \in I).$$

Worked example 3: good and bad round trips (computation). For $X = \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}$ and $I = \{1, 2, 3\}$, a good pair is

x	$\mathbf{a}$	$\mathbf{b}$	$\mathbf{c}$	i	1	2	3
$\text{enc}(x)$	1	2	3	$\text{dec}(i)$	$\mathbf{a}$	$\mathbf{b}$	$\mathbf{c}$.

Both round trips return their inputs. A bad assignment is

x	$\mathbf{a}$	$\mathbf{b}$	$\mathbf{c}$
$\text{enc}_{\text{bad}}(x)$	1	1	2

because the collision at 1 forces any decoder to choose at most one of $\mathbf{a}, \mathbf{b}$. Index 3 is also unused. No decoder can satisfy both round trips, regardless of the interface name.

Shannon's codebook is another map, $C : I \rightarrow \mathcal{W}$, not the index assignment. With a declared discrete distribution $p(i) > 0$,

$$I(i) = -\log_2 p(i), \quad \bar{\ell} = \sum_i p(i) \ell(C(i)).$$

These require probabilities and word lengths [9]. Unambiguous decoding requires a declared uniquely decodable or prefix condition.

Worked example 4: an index is not its word (communication model).

Let $p(1) = 1/2$, $p(2) = p(3) = 1/4$, with $C(1) = 0$, $C(2) = 10$, and $C(3) = 11$. The words are prefix-free, and direct calculation gives entropy $3/2$ bits and expected length $3/2$ bits. This equality is demonstrated for this distribution and codebook only. It identifies neither an index with its word nor a capability orbit with an encoded message.

Worked example 5: counted threshold crossings (physics, then interpretation). Suppose an apparatus yields a finite acquisition record of threshold crossings. Physics owns the sensor, threshold, dead time, sampling interval, missed-event rate, and false-event rate. The external traversal may index accepted records. Its certificate counts those accepted records, not unobserved physical events. Interpreting the ledger as an event count remains reserved until the experimental error model is supplied.

The action ledger now separates the claims. The four capabilities, their defaults, and the external $L/K/N$ experiment are *posited*. A finite orbit trace, fiber equality, cycle, slice, or pair of checked round trips is *demonstrated*. Questions on orbit values or indices are *read* contravariantly through forward maps. Full enumeration, bijection, cardinality, and physical event count remain *reserved*. Counting is an interface for finite updates and questions, not a guarantee hidden in a name.

2.2 The update and the alleged endpoint

The local capability stores an indexing capability, a named endpoint record, a current chain, and a chain update. The step capability stores that local capability, a current piece, a named accumulation, and a second update. A neutral grammar is

$$\begin{aligned}
 \text{chain} &::= \text{end}(\text{receipt}) \mid \text{entry}(\text{receipt}, \text{piece}, \text{chain}), \\
 \text{accumulation} &::= \text{empty}(\text{receipt}) \mid \text{indexed}(\text{receipt}, \text{chain}, \text{accumulation}), \\
 \text{local-process} &::= \text{local}(\text{indexing}, \text{named-endpoint}, \text{current-chain}, W), \\
 \text{step-process} &::= \text{step}(L_{\text{self}}, \text{current-piece}, \text{named-accumulation}, U).
 \end{aligned}$$

These capabilities posit records and updates. The words endpoint and accumulation attach names; they supply no metric, error bound, or statement about an infinite tail.

Let L_{ext} be a separately supplied local process, with stored chain S_{ext} and

update W_{ext} . Let A_* be the stored accumulation, O_ρ the origin piece made from receipt ρ , and V the indexing update on pieces. The external local update is

$$\begin{aligned} W_{\text{ext}}(\text{end}(\rho)) &= \text{entry}(\rho, O_\rho, S_{\text{ext}}), \\ W_{\text{ext}}(\text{entry}(\rho, p, T)) &= \text{entry}(\rho, V(p), S_{\text{ext}}). \end{aligned}$$

Both branches reset the output tail to S_{ext} ; the input tail T is discarded. The default step update is

$$\begin{aligned} U(\text{empty}(\rho)) &= \text{indexed}(\rho, S_{\text{ext}}, A_*), \\ U(\text{indexed}(\rho, S, T)) &= \text{indexed}(\rho, W_{\text{ext}}(S), A_*). \end{aligned}$$

Both branches reset the outer tail to A_* . This update also ignores the stored current piece in the step capability and the stored L_{self} ; it uses the chain and update from external dependency L_{ext} . A name suggesting approximation cannot restore any discarded tail or make either unused record operative.

Worked example 1: equal head, unequal tails (default recurrence).

Let

$$C_1 = \text{indexed}(\rho, S, T_1), \quad C_2 = \text{indexed}(\rho, S, T_2), \quad T_1 \neq T_2.$$

Then

$$U(C_1) = U(C_2) = \text{indexed}(\rho, W_{\text{ext}}(S), A_*).$$

The two inputs occupy one output fiber. This tail-reset equality is demonstrated directly from the recurrence. It does not establish that either input approaches the named accumulation.

For a finite run, choose origin x_0 and budget N , then compute

$$x_{k+1} = U(x_k), \quad k = 0, \dots, N-1.$$

The budget certificate contains x_0 , N , all transition pairs, the $N+1$ recorded

values, receipts retained in them, declared erasures, and BUDGET. It certifies only those transitions. A slice may ask any declared question q of the recorded values. Construction is covariant along U ; reading is contravariant:

$$U^*q = q \circ U, \quad (U \circ Z)^*q = Z^*(U^*q).$$

Stable accepted and rejected slices preserve within-slice order and lose cross-slice interleaving unless ledger positions are retained.

A Cauchy assertion requires independent mathematical data. On a declared metric space (X, d) , it is

$$\forall \varepsilon > 0 \exists M \forall m, n \geq M, \quad d(x_m, x_n) < \varepsilon.$$

A modulus gives a rule $M(\varepsilon)$. The local and step capabilities contain no metric or modulus, and their default recurrences do not establish this quantified statement.

Worked example 2: fixed update (external metric experiment).

Outside the capability grammar, declare $X = \{3\}$, $U(3) = 3$, and the discrete metric. With $N = 4$, the finite trace is

$$[3, 3, 3, 3, 3].$$

All four transition pairs check. The separately declared rule and metric also justify the Cauchy assertion, because all pairwise distances vanish. The justification comes from those external premises, not from a named endpoint.

Worked example 3: alternating update (external metric experiment). Declare $X = \{-1, 1\}$, $U(-1) = 1$, $U(1) = -1$, ordinary distance, and $N = 6$. The exact trace is

$$[-1, 1, -1, 1, -1, 1, -1].$$

For $\varepsilon = 1$, values arbitrarily far along the infinite-tail rule remain distance 2 apart. This supplies a counterexample to the Cauchy assertion while preserving a valid finite update trace.

Worked example 4: first tolerance hit (external numerical experiment). Declare $x_0 = 1$, $U(x) = x/2$, alleged endpoint $a = 0$, ordinary distance, and $\varepsilon = 1/8$. Four transitions give

$$[1, \frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{16}].$$

The question $q_\varepsilon(x) = [d(x, a) \leq \varepsilon]$ first answers yes at ledger position three, producing

$$R_\varepsilon = [1, \frac{1}{2}, \frac{1}{4}], \quad A_\varepsilon = [\frac{1}{8}, \frac{1}{16}].$$

This demonstrates a first hit in the finite trace. Persistence requires an additional invariant; the stopping question alone does not certify the unread infinite tail.

Finite descriptions provide a separate computation-side resonance. Fix a declared prefix-free machine M . Its description complexity is

$$K_M(x) = \min\{|p| : M(p) = x\},$$

when at least one halting description p produces x . The value depends on the chosen machine and encoding. Chaitin's results connect such measures with algorithmic complexity and irreducibility [2]; they do not turn every short update into an uncomputability result.

Worked example 5: description budget and displayed storage (external accounting model). Assume a fixed-width display in which the update description occupies h symbols and each recorded value occupies ex-

actly b symbols. Displaying an N -transition trace then uses

$$h + (N + 1)b$$

symbols before receipts and formatting overhead. This is an accounting formula under the fixed-width assumptions, not a universal lower bound and not a value of K_M . A compact update may coexist with a long displayed trace.

The action ledger is exact. The local and step capabilities, their stored records, and the default W, U recurrences are *posited*. The two tail resets, ignored current piece, ignored stored local process, use of the external dependency, finite traces, output-fiber equality, and checked slices are *demonstrated*. Questions are *read* contravariantly through forward updates. A Cauchy property, modulus, persistent enclosure, mathematical endpoint, physical steady state, or algorithmic-complexity claim remains *reserved* until its additional premises are supplied. The capabilities update finite records; their names do not certify an infinite tail.

2.3 A finite trace is not an infinite object

A finite run and a request without a preset bound are different grammatical forms:

$$\begin{aligned} \textit{finite-run} &::= \text{run}(U, x_0, N), \\ \textit{open-request} &::= \text{continue}(U, x_0, \textit{stop-question}), \\ \textit{trace} &::= [x_0, x_1, \dots, x_k], \quad x_{j+1} = U(x_j). \end{aligned}$$

The first supplies a transition budget N . The second supplies a stopping question but no advance claim that the question will ever answer yes. Neither grammar stores an infinite object. Words such as iterative, infinite, or unbounded can name capabilities or intentions; they do not establish termi-

nation, nontermination, distinctness, or coverage.

For a budgeted run, apply U for $j = 0, \dots, N-1$. The certificate contains the origin, budget, all transition pairs, the $N+1$ recorded values, retained receipts, declared erasures, and stop reason. It certifies exactly the visited prefix. Construction carries states forward through U . A question q about a state is read contravariantly:

$$U^*q = q \circ U, \quad (U \circ V)^*q = V^*(U^*q).$$

A slice may separate first occurrences from repeated occurrences, accepted from rejected states, or positions before and after a detected collision. Stable slicing preserves within-slice traversal order and loses interleaving unless indices remain attached.

Pigeonhole reasoning must be scoped. If a finite set X has m states, any list of $m+1$ states contains a collision. That statement alone proves only

$$\exists i < j \leq m, \quad x_i = x_j.$$

It says nothing about how the list was generated. To infer an eventual periodic orbit, one additionally needs a single deterministic, time-independent update satisfying $x_{k+1} = U(x_k)$. Then equality at $i < j$ implies by induction

$$x_{i+t} = x_{j+t} \quad (t \geq 0),$$

so the orbit is periodic from i with candidate period $\lambda = j - i$. Calling i the least preperiod μ , or λ the least period, additionally requires first-repeat and minimality checks. Collision plus transition compatibility earns a periodic tail; collision alone does not.

Worked example 1: collision without dynamics (computation). Take the finite list

$$[a, b, a, c].$$

Pigeonhole identifies the repeated a . No update rule is declared, so there is no licensed next state after either occurrence and no cycle certificate. Calling the list an orbit would add a premise not present in the data.

Worked example 1b: projected collision is not state collision (computation). Let $S = \{a, b, c\}$, with

$$U(a) = b, \quad U(b) = c, \quad U(c) = c,$$

and observation projection

$$\pi(a) = 0, \quad \pi(b) = 0, \quad \pi(c) = 1.$$

The first two observations collide even though $a \neq b$; their successors also project differently. Thus a projection collision proves neither state equality nor a cycle, and it does not select which underlying states should be identified. For m observed values in an alphabet of size $r < m$, finite pigeonhole reasoning proves only

$$\exists i < j < m, \quad \pi(x_i) = \pi(x_j).$$

The projection carries observations forward. A question q on the readout alphabet is read contravariantly:

$$\pi^* q = q \circ \pi.$$

Forward observation and pulled-back reading do not upgrade equality of readouts to equality of states.

A visited-map detector makes the missing premise explicit. Initialize a finite map $H[x_0] = 0$. For $j = 1, \dots, N$, compute $x_j = U(x_{j-1})$. If $x_j \notin H$, store $H[x_j] = j$. If $x_j \in H$, return

$$\mu = H[x_j], \quad \lambda = j - \mu,$$

together with the two equal states and all transitions between them. Stop with CYCLE, BUDGET, or RESOLVER-DOMAIN. This requires decidable state equality and a declared deterministic update.

Worked example 2: visited-map cycle certificate (computation).

Let U act on $\{0, 1, 2, 3, 4\}$ by

$$U(0) = 1, \quad U(1) = 2, \quad U(2) = 3, \quad U(3) = 2, \quad U(4) = 4.$$

From $x_0 = 0$, the detector records

$$[0, 1, 2, 3, 2]$$

and stops at $j = 4$, because state 2 first appeared at index 2. It returns

$$\mu = 2, \quad \lambda = 2.$$

The transition pairs $2 \mapsto 3 \mapsto 2$, together with determinism, certify the repeating tail $2, 3, 2, 3, \dots$. State 4 is irrelevant to this orbit; a cycle certificate is not coverage of the state set.

Floyd's detector trades stored history for additional updates. Set the tortoise to $U(x_0)$ and the hare to $U(U(x_0))$. Repeatedly update the tortoise once and the hare twice until they meet or a transition budget is exhausted. A meeting is a collision under the same deterministic update. Separate finite passes then recover μ and λ . Its certificate must record every evaluated transition, the meeting state, budgets for all passes, and the recovered cy-

cle parameters. Without decidable equality or total updates on the visited states, the method has no licensed comparison.

Worked example 3: Floyd detection on a finite orbit (computation).

For the update in the previous example and origin 0,

round	0	1
tortoise	1	2
hare	2	2

so the first meeting is at round one. A subsequent entry-point pass recovers $\mu = 2$, and a lap from state 2 recovers $\lambda = 2$. The arithmetic agrees with the visited-map result, but the algorithms have different traces and storage costs. Agreement of (μ, λ) does not identify the methods.

Worked example 4: budget exhaustion is inconclusive (computation). Let $U(n) = n + 1$ on nonnegative integer records, origin 0, and $N = 5$. The certificate is

$$[0, 1, 2, 3, 4, 5], \quad \text{BUDGET.}$$

No collision appears in the prefix. This proves only six distinct visited values. It proves neither that a later collision occurs nor that the run continues forever. The open request and the finite certificate must remain separate.

Turing’s halting result concerns a precisely encoded universal decision problem: no single algorithm decides for every encoded machine and input whether that machine halts [10]. It does not imply that a particular finite run lacks a termination certificate, and a budget stop is not evidence of undecidability. Conversely, a local cycle certificate settles the behavior of that declared deterministic orbit without solving the universal problem.

Shannon supplies a probability fence. Information calculations require a declared finite alphabet and readout map; empirical frequencies in one finite

trace are estimates, not automatically source probabilities. Entropy needs a probability distribution, while capacity additionally needs a channel law and an optimization domain [9]. Repetition introduced by an encoding or deterministic update is not, by itself, a stochastic process.

Chaitin supplies a second computation-side fence. Fix a declared prefix-free machine M and define

$$K_M(x) = \min\{|p| : M(p) = x\}.$$

The value depends on M up to the usual machine-dependent additive constants, and claims about algorithmic randomness require an infinite bit specification and the relevant complexity bounds [2]. A finite trace, a repeated state, or a capability labeled infinite supplies none of those assumptions. No uncomputability or randomness conclusion follows from naming alone.

Worked example 5: finite trace storage (external accounting model).

Under a fixed-width display with b symbols per state and h symbols for the update description, an N -transition trace occupies

$$h + (N + 1)b$$

symbols before receipts, indices, and formatting. This is a conditional storage calculation, not a universal lower bound and not K_M . A cycle certificate may compress future description to (μ, λ) plus one period, but that compression is licensed only after the cycle has been demonstrated.

The action ledger closes the chapter. The update, origin, equality question, budgets, and stopping policies are *posited*. Finite traces, slices, collisions, transition compatibility, and checked cycle parameters are *demonstrated*. Questions are *read* contravariantly through covariant forward updates. An infinite object, universal termination decision, algorithmic randomness claim, or physical periodic process remains *reserved* until its ad-

ditional premises and provenance are supplied. A finite trace may certify a cycle; it never becomes an infinite object merely by being given an unbounded name.

Chapter 3

The Residue Refuses To Disappear

3.1 The branch that cannot be silently discarded

The residue capability consists of a step process and a replaceable representative question $\mathcal{R}(a, b)$. For neutral accumulation forms $\text{empty}(\rho)$ and $\text{indexed}(\rho, S, L)$, its default branch table is

a	b	$\mathcal{R}(a, b)$
$\text{empty}(\rho_1)$	$\text{empty}(\rho_2)$	$\rho_1 = \rho_2$
$\text{empty}(\rho_1)$	$\text{indexed}(\rho_2, S_2, L_2)$	yes
$\text{indexed}(\rho_1, S_1, L_1)$	$\text{empty}(\rho_2)$	no
$\text{indexed}(\rho_1, S_1, L_1)$	$\text{indexed}(\rho_2, S_2, L_2)$	$[\rho_1 = \rho_2] \wedge Q_{\leq}(S_1, S_2) \wedge Q_{<}(L_1, L_2)$.

The capability supplies no resolver for this question and proves no reflexivity, symmetry, transitivity, or equivalence law. Another declaration may replace $\mathcal{R}$; every subsequent read must then name the replacement.

A history-bearing sample has the grammar

$$sample ::= \text{initial}(\rho, L) \mid \text{response}(\rho_s, S, \rho_r, R, H).$$

The response stores a first receipt and stimulus, a second receipt and response, and prior history H . Its default comparison question $\mathcal{C}(x, y)$ has four branches:

x	y	$\mathcal{C}(x, y)$
$\text{initial}(\rho_1, L_1)$	$\text{initial}(\rho_2, L_2)$	$[\rho_1 \sim \rho_2 \wedge Q_{\leq}(L_1, L_2)] \vee [\rho_1 \not\sim \rho_2 \wedge Q_{\leq}(L_1, L_2)]$
$\text{response}(\rho_1, S_1, \rho'_1, R_1, H_1)$	$\text{response}(\rho_2, S_2, \rho'_2, R_2, H_2)$	$[\rho_1 \sim \rho_2 \wedge Q_{\leq}(R_1, R_2)] \vee [\rho_1 \not\sim \rho_2 \wedge Q_{\leq}(S_1, S_2)]$
$\text{initial}(\rho_1, L_1)$	$\text{response}(\rho_2, S_2, \rho'_2, R_2, H_2)$	$[\rho_1 \sim \rho_2 \wedge Q_{\leq}(L_1, S_2)] \vee [\rho_1 \not\sim \rho_2 \wedge Q_{\leq}(S_1, S_2)]$
$\text{response}(\rho_1, S_1, \rho'_1, R_1, H_1)$	$\text{initial}(\rho_2, L_2)$	no.

Here $\rho_1 \sim \rho_2$ means derived receipt agreement. In the response/response branch, agreement selects the response comparison, while disagreement selects the stimulus comparison. The second receipts ρ'_1, ρ'_2 and both history tails are ignored. Again, the table is a question, not an established order. The sample grammar supplies no resolver for $\mathcal{C}$.

Worked example 1: receipt-selected comparison (computation).

Take response samples X, Y . If their first receipts agree, the question reduces to $Q_{\leq}(R_X, R_Y)$, regardless of their stimuli, second receipts, or histories. If the first receipts disagree, it reduces to $Q_{\leq}(S_X, S_Y)$, regardless of their responses. Two samples can therefore receive the same comparison answer while carrying different ignored histories. Receipt relation selects an argument pair; it does not select which pair has physical meaning.

Forward history construction prepends responses:

$$\begin{aligned} H_0 &= \text{initial}(\rho_0, L_0), \\ H_1 &= \text{response}(\rho_1, S_1, \rho'_1, R_1, H_0), \\ H_2 &= \text{response}(\rho_2, S_2, \rho'_2, R_2, H_1), \\ H_3 &= \text{response}(\rho_3, S_3, \rho'_3, R_3, H_2). \end{aligned}$$

Traversal reads newest to oldest: H_3, H_2, H_1, H_0 .

Worked example 2: budgeted history traversal (computation). With visit budget $N = 2$, traversal records the heads of H_3 and H_2 , then stops with BUDGET and returns H_1 as the residual prior tail. Its certificate contains those two visited heads, their receipts and payloads, the untouched tail H_1 , and the stop reason. It makes no claim about the contents of H_1 or H_0 beyond retaining their finite record.

Euclidean division is an external numerical model for explicit remainder accounting. For $n \geq 0$, $d > 0$, initialize $q = 0, r = n$. While $r \geq d$ and the budget remains, update

$$q \leftarrow q + 1, \quad r \leftarrow r - d.$$

The invariant is

$$n = dq + r.$$

A complete stop requires $0 \leq r < d$; a budget stop preserves the invariant but does not certify a terminal Euclidean remainder.

Worked example 3: iterative division (external numerical experiment). For $n = 23, d = 5$, the trace is

$$(q, r) : (0, 23) \rightarrow (1, 18) \rightarrow (2, 13) \rightarrow (3, 8) \rightarrow (4, 3).$$

Each row satisfies $23 = 5q + r$. The complete certificate gives $q = 4, r = 3$, four transitions, the invariant checks, $d > 0$, and $0 \leq 3 < 5$. With budget two, the run stops at $(2, 13)$. It certifies $23 = 5 \cdot 2 + 13$, but not a completed division because $13 \geq 5$. This algorithm is an external arithmetic experiment, not a property supplied by the residue capability.

Projection makes information loss measurable. Define

$$\pi_{\text{now}}(\text{response}(\rho, S, \rho', R, H)) = (\rho, S, \rho', R), \quad \pi_{\text{full}}(H) = H.$$

The full projection is injective. The current projection is many-to-one whenever histories differ behind an equal head.

Worked example 4: four histories, two summaries (Shannon model).

Let four histories H_1, H_2, H_3, H_4 be equiprobable, with

$$\pi_{\text{now}}(H_1) = \pi_{\text{now}}(H_2) = u, \quad \pi_{\text{now}}(H_3) = \pi_{\text{now}}(H_4) = v.$$

Under this declared discrete distribution,

$$H(\text{history}) = 2 \text{ bits}, \quad H(\text{summary}) = 1 \text{ bit}, \quad H(\text{history} \mid \text{summary}) = 1 \text{ bit}$$

by Shannon's definitions [9]. The missing bit measures unresolved history within each two-member fiber. It is not heat, energy, or a physical erasure cost.

If $H_A \neq H_B$ but $\pi_{\text{now}}(H_A) = \pi_{\text{now}}(H_B)$, a separating question $q_{\text{full}} : \text{History} \rightarrow \text{Answer}$ exists on the full finite history set, for example

$$q_{\text{full}}(H) = [H = H_A].$$

Then $q_{\text{full}} \circ \pi_{\text{full}}$ distinguishes them. For any $q_{\text{now}} : \text{Summary} \rightarrow \text{Answer}$, no q_{now} can separate equal summaries after π_{now} , because both inputs have the same image. Construction carries histories forward through either projection;

reading pulls the separately typed questions back:

$$\pi_{\text{full}}^* q_{\text{full}} = q_{\text{full}} \circ \pi_{\text{full}}, \quad \pi_{\text{now}}^* q_{\text{now}} = q_{\text{now}} \circ \pi_{\text{now}}.$$

A checksum gives another lossy external projection. For bytes $b_1, \dots, b_m$, let $c = (\sum_j b_j) \bmod 7$. Records $[1, 2, 3]$ and $[0, 0, 6]$ share checksum six while remaining distinct. Equality of checksums is not equality of records; the collision certificate identifies only their common projection.

Worked example 5: truncated acquisition (physics, then interpretation). Suppose an apparatus acquires five threshold-event records but a display retains three. Computation can certify a three-record prefix, two-record dropped suffix, and coverage $3 + 2 = 5$. Physics owns the sensor, threshold, acquisition window, dead time, and missed-event model. The displayed three is not a physical event count until that apparatus map and its uncertainty are supplied.

The action ledger is explicit. The residue capability, exact default representative and sample questions, history updates, projections, budgets, and external arithmetic experiments are *posited*. The branch reductions, ignored second receipts and histories, three-response trace, residual prior tail, division invariant, Shannon fiber loss, checksum collision, and truncation coverage are *demonstrated*. Questions are *read* contravariantly through covariant projections and updates. Order laws, numerical remainder meaning, physical residue, and erasure energy remain *reserved*. A branch may be discarded intentionally, but the certificate must state which branch, which fiber, and what information can no longer be recovered.

3.2 Iterative error versus structural residue

A state and a structural residue answer different questions. The recurrence evolves states,

$$x_{k+1} = T(x_k).$$

A state error exists only relative to a declared reference x_* , as $e(x) = x - x_*$. A structural residue is a retained branch, tail, receipt, or complement slice that a projection does not display. The former may shrink, grow, alternate, or reach a fixed point. The latter may have no numerical magnitude at all. Calling both a remainder does not identify them.

For state x , alleged target x_* , update T , finite approximation s_N , exact target s_* , and rounding map Q , the categories are

category	definition	required extra data
state error	$e(x) = x - x_*$	x_*
fixed-point residual	$r(x) = T(x) - x$	T
truncation error	$\tau_N = s_* - s_N$	s_*
rounding error	$\eta_Q(x) = Q(x) - x$	Q
structural residue	retained branch or projection fiber	record projection

A dropped suffix is not automatically numerical. Without a valuation into an additive quantity, its numerical error entry is *absent*, not guessed.

Use the finite audit grammar

$$\begin{aligned}
\textit{state-run} &::= \text{run}(x_0, T, N, \textit{stop-question}), \\
\textit{loss-ledger} &::= \text{ledger}(\textit{stopping-loss}, \textit{rounding-loss}, \textit{truncation-loss}, \textit{retained-branch}), \\
\textit{stop} &::= \text{BUDGET} \mid \text{TOLERANCE} \mid \text{FIXED} \mid \text{RESOLVER-DOMAIN}.
\end{aligned}$$

For budget N , compute $x_{k+1} = T(x_k)$ for $k = 0, \dots, N - 1$. The certificate contains x_0 , N , all transition pairs, the $N + 1$ visited states, declared projections and erasures, and stop reason. State errors e and fixed-point residuals

r are computed ledger columns when their required data are supplied. The certificate certifies no unvisited update.

If an externally justified target x_* exists, a stopping report may include state error $d(x_N, x_*)$ under a declared distance. For a finite record L sliced at K , structural truncation loss is the dropped suffix $\text{drop}_K(L)$, or a declared summary of it. None of these is supplied merely by the recurrence.

Fixed-point and tolerance questions are reads on forward updates. For

$$q_{\text{fix}}(x) = [T(x) = x], \quad q_\varepsilon(x) = [d(x, x_*) \leq \varepsilon],$$

the pulled-back questions are

$$T^*q = q \circ T.$$

The update is covariant; reading is contravariant. A yes-answer to q_{fix} certifies one fixed value. A first yes-answer to q_ε certifies one visited crossing. Neither answer alone proves attraction, uniqueness, or persistence.

Worked example 0: fixed residual, nonzero state error (external numerical experiment). Let T be the identity, $x_* = 0$, and $x_0 = 7$. Then

$$r(x_0) = T(7) - 7 = 0, \quad e(x_0) = 7 - 0 = 7.$$

The update is at a fixed point, but that point is not the alleged target. A zero fixed-point residual does not imply zero state error without uniqueness or attraction premises.

Worked example 1: decreasing update error (external numerical experiment). Let $x_0 = 8$, $T(x) = x/2$, target $x_* = 0$, the ordinary norm, and $N = 4$. Then

$$x_k = \frac{8}{2^k}, \quad e(x_k) = \frac{8}{2^k}, \quad r(x_k) = -\frac{4}{2^k},$$

with trace $[8, 4, 2, 1, \frac{1}{2}]$. If T is a contraction satisfying $\|T(x) - T(y)\| \leq L\|x - y\|$, $0 \leq L < 1$, with fixed point $x_* = T(x_*)$, then

$$\|e(x)\| \leq \frac{\|r(x)\|}{1 - L}.$$

For halving $L = 1/2$, so the bound is exact: $\|e(x_k)\| = 2\|r(x_k)\|$. For tolerance $\varepsilon = 1$, the first accepted value is $x_3 = 1$. A first-hit policy stops there with TOLERANCE; a budget policy records one more update. The trace demonstrates finite decrease. Persistence follows only after separately establishing $0 \leq T(x) \leq x$ for nonnegative x .

Worked example 2: fixed magnitude without error reduction (external numerical experiment). Let $T(e) = -e$, $e_0 = 1$, and $N = 4$. Then

$$[1, -1, 1, -1, 1].$$

The magnitude $|e_k| = 1$ is fixed while the signed error alternates. A question about magnitude returns a constant answer; a question about signed equality does not. Selecting the former projection cannot justify a claim that the full error reached a fixed point.

Worked example 3: nearest-hundredth ledger (external numerical experiment). Declare Q_2 to round to the nearest hundredth, with exact halfway cases sent to the candidate whose final decimal digit is even. Then

$$Q_2(\tfrac{1}{3}) = 0.33 = \frac{33}{100}, \quad \eta_{Q_2}(\tfrac{1}{3}) = \frac{33}{100} - \frac{1}{3} = -\frac{1}{300}.$$

The certificate records exact input, rounded output, scale, tie rule, and signed error. Changing the tie rule can change halfway cases; the name rounding does not select a policy.

Worked example 4: truncation and retained complement (computation). For

$$L = [\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d}, \mathbf{e}], \quad K = 3,$$

stable slicing gives

$$\text{keep}_3(L) = [\mathbf{a}, \mathbf{b}, \mathbf{c}], \quad \text{drop}_3(L) = [\mathbf{d}, \mathbf{e}].$$

Coverage is $3 + 2 = 5$. The dropped slice is structural residue relative to the retained-prefix projection. It is not a scalar error until a separate valuation maps records to numbers. Erasing the suffix destroys reconstruction; keeping it in a complement ledger makes the loss explicit.

A history-bearing response provides the same distinction. Let

$$H_A = \text{response}(\rho, S, \rho', R, H_1), \quad H_B = \text{response}(\rho, S, \rho', R, H_2), \quad H_1 \neq H_2.$$

The current projection maps both to (ρ, S, ρ', R) . Their tails form a structural distinction even when the current numerical error is equal. A full-history question can separate them; no question on the common current summary can.

Worked example 4b: projected fixed point, distinct histories (computation). Let a current-summary update be the identity. The shared summary $z = \pi_{\text{now}}(H_A) = \pi_{\text{now}}(H_B)$ has projected residual

$$T(z) - z = 0.$$

Nevertheless $H_A \neq H_B$ because their tails differ. The zero projected residual certifies stability of the summary only; it does not eliminate the structural residue in the history fiber.

Worked example 4c: finite geometric truncation (external numerical experiment). For

$$s_N = \sum_{k=1}^N 2^{-k} = 1 - 2^{-N},$$

declare exact target $s_* = 1$. Then

$$\tau_N = s_* - s_N = 2^{-N}.$$

The finite run owns the computed s_N . The symbolic error formula additionally depends on the declared target and the separately established geometric identity; a finite prefix alone does not own an unrecorded infinite tail.

Worked example 5: finite information loss (Shannon model). Suppose four histories are equiprobable and a summary projection groups them into two equal fibers. Then, under that declared distribution,

$$H(\text{history}) = 2 \text{ bits}, \quad H(\text{summary}) = 1 \text{ bit}, \quad H(\text{history} \mid \text{summary}) = 1 \text{ bit}$$

by Shannon's definitions [9]. The conditional bit quantifies unresolved history in this model. It is not a temperature, energy, or universal cost of erasure.

A physical error requires another provenance chain. If a sensor report is y_k and a calibrated reference is y_* , physics may define $e_k = y_k - y_*$ with units and uncertainty. Computation can update, slice, and round the finite recorded values. It cannot infer that y_* is physically correct, that noise is independent, or that an unobserved environment received discarded information. Those are experimental premises.

The error ledger for a run is therefore

$$\mathcal{E} = (x_0, T, N, d, x_*, Q, K, \text{trace}, \text{slices}, e, r, \tau, \eta, \text{structural branches}, \text{valuations}, \text{stop}).$$

Every unjustified entry is tagged *absent* rather than guessed. Every supplied target, norm, contraction constant, rounding rule, and valuation is tagged *external* or *demonstrated*. A budget stop reports the last visited state and any error or residual columns justified by supplied reference data, together with the unexecuted suffix of the request. A tolerance stop reports its first accepted index. A fixed stop reports the checked pair $(x, T(x))$. A truncation report records both slice sizes and the fate of the complement.

The action ledger closes the distinction. Updates, metrics, targets, tolerances, contraction premises, rounding rules, truncation budgets, valuations, and projections are *posited*. Finite traces, first hits, fixed pairs, residual identities, contraction bounds under their premises, local rounding losses, geometric finite sums, coverage identities, and projection fibers are *demonstrated*. Error and history questions are *read* contravariantly through covariant updates and projections. Attraction, persistent enclosure, physical accuracy, erasure energy, and numerical meaning for a structural branch remain *reserved*. Iterative error may be reduced; structural residue must be accounted for even when it is not a number.

3.3 Observation becomes history

An observation capability stores a step process, records named before B and after A , a relative question Q_{rel} , and a sample update W . For accumulation forms $\text{empty}(\rho)$ and $\text{indexed}(\rho, S, L)$, the relative question has the exact shape table

B	A	$Q_{\text{rel}}(B, A)$
$\text{empty}(\rho_1)$	$\text{empty}(\rho_2)$	$R(\rho_1, \rho_2)$
$\text{indexed}(\rho_1, S_1, L_1)$	$\text{indexed}(\rho_2, S_2, L_2)$	$R(\rho_1, \rho_2)$
$\text{empty}(\rho_1)$	$\text{indexed}(\rho_2, S_2, L_2)$	no
$\text{indexed}(\rho_1, S_1, L_1)$	$\text{empty}(\rho_2)$	no.

Here R asks whether the first receipts agree in the declared sense. The question ignores both chains and tails, and the capability supplies no resolver. Its name proves no temporal order, causality, or covariance.

Samples have the grammar

$$sample ::= \text{initial}(\rho, L) \mid \text{response}(\rho_s, S, \rho_r, R_s, sample).$$

With fixed receipt ρ_* , the default observation update is

$$\begin{aligned} W(\text{initial}(\rho, L)) &= \text{response}(\rho, L, \rho, A, \text{initial}(\rho, L)), \\ W(\text{response}(\rho_s, S, \rho_r, R_s, H)) &= \text{response}(\rho_s, S, \rho_*, A, \text{response}(\rho_s, S, \rho_r, R_s, H)). \end{aligned}$$

The update retains the entire prior sample. It preserves the first receipt and stimulus and replaces the current response receipt and response. It ignores the stored B and the stored step process.

For origin H_0 and budget N , compute $H_{k+1} = W(H_k)$ for $k = 0, \dots, N - 1$. The certificate contains all transition pairs, $N + 1$ heads, retained receipts, declared erasures, and stop reason. Tail traversal reads newest to oldest and returns the unvisited prior tail on BUDGET.

Worked example 1: accumulating observation history (computation). For $H_0 = \text{initial}(\rho_0, L_0)$,

$$\begin{aligned} H_1 &= \text{response}(\rho_0, L_0, \rho_0, A, H_0), \\ H_2 &= \text{response}(\rho_0, L_0, \rho_*, A, H_1), \\ H_3 &= \text{response}(\rho_0, L_0, \rho_*, A, H_2). \end{aligned}$$

A traversal budget two records the heads of H_3, H_2 , returns H_1 , and certifies nothing about the unvisited tail beyond retaining it.

A trial layer uses a different grammar:

$$\begin{aligned}
 \textit{trial} &::= \text{hypothesis}(\rho, \textit{sample}) \\
 &\quad | \text{trial-response}(\rho_1, \textit{sample}, \rho_2, \textit{sample}, \textit{trial}), \\
 \textit{repeat-process} &::= \text{repeating}(\textit{observation-capability}, \textit{stored-stimulus}, \textit{stored-expectation}, P), \\
 \textit{repeat-interface} &::= \text{repeatability}(\textit{repeat-process}, \textit{typical-question}).
 \end{aligned}$$

Let S_* be the stored stimulus, E_* the stored expectation, and ρ_T, ρ_D fixed receipts. The default trial update is

$$\begin{aligned}
 P(\text{hypothesis}(\rho, S)) &= \text{trial-response}(\rho, S, \rho_T, S_*, E_*), \\
 P(\text{trial-response}(\rho_1, S_1, \rho_2, S_2, T)) &= \text{trial-response}(\rho_2, S_*, \rho_D, S_2, E_*).
 \end{aligned}$$

The second branch ignores the first receipt ρ_1 , first sample S_1 , and prior tail T . Both branches reset the prior tail to the stored expectation E_* . Thus W accumulates old samples, while P resets old trial history.

Worked example 2: trial reset fiber (computation). Let

$$T_1 = \text{trial-response}(\rho_a, S_a, \rho_2, S_2, U_1), \quad T_2 = \text{trial-response}(\rho_b, S_b, \rho_2, S_2, U_2),$$

where the ignored first fields and tails may all differ. Then

$$P(T_1) = P(T_2) = \text{trial-response}(\rho_2, S_*, \rho_D, S_2, E_*).$$

The update has a many-to-one fiber. Equality after the reset does not reconstruct either prior trial.

The outer repeat interface supplies the default typical-response question

$$\text{typical}(s, t) := [P(s) = t].$$

No resolver is supplied for it. This unresolved question proves neither that two runs agree, that repeated applications return one result, that a cadence

is constant, nor that a laboratory experiment is repeatable. Those require finite run certificates and, for physics, an apparatus protocol.

Current and full-history projections remain distinct:

$$\pi_{\text{current}}(\text{response}(\rho_s, S, \rho_r, R_s, H)) = (\rho_s, S, \rho_r, R_s), \quad \pi_{\text{full}}(H) = H.$$

The first is many-to-one; the second is injective. Construction moves histories forward through W or projections. Questions are read contravariantly:

$$\pi_{\text{current}}^* q_c = q_c \circ \pi_{\text{current}}, \quad \pi_{\text{full}}^* q_h = q_h \circ \pi_{\text{full}}.$$

No current-summary question separates equal summaries with different tails.

Worked example 3: current slice versus accumulated history (computation). Let two histories have equal current heads and unequal tails. A current slice puts them in one fiber, while the full-history question $q_A(H) = [H = H_A]$ separates them. With a traversal budget N , coverage and exclusivity apply only to the N visited rows; a stopped traversal makes no statement about its returned tail.

A sufficient statistic requires a declared probability family and target. Let X, Y be independent uniform bits and let the target be their parity

$$\Theta = X \text{ xor } Y.$$

Use summary $Z = \Theta$. The four histories are equiprobable, so

$$H(X, Y) = 2, \quad H(Z) = 1, \quad H(X, Y \mid Z) = 1$$

in bits. The summary discards one bit of history but is sufficient for the declared parity target because

$$H(\Theta \mid Z) = 0.$$

By contrast, the first bit alone is not sufficient:

$$H(\Theta \mid X) = 1.$$

Worked example 4: sufficiency is target-relative (Shannon model).

The parity summary perfectly answers the parity question but cannot reconstruct which of the two histories in its fiber occurred. These entropy identities require the displayed joint distribution [9]. Changing the target or distribution requires a new sufficiency calculation. The discarded bit is information in this model, not heat or energy.

Worked example 5: apparatus history (physics, then interpretation).

Suppose a detector creates time-stamped threshold records. Physics owns the sensor, clock calibration, threshold, dead time, missed-record rate, and uncertainty. Computation can apply W , P , project the latest record, and traverse finite history. Neither prepend orientation nor a typical-response question establishes physical time, constant cadence, causality, or experimental repeatability.

The observation ledger is

$$\mathcal{O} = (B, A, Q_{\text{rel}}, W, \text{repeat-process}, P, \text{repeat-interface}, S_*, E_*, N, \text{traces}, \pi_{\text{current}}, \pi_{\text{full}}, \text{slices}, \text{probability})$$

Absent resolvers, distributions, apparatus maps, cadence models, and uncertainty are marked absent.

The action ledger is exact. The observation capability, trial grammar, inner repeat process, outer repeat interface, default Q_{rel}, W, P and typical question, budgets, projections, probability family, and target are *posited*. The ignored stored records and arguments, accumulating observation trace, resetting trial fiber, finite slices, entropy identities, and target-relative parity sufficiency are *demonstrated*. Current and history questions are *read* contravariantly through covariant updates and projections. Temporal order,

cadence, causal direction, whole-history recovery from a current summary, laboratory repeatability, and sufficiency outside the declared model remain *reserved*. Observation becomes history under W ; the later trial update P deliberately resets that history to its stored expectation.

Part II

The Turn

Chapter 4

The Tower Turns Around

4.1 Requirements pull backward

A requirement is a question on reported values. Let S be a finite setup domain, R a report domain, and

$$O : S \rightarrow R$$

a forward observation map. Let Q be requirement syntax on R , and separately posit a partial evaluator

$$\text{eval}_Q : R \rightarrow \text{Option}\{\text{yes}, \text{no}\}.$$

Its resolved reading pulls backward to setups:

$$O^* \text{eval}_Q = \text{eval}_Q \circ O.$$

Construction or observation carries a setup forward to a report; reading asks which setups satisfy a report-side question. The requirement does not choose a setup, construct a witness, or perform an experiment. The evaluator may return evidenced yes, evidenced no, or no value; the last case leaves Q

unresolved.

Composition reverses:

$$S \xrightarrow{O} R \xrightarrow{G} T, \quad (G \circ O)^* \text{eval}_P = O^*(G^* \text{eval}_P)$$

for question syntax P on T and its partial evaluator. The outer evaluator is pulled through G first and then through O . This contravariant reading is distinct from reversing arguments inside a comparison rule.

An admissible fiber is

$$F_Q = \{s \in S : \text{eval}_Q(O(s)) = \text{some}(\text{yes})\}.$$

Using the word admissible requires a declared Q . The set F_Q is defined even when $S = \emptyset$, and may itself be empty for nonempty S . Claiming that a selectable setup exists requires

$$F_Q \neq \emptyset.$$

Even that statement is only an existence requirement. It does not select an element of F_Q ; selection and witness construction belong to the next action.

For a family $(F_i)_{i \in I}$, a global selection claim requires

$$\forall i \in I, \quad F_i \neq \emptyset.$$

For a finite explicitly listed family with decidable membership, exhaustive search selects the first member of each fiber and certifies every choice; no axiom of choice is needed. An arbitrary family requires a separate selection principle and still cannot select from an empty fiber.

Tolerance must also be explicit. A raw metric test

$$d(r, r') \leq \varepsilon$$

is reflexive and symmetric but generally not transitive, so it is not automatically an equivalence relation. For a genuine tolerance equivalence, declare a finite bin map $b_{h,c} : R \rightarrow B$, with scale h and boundary convention c , and define

$$r \equiv_{h,c} r' \iff b_{h,c}(r) = b_{h,c}(r').$$

Equality in B makes this relation reflexive, symmetric, and transitive. A desired report bin $\beta \in B$ gives

$$\text{eval}_{Q_\beta}(r) = \begin{cases} \text{some}(\text{yes}), & b_{h,c}(r) = \beta, \\ \text{some}(\text{no}), & b_{h,c}(r) \neq \beta, \end{cases} \quad F_\beta = O^{-1}(b_{h,c}^{-1}(\{\beta\})).$$

The scale h , cell diameter, boundary convention, and units are part of the requirement. A bin scale is not automatically the metric tolerance ε ; their relation must be demonstrated for the chosen geometry.

Worked example 1: a nonempty tolerance fiber (external numerical experiment). Let

$$S = \{\mathbf{a}, \mathbf{b}, \mathbf{c}\}, \quad O(\mathbf{a}) = 1.02, \quad O(\mathbf{b}) = 1.11, \quad O(\mathbf{c}) = 1.27,$$

and use left-closed bins $b_{0.1,\text{left}}(r) = \lfloor 10r \rfloor$, whose cells have diameter 0.1. For desired bin $\beta = 11$,

$$F_{11} = \{\mathbf{b}\}.$$

The fiber is nonempty, so an admissible setup exists. Its singleton form identifies $\mathbf{b}$ as the sole admissible candidate, but no subsequent setup-selection action has yet been performed.

A bounded filter realizes a finite tolerance read. Given candidates $S_N = [s_1, \dots, s_N]$, target r_* , tolerance ε , and budget $B \leq N$, visit $i = 1, \dots, B$, compute

$$y_i = O(s_i), \quad \delta_i = d(y_i, r_*),$$

and retain s_i exactly when $\delta_i \leq \varepsilon$. Stop with COMPLETE when $B = N$, BUDGET when $B < N$, or RESOLVER-DOMAIN. The certificate contains candidate indices, readings, residuals, evidenced decisions, retained and rejected slices, N, B , completeness, erasures, and stop reason. An empty retained slice means only that no visited candidate passed. It proves an empty fiber only when S_N is certified as a complete enumeration of S and the run is complete.

Projecting a residual to a pass bit or bin erases information. Equal residuals do not identify candidates or readings; equal pass bits erase still more. The report records the fate of residual, pass, and bin values.

Worked example 2: a requirement can be unsatisfied (external numerical experiment). With the same domain and observation map, desired bin $\beta = 14$ gives

$$F_{14} = \emptyset.$$

The requirement remains a well-formed question even though no supplied setup meets it. Neither a name nor a choice principle manufactures a member of an empty fiber.

Requirements may themselves depend on prior requirements. Represent a finite dependency record as nodes (v, Q_v) and directed edges $v \rightarrow u$, meaning that resolving Q_v requires the report for u . A budgeted backward traversal uses a worklist initialized with the requested node. At each visit:

1. remove one unresolved requirement v ,
2. append each unvisited dependency u to the worklist,
3. when dependencies are resolved, evaluate the declared resolver for Q_v ,
4. record v in the yes, no, or unresolved slice.

Stop with COMPLETE, BUDGET, RESOLVER-DOMAIN, or DEPENDENCY-CYCLE. The certificate contains the requested node, visited nodes and edges,

traversal order, all resolver outputs, unresolved worklist, slices, budget, and stop reason. It certifies only the visited dependency subgraph.

Worked example 3: finite backward dependency traversal (computation). Let requirement A depend on B, C , and C depend on D :

$$A \rightarrow \{B, C\}, \quad C \rightarrow D.$$

A depth-first backward traversal may visit A, B, C, D . With budget three, it stops after A, B, C , returns D on the unresolved worklist, and cannot resolve C or A . The certificate demonstrates the explored edges and budget stop; it does not infer the missing answer.

Worked example 4: composed reading (computation). Let $O : S \rightarrow R$ report an integer and $G : R \rightarrow \{\text{even}, \text{odd}\}$ report parity. For the question P whose evaluator returns `some(yes)` on even and `some(no)` on odd,

$$(G \circ O)^* \text{eval}_P(s) = \text{eval}_P(G(O(s))) = O^*(G^* \text{eval}_P)(s).$$

The equality demonstrates two descriptions of one pulled-back question. It does not identify the maps O, G , their costs, or their report domains.

Cohen’s finite-condition method offers a computation-side resonance. A finite partial assignment

$$p = \{v_1 \mapsto a_1, \dots, v_k \mapsto a_k\}$$

states requirements on finitely many coordinates. An extension $q \supseteq p$ must preserve earlier assignments. This resembles a dependency ledger whose resolved requirements cannot be silently changed [3]. In forcing vocabulary, a condition set D_Q is dense precisely when

$$\forall p \exists q \supseteq p, \quad q \in D_Q,$$

where membership in D_Q means that q decides or satisfies the declared requirement Q . The finite worklist does not demonstrate this quantified density statement. The analogy stops there: no forcing relation, generic filter, generic extension, or independence result follows from a finite worklist.

Worked example 5: physical acceptance window (physics, then interpretation). Suppose an apparatus reports a voltage $O(s)$ for each finite setup. Physics owns the apparatus, voltage units, calibration, noise model, and repeated observations. A bin requirement can classify recorded voltages, but it does not show that a setup will reproduce the bin on another trial. Nonempty empirical fibers and physical reproducibility need their own uncertainty and sampling certificates.

The requirement audit is

$$\mathcal{D} = (S, R, O, Q, \text{eval}_Q, \varepsilon, b_{h,c}, h, c, \text{cell diameter}, \text{dependency graph}, \text{resolvers}, S_N, \text{candidate indices},$$

An absent resolver, empty domain, empty admissible fiber, or unvisited dependency is reported directly.

The action ledger closes the section. Domains, forward maps, question syntax, partial evaluators, metric tolerances, bin maps, dependency edges, candidate lists, and budgets are *posited*. Pullback identities, bin-equivalence laws, coverage and exclusivity invariants, completeness implications, finite exhaustive-selection correctness, and quantified nonemptiness checks are *demonstrated*. Actual traversal order, visited nodes, worklist, stop reason, resolver outputs, fiber membership, residual comparisons, and dependency answers are *read* through contravariant questions on covariant forward maps. Arbitrary or global selection, witness action, physical reproducibility, forcing conclusions, and answers outside the visited subgraph remain *reserved*. Requirements pull backward; actions that satisfy them come later.

4.2 Witnesses run forward

A requirement, an existence statement, a selection, and a witness are four different records. Retain the setup and report domains S, R , forward map $O : S \rightarrow R$, question syntax Q , and partial evaluator

$$\text{eval}_Q : R \rightarrow \text{Option}\{\text{yes}, \text{no}\}.$$

The requirement read is contravariant:

$$O^* \text{eval}_Q = \text{eval}_Q \circ O.$$

Its admissible fiber is

$$F_Q = \{s \in S : \text{eval}_Q(O(s)) = \text{some}(\text{yes})\}.$$

The statement $F_Q \neq \emptyset$ establishes existence. A selector action must then produce a particular $s \in F_Q$. Only after that action can forward construction package a witness

$$w = \text{witness}(s, y, c_O, c_Q, c_D),$$

where $y = O(s)$, c_O certifies the forward evaluation, c_Q is the evidenced yes-answer, and c_D records dependency certificates. Neither the requirement nor nonemptiness contains this record.

For a finite candidate list $S_N = [s_1, \dots, s_N]$, a budgeted witness search visits at most $B \leq N$ candidates. For each visited s_i :

1. $y_i \leftarrow O(s_i)$,
2. $a_i \leftarrow \text{eval}_Q(y_i)$,
3. append (i, s_i, y_i, a_i) to yes, no, or unresolved slice,
4. on the declared first-yes policy, construct w_i and stop.

Stop reasons are WITNESS, COMPLETE-NO, BUDGET, and RESOLVER-DOMAIN.

The certificate contains candidate indices, forward reports, evaluator answers, all slices, selected index when present, budget, completeness status, erasures, and stop reason. A budget stop with no yes-answer does not prove F_Q empty. A complete-no result proves emptiness only when S_N is a certified complete enumeration of S .

Worked example 1: finite witness search (computation). Let $S = \{2, 3, 4, 5\}$, $O(s) = s^2$, and let Q ask whether the report exceeds 15. In list order, the reports are 4, 9, 16, 25. A first-yes policy selects $s = 4$ and returns

$$w = \text{witness}(4, 16, [16 = 4^2], [16 > 15], c_D).$$

The backward requirement identifies the fiber $\{4, 5\}$; the forward action selects one member according to the declared list order. A different selection policy may return 5 without changing the requirement.

This finite exhaustive selection uses the explicit list and evidenced evaluator; it needs no axiom of choice. Selecting simultaneously from an arbitrary family of nonempty fibers is a separate global premise and remains reserved.

Dependencies require an exact two-pass finite algorithm. First traverse edges backward from the requested target, record the reverse dependency slice, reject missing predecessors, test acyclicity, and compute a topological order with predecessors before consumers. Then, for each node in that order:

1. wait until every predecessor certificate is present,
2. form a cache key from inputs, predecessor results, policies, and versions,
3. if an entry exists, run its declared checker and reuse only on success,
4. otherwise spend construction budget, run the node builder, run the checker,
5. memoize only a checked result and append the node to the new or reused slice.

Stops MISSING-DEPENDENCY, CHECKER-FAILURE, BUDGET, and DEPENDENCY-CYCLE return the unresolved frontier. None proves an admissible fiber empty.

The run ledger slices nodes into newly constructed, verified-reused, checker-failed, and unresolved rows.

Worked example 2: diamond dependency and memoized reuse (computation). Let

$$A \leftarrow \{B, C\}, \quad B \leftarrow D, \quad C \leftarrow D.$$

The reverse slice has four nodes, and one valid forward order is

$$D, B, C, A.$$

With an empty cache and sufficient budget, the run invokes four builders; D is constructed once and its checked certificate is consumed by both B and C . If a valid D entry is preloaded, the run performs one cache verification and three new constructions, for B, C, A . The ledger reports exactly three new results, one verified reuse, and the dependency edges each consumer checked.

Worked example 3: cache invalidation and erasure (computation).

A cache entry is invalid when any dependency result, builder version, checker version, policy, configuration, forward map O , or evaluator changes. Matching a node name alone is unsound. A compact hash key is acceptable only under an explicit collision-resistance or collision-checking assumption; hash equality is not mathematical identity. Storing only the target result may preserve a value while erasing construction trace, checker trace, and cost. The audit must record each such erasure rather than treating a cache hit as a complete historical witness.

Turing’s framework separates executable evidence from oracle assertion. For a declared deterministic transition system and encoded input, a finite halting trace

$$c_0 \rightarrow c_1 \rightarrow \cdots \rightarrow c_m = \text{halt}$$

is an executable run-specific witness: every transition and the final halt condition can be checked. An assertion $\mathcal{H}(M, x)$ supplied by an oracle without such a trace is openly posited, not constructed. For an effective universal machine model, Turing’s result rules out a total computable procedure that correctly decides halting for every encoded machine-input pair [10]. It does not invalidate a finite halting trace for one run, and such a trace witnesses only the positive halting case.

Worked example 4: trace witness versus oracle assertion (computation). Consider a finite transition rule that decrements a positive counter and halts at zero. From three, the trace

$$3 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow \text{halt}$$

is a checkable witness under the declared rule. The bare assertion *this run halts* may be correct, but without the rule and trace it is only an assertion. Conversely, a finite budget stop before halt is not evidence that the run never halts.

Worked example 5: apparatus witness (physics, then interpretation). Suppose a setup produces a voltage inside a declared acceptance bin. Physics owns the apparatus, calibration, units, noise model, and repeated measurements. A computational witness can package the selected setup, recorded voltage, bin decision, and acquisition certificate. It does not establish that every future trial will agree or that the voltage has another physical name. Those conclusions need separate experimental evidence.

The witness audit is

$\mathcal{W} = (S, R, O, Q, \text{eval}_Q, S_N, N, B, \text{selection policy}, \text{target node}, \text{dependency graph}, \text{reverse slice}, \text{acyclic})$

Missing dependencies, unresolved answers, stale memo entries, partial can-

candidate coverage, and oracle assumptions are reported directly.

The action ledger closes the section. Domains, maps, question syntax, evaluators, candidate order, selection policy, dependency graph, builders, checkers, budgets, versions, hash assumptions, and memo policy are *posited*. Topological-order validity, traversal invariants, cache-verification implications, certificate composition, and the fact that a completed witness satisfies its pulled-back requirement are *demonstrated*. Actual reverse slice, traversal order, visited nodes, frontier, stop reason, reports, answers, cache hits, checker outputs, costs, and fiber membership are *read*. The witness is the covariant action and result satisfying a contravariantly pulled requirement; it is not the requirement itself. Arbitrary global selection, unsupported oracle answers, future physical reproducibility, and claims beyond the visited dependencies remain *reserved*. Requirements pull backward; witnesses run forward only after selection acts.

4.3 Two-pass calibration

A two-pass calibration earns a name by surviving an exchange of roles. Let three labels be X, Y, Z . Pass one returns and verifies two direct values x_1, y_1 , then forms an explicit composite

$$z_\Sigma = \Sigma_1(x_1, y_1).$$

Pass two rotates the roles: it returns and verifies y_2, z_2 , then reconstructs

$$x_\Sigma = \Sigma_2(y_2, z_2).$$

The maps Σ_1, Σ_2 , their parameters, budgets, stopping rules, scales, and comparison resolution must be registered before either pass is evaluated and must not depend on an outside target.

Three comparisons close the protocol:

$$y_1 \equiv_\varepsilon y_2, \quad z_\Sigma \equiv_\varepsilon z_2, \quad x_\Sigma \equiv_\varepsilon x_1.$$

The first checks the direct overlap. The second checks the pass-one reconstruction against a pass-two direct value. The third checks the pass-two reconstruction against the pass-one direct value and closes the round trip. A shared name is licensed only after all three comparisons pass. Two direct values and one composite are recorded at each pass; the composite is never silently promoted to a direct reading.

Construction runs forward:

$$(x_1, y_1) \xrightarrow{\Sigma_1} z_\Sigma, \quad (y_2, z_2) \xrightarrow{\Sigma_2} x_\Sigma.$$

Requirements read contravariantly. If F maps the complete pass inputs to the three comparison pairs and q_ε asks whether a pair agrees at the declared resolution, then

$$F^* q_\varepsilon = q_\varepsilon \circ F.$$

The comparison question does not construct a report, and the forward maps do not choose their acceptance criterion.

Neither composite map is assumed injective. Distinct direct pairs may occupy one composite fiber:

$$\Sigma_1(x, y) = \Sigma_1(x', y') \not\Rightarrow (x, y) = (x', y').$$

The pass ledger therefore retains direct pairs, composite values, and fiber evidence separately, and records any erasure caused by keeping only a composite. This is a computational handoff between finite records, not a physical or quantum superposition.

Worked check 1: exact finite role rotation (external numerical design). Take direct values $x_1 = 2, y_1 = 3$ and preregister

$$\Sigma_1(x, y) = x + y.$$

Then $z_\Sigma = 5$. Pass two directly returns $y_2 = 3, z_2 = 5$ and uses the preregistered inverse reconstruction

$$\Sigma_2(y, z) = z - y,$$

giving $x_\Sigma = 2$. All three comparisons close exactly. This demonstrates the protocol shape; it is not the instrument's measured calibration and carries no physical interpretation.

The actual instrument uses two finite numerical routes over shared calibration data

$$C = 18, \quad \text{target} = 5, \quad d_* = \sqrt{C/\text{target}} = \sqrt{18/5}.$$

The current report directly certifies the two route endpoints and the jar. It does not by itself print every X, Y, Z role-rotation entry above; the full three-role certificate remains a calibration requirement until those direct/composite records are emitted. The direct $1 \rightarrow 3$ route evaluates the measured second-variation formula at d_* and reports

$$d = 137011290548979455469 \quad \text{at scale } 10^{18}.$$

The stepped $1 \rightarrow 2 \rightarrow 3$ route carries its residue through an intermediate quasi-Newton descent and reports

$$s = 137011290753751157155 \quad \text{at scale } 10^{18}.$$

Its finite count trace is

count	distance report	inverse report
0	2/1	129.6
1	17/9	137.7
2	≈ 1.897309	≈ 137.016
3	≈ 1.897366	$s/10^{18}$.

Mediant/continued-fraction, Newton/BFGS, and residue-carrying secant realizations provide separately declared stepped algorithms. Agreement of their displayed endpoint does not identify their traces or costs.

Worked check 2: path comparison at declared resolution (instrument computation). The scaled discrepancy is

$$|d - s| = 204771701686, \quad \frac{|d - s|}{10^{18}} = 2.04771701686 \times 10^{-7}.$$

The scaled integers are not equal. Their decimal reports agree to nine digits, and the discrepancy is below the count-three resolution. Let b_3 be the preregistered finite partition whose owned cell is the count-three jar, and write

$$a \equiv_3 b \iff b_3(a) = b_3(b).$$

This is an equivalence induced by equality of bin labels; for these reports, $d \equiv_3 s$. The house slogan $1 \rightarrow 3 = 1 \rightarrow 2 \rightarrow 3$ abbreviates $d \equiv_3 s$, not integer equality and not identity of algorithms.

The one discrepancy $\Delta = |d - s|/10^{18}$ is not a variance estimate. Statistical variance requires at least $n \geq 2$ comparable repeated discrepancies $\Delta_1, \dots, \Delta_n$:

$$\bar{\Delta} = \frac{1}{n} \sum_{j=1}^n \Delta_j, \quad s_{\Delta}^2 = \frac{1}{n-1} \sum_{j=1}^n (\Delta_j - \bar{\Delta})^2.$$

Inference from s_{Δ}^2 additionally requires a declared sampling model, stability assumptions, and treatment of dependence. The direct and stepped routes share calibration inputs and are not automatically independent. For random reports D, S ,

$$\text{Var}(D - S) = \text{Var}(D) + \text{Var}(S) - 2 \text{Cov}(D, S).$$

This probabilistic covariance is distinct from categorical covariance of forward maps. No statistical variance, covariance, or uncertainty interval has been computed by the current two-route comparison.

The owned result is the ordered-wall jar written

$$[129.6, 137.7].$$

The word open is the repository's metaphor for unresolved content, not topological interval notation. Both finer reports near 137.011290 fall between the ordered walls, but the instrument hands over the jar rather than either point.

Worked check 3: wall and ownership audit (instrument computation). The lower and upper walls come from the first two finite stepped reports, $2/1 \mapsto 129.6$ and $17/9 \mapsto 137.7$. Their order is checked, the count-three stop is recorded, and both direct and stepped endpoint reports are compared against the walls. These checks establish the owned bracket and path agreement at its resolution. They do not establish agreement with an external physical constant.

Richardson extrapolation remains conditional. The direct and stepped values arise from different algorithms; no common family $A(h)$, refinement ratio r , or error order p is established. Under the counterfactual assumptions $D = A(h)$, $S = A(h/2)$, one could write

$$A_R = S + \frac{S - D}{2^p - 1}.$$

Those sensitivity values depend on the unestablished order and orientation. Licensing the method requires one implemented ordered h -family, at least three refinement levels, observed-order estimation, and stability under further refinement. A machine/configuration/cost index is one proposed candidate for h , but it requires an implemented ordering and an error model. Greater budget or cost is not automatically finer resolution. The current direct and stepped pair is therefore invalid for Richardson extrapolation. Until a qualifying family exists, no extrapolate is a measurement, uncertainty bound, or owned result.

Worked check 4: target-dependent transition rejected (adversarial test). Suppose a proposed composite map is adjusted after seeing a desired number t , for example

$$\tilde{\Sigma}_1(x, y) = t.$$

It will reproduce t regardless of the direct readings. The numerical comparison may pass, but preregistration and target-independence fail, so calibration is rejected. Agreement manufactured by the transition is not path independence.

Each route and pass has an independent budget. A pass certificate records direct values and their verification, composite values, map definitions and versions, inputs, scales, update traces, stopping rules, budgets spent, residuals, comparison decisions, erasures, and provenance. Stops are COMPLETE, BUDGET, MISSING-DIRECT, COMPARISON-UNRESOLVED, OVERLAP-FAILURE, RECONSTRUCTION-FAILURE, CLOSURE-FAILURE, VERSION-MISMATCH, and DOMAIN-ERROR. Any noncomplete stop returns the unresolved frontier; it does not license the shared name.

The calibration audit is

$\mathcal{C} = (X, Y, Z, \text{role assignments, overlap identities, } \Sigma_1, \Sigma_2, \text{versions, preregistration, target-independence, pass-one direct/composite records, pass-two direct/composite records, overlap, reconstructions, } d, s, \text{scale, raw discrepancy, resolution relation, bin-map version, discrepancy-bin version, jar wall, conditional statistical assumptions/results, budgets, traces, stops, unresolved frontier, erasures, ...})$

The action ledger is exact. Calibration data, role rotation, composite maps, algorithms, budgets, resolution relation, versions, bin map, statistical assumptions, and stop policies are *posited*. Arithmetic identities, wall-order checks, join/fiber implications, resolution-equivalence laws, and round-trip acceptance implications are *demonstrated*. Actual direct and stepped outputs, raw discrepancy, residual and comparison decisions, traces, budgets spent, statistical outputs when any, owned walls, unresolved frontier, and stops are *read*. The name α_c is licensed for the path-independent computational result at declared resolution. Equality of methods, integer reports, costs, physical provenance, an external empirical constant, and conditional extrapolates remain *reserved*. The honest handed-over result is the jar.

Chapter 5

The Slip Becomes A Number

5.1 A crossing predicate

A crossing predicate is a finite question, not yet a root or a physical event. Let the positive rational domain be

$$\mathbb{Q}_{>0}^{\text{rec}} = \{(p, q) : p, q \in \mathbb{N}, p > 0, q > 0\}, \quad d = p/q.$$

Suppose a report is represented by integers $A(p, q) \geq 0$ and $B(p, q) > 0$, with reported ratio A/B , and let target $t \geq 0$ be an integer in compatible declared units. The exact crossing question is

$$C_t(p, q) := [t B(p, q) \leq A(p, q)].$$

It uses integer cross-multiplication. No decimal approximation or floor is required to decide the branch.

A forward evaluator constructs

$$E_t(p, q) = (A(p, q), B(p, q), tB(p, q), A(p, q)).$$

The requirement question on this report asks whether the third entry is at

most the fourth. Reading pulls that question back through E_t ; the evaluator is covariant and the requirement read is contravariant. The question does not construct a distance or choose a side of a bracket.

For the instrument's finite crossing model,

$$g(d) = \frac{18}{d^2}, \quad t = 5.$$

At $d = p/q > 0$,

$$C_5(p, q) := [5p^2 \leq 18q^2].$$

The constants 18 and 5, the positive domain, and the interpretation of d are posited calibration data. If units are attached, $18/d^2$ and 5 must have matching units before comparison.

The monotone orientation follows directly. For $0 < d_1 \leq d_2$,

$$d_1^2 \leq d_2^2, \quad \frac{1}{d_1^2} \geq \frac{1}{d_2^2}, \quad \frac{18}{d_1^2} \geq \frac{18}{d_2^2}.$$

Therefore the yes region for C_5 lies downward in distance. The floored report $[18/d^2]$ is also nonincreasing, but has plateaus on which distinct ratios receive the same integer.

A bracket claim needs additional premises. On an ordered domain D , require:

$$a < b, \quad C_t(a) = \text{yes}, \quad C_t(b) = \text{no},$$

and monotonicity in the declared orientation,

$$d_1 < d_2 \implies g(d_1) \geq g(d_2).$$

These facts certify that the bracket straddles a truth transition. They do not automatically prove that an exact rational equality $g(d) = t$ exists inside; the boundary may lie outside the represented rational set.

A finite scan makes the method auditable. Given ordered candidates $D_N = [d_1, \dots, d_N]$ and budget $B \leq N$, visit $i = 1, \dots, B$. Verify the domain conditions, construct the cross-products, resolve $C_t(d_i)$, and append the full row to crossed, not-crossed, or domain-error slices. Stop with COMPLETE, BUDGET, DOMAIN-ERROR, or RESOLVER-DOMAIN. The certificate contains candidate indices and rationals, unreduced and reduced forms when used, cross-products, decisions, slices, units, budget, completeness, erasures, and stop reason. An unvisited suffix carries no decision.

Worked example 1: exact endpoint crossing (instrument computation). At $d = 1 = 1/1$,

$$5 \cdot 1^2 = 5 \leq 18 \cdot 1^2 = 18,$$

so the predicate answers yes. At $d = 2 = 2/1$,

$$5 \cdot 2^2 = 20 > 18,$$

so it answers no. The ordered endpoints straddle the transition. At $d = 3/2$,

$$5 \cdot 3^2 = 45 \leq 18 \cdot 2^2 = 72,$$

so the midpoint candidate remains on the crossed side.

Worked example 2: a finite rational slice (instrument computation). Visit $d = 9/5, 19/10, 2$. The exact comparisons are

d	$5p^2$	$18q^2$
9/5	405	450
19/10	1805	1800
2/1	20	18

and the decision slice is

$$\text{crossed} = [9/5], \quad \text{not-crossed} = [19/10, 2].$$

Thus the sampled transition lies between $9/5$ and $19/10$. This is a statement about the sampled exact predicate, not yet a certified narrowest bracket.

A scan certifies only the visited rows and, when complete, the transitions between adjacent listed candidates. It does not produce the canonical seed-free continued-fraction jar. The canonical distance walls begin with $17/9$ and 2 :

$$17/9 < 2, \quad 5 \cdot 17^2 = 1445 \leq 1458 = 18 \cdot 9^2, \quad 5 \cdot 2^2 = 20 > 18.$$

Thus $17/9$ is on the yes side and 2 on the no side. Their later continued-fraction role may be developed in the jar section; the exact opposed-wall arithmetic already belongs in this certificate.

Monotonicity cannot be inferred from one change of branch. It must be established from the report formula or tested only as a finite empirical pattern.

Worked example 3: nonmonotone counterexample (external numerical experiment). Let

$$h(d) = (d - 1)(d - 2), \quad C(d) = [h(d) \geq 0].$$

The predicate is yes at $d = 1/2$, no at $d = 3/2$, and yes again at $d = 3$. A scan sees two transitions. Keeping one half after a branch test without a monotonicity or sign-change argument could discard another crossing. The shared word crossing does not license a bisection invariant.

Worked example 4: malformed rational record (computation). The record $(p, q) = (3, 0)$ has no represented positive rational value. Cross-

multiplication might still produce integers, but the domain certificate fails first. The correct result is DOMAIN-ERROR, not a crossing decision. Likewise, a negative or unit-incompatible target requires a different declared domain and question.

A compact rational bisection contract is also available. Validate positive ordered endpoints

$$\ell < u, \quad C_t(\ell) = \text{yes}, \quad C_t(u) = \text{no}.$$

Form the exact rational midpoint $m = (\ell + u)/2$. If $C_t(m)$ is yes, replace ℓ by m ; otherwise replace u by m . Each accepted step preserves endpoint order and the opposed-bit invariant. With finite fuel N , the certificate returns the final rational bracket, every midpoint and branch decision, unused fuel, and stop reason. It certifies a truth-transition bracket, not an exact represented root. This legacy fuel-bisection route is distinct from the canonical seed-free continued-fraction jar; agreement between their walls requires a separate comparison.

The floor report is related but distinct. For nonnegative A and positive B ,

$$\left\lfloor \frac{A}{B} \right\rfloor \geq t \iff tB \leq A.$$

This identity explains why the exact crossing question can be evaluated without performing division. The certificate must retain A, B, tB , and the comparison result; storing only the Boolean branch erases the distance from threshold. Equal floors do not imply equal ratios or equal distances. The fractional residue

$$A - B \lfloor A/B \rfloor$$

records information erased by the floor. Also, unreduced pairs such as (p, q) and $(2p, 2q)$ represent the same rational distance while remaining different records; canonicalization by a positive greatest common divisor must be de-

clared when record equality is intended to mean rational equality.

Worked example 5: static and kinetic slip (physics, then interpretation). A friction experiment may define a physical breakaway event through applied force, normal load, surface preparation, velocity, temperature, and an uncertainty model. Physics owns those variables and the distinction between static and kinetic regimes. Reading $C_t(d)$ as that event is an interpretation requiring an apparatus map from rational distance records to measured forces. The finite inequality alone supplies no friction law, discontinuity, or unpredictability claim.

The crossing audit is

$\mathcal{X} = (D, t, \text{units}, A, B, C_t, \text{monotonicity premise}, \text{endpoint orientation/bits}, \text{raw } A/B, \text{floor residue},$

An absent monotonicity argument, failed domain check, incompatible unit, partial scan, or unresolved arithmetic operation is recorded directly. The audit must state whether integers are unbounded; for bounded integers it must instead certify that every square and cross-product avoids overflow.

The action ledger closes the section. Domain, target, units, report formula, candidate order, canonicalization, arithmetic model, route, budgets, and stop policies are *posited*. Monotonicity, cross-product identities, endpoint inequalities, bisection invariants, finite slice coverage, and implications proved from declared premises are *demonstrated*. Actual candidate reports, floors, residues, midpoint and branch decisions, slices, budgets spent, frontiers, and stops are *read*. Existence of an exact represented root, uniqueness, physical breakaway, and claims beyond the scanned prefix remain *reserved*. A crossing predicate supplies one finite question; bracketing requires the premises that let its answers be sliced safely.

5.2 Bisection is iteration by slicing

A bisection bracket is finite data:

$$\mathcal{B} = (\ell, u, c_\ell, c_u), \quad \ell, u \in \mathbb{Q}_{>0}^{\text{rec}},$$

where $c_\ell = C_t(\ell)$ and $c_u = C_t(u)$. In the downward-yes orientation, a valid input satisfies

$$\ell < u, \quad c_\ell = \text{yes}, \quad c_u = \text{no}.$$

The bracket does not contain a point value or a guarantee that a represented equality exists.

For $\ell = p/q$ and $u = r/s$, with positive denominators, the exact midpoint is

$$m = \frac{ps + rq}{2qs}.$$

A one-step slice evaluates $c_m = C_t(m)$ and returns

$$\text{bisectOnce}(\mathcal{B}) = \begin{cases} ((\ell, u, m, c_m), (m, u, \text{yes}, \text{no})), & c_m = \text{yes}, \\ ((\ell, u, m, c_m), (\ell, m, \text{yes}, \text{no})), & c_m = \text{no}. \end{cases}$$

The first component is the step ledger; the second is the retained half. Exact arithmetic may leave the midpoint unreduced, so a canonicalization policy must be declared separately.

The pure loop is fuel-only and recursive:

$$\begin{aligned} \text{run}(0, \mathcal{B}) &= (\mathcal{B}, []), \\ \text{run}(n+1, \mathcal{B}) &= \text{let } (\sigma, \mathcal{B}') = \text{bisectOnce}(\mathcal{B}), \\ &\quad \text{let } (\mathcal{B}'', T) = \text{run}(n, \mathcal{B}') \text{ in } (\mathcal{B}'', \sigma :: T). \end{aligned}$$

Thus zero fuel returns the current bracket and an empty trace. Successor fuel performs one step, recurses on the updated bracket with the remaining fuel,

and prepends the current step to the remaining trace, preserving chronological order. A successful run with fuel N has trace length N . Width-target stopping is a proposed extension, not part of this pure recurrence.

The certificate contains initial and final brackets, every midpoint, raw cross-products, branch decisions, canonical forms when used, fuel and trace length, widths, erasures, and stop reason. A fuel-complete result owns a finite bracket, not an unread infinite tail.

A replay checker validates each adjacent row: recompute the exact midpoint; recompute cross-products and crossing bit; verify that the selected branch produces the recorded next bracket; verify that the next row's old bracket equals that result; recheck opposed endpoint bits, order, and width; and finally check fuel accounting and trace length. Replay success certifies adjacency and invariant preservation for the recorded finite run.

The familiar width law is conditional. Interpret the positive rationals in an ordered metric line, use the exact arithmetic midpoint, preserve endpoint order and opposed crossing bits, and assume the monotone enclosure orientation from the previous section. Then

$$w_k = u_k - \ell_k, \quad w_{k+1} = \frac{w_k}{2}, \quad w_n = w_0 2^{-n}.$$

This follows by induction from retaining one of two equal halves. The formula proves shrinking width under those hypotheses. It does not prove an exact root, uniqueness without stricter premises, or physical correctness.

Worked example 1: four exact slices (instrument computation).

For $C_5(p, q) = [5p^2 \leq 18q^2]$, begin with $[1, 2]$. The exact trace is

k	$[\ell_k, u_k]$	m_k	$C_5(m_k)$
0	$[1, 2]$	$3/2$	yes
1	$[3/2, 2]$	$7/4$	yes
2	$[7/4, 2]$	$15/8$	yes
3	$[15/8, 2]$	$31/16$	no

and the retained bracket is

$$[15/8, 31/16].$$

Its width is $1/16 = (2 - 1)2^{-4}$. The certificate owns four cross-multiplication decisions and this bracket; it does not own the irrational equality boundary $\sqrt{18/5}$ as a represented rational point.

Worked example 2: malformed enclosure (computation). Take $[1, 3/2]$.

Both endpoints answer yes. A midpoint can still be computed, but there are no opposed endpoint bits, so the bisection contract stops with INVARIANT-FAILURE. Repeated halving of an arbitrary interval is not a crossing enclosure.

Worked example 3: opposed bits without monotonicity (external counterexample). Let

$$h(d) = -(d - 1)(d - 2)(d - 3), \quad C(d) = [h(d) \geq 0].$$

On $[1/2, 7/2]$, the lower endpoint is yes and the upper endpoint no, but three crossings lie inside. The predicate is not monotone. Binary slicing may retain a half containing one transition, yet it cannot claim a unique boundary or safely apply the downward-yes invariant globally. Opposed bits alone are

insufficient.

Endpoint projection is lossy. Define π_{final} to retain only the final (ℓ_N, u_N) . Distinct step histories can share that pair while differing in raw midpoint records, reductions, costs, or repeated checks. A question on final endpoints cannot recover those histories.

Worked example 4: same bracket, different ledger (computation).

One run may reduce every midpoint immediately; another may retain unreduced pairs and canonicalize only the final endpoints. If both branch decisions agree, their rational final brackets may be equal while their stored traces and costs differ. The audit must retain or explicitly erase canonicalization history; endpoint equality does not identify the algorithms.

Newton’s method is a different route:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

It requires a differentiable model, nonzero derivative at each used point, a starting value, and its own stopping argument. It need not preserve an enclosure. Quasi-Newton methods replace exact derivative information with updated approximations and require still different premises. Their speed or endpoint agreement does not transfer bisection’s opposed-bit invariant.

Worked example 5: Newton step leaves an interval (external counterexample). For $f(x) = x^3 - 2x + 2$, start at $x_0 = 0$. Then

$$x_1 = 0 - \frac{2}{-2} = 1, \quad x_2 = 1 - \frac{1}{1} = 0,$$

so the route cycles between 0 and 1. This does not make Newton invalid; it shows that tangent iteration and bisection slicing have different guarantees and failure certificates.

The canonical jar route is different again. It uses the seed-free continued-

fraction algorithm: Euclidean division-with-remainder produces whole partial quotients, and their convergents produce the ordered distance walls $17/9$ and $2/1$. It is not midpoint bisection, a raw mediant crawl, or a grid search. Those walls map to the canonical count-three report jar written [129.6, 137.7]. The rational distance bisection above is a legacy fuel-driven route. Neither route inherits the other's certificate; wall agreement needs an explicit comparison map.

The bisection audit is

$$\mathcal{B}_{\text{audit}} = (C_t, \text{domain}, \text{orientation}, \mathcal{B}_0, \text{monotonicity/enclosure evidence}, N, \text{proposed width-target extension})$$

The action ledger is exact. Domain, crossing question, initial bracket, orientation, arithmetic model, canonicalization, fuel, and any proposed width-target extension are *posited*. Midpoint identities, recursive trace-length law, replay-checker implications, invariant preservation, conditional width law, and final-bracket implications are *demonstrated*. Actual midpoints, cross-products, branch decisions, bracket trace, fuel use, widths, and stop are *read*. Exact root existence, uniqueness, Newton or quasi-Newton guarantees, continued-fraction wall identity, and physical slip meaning remain *reserved*. Bisection is iteration by slicing because every update keeps one evidenced half and records the one it discarded.

5.3 A report is not a certificate

A numerical routine may return data without establishing every claim a reader wants to attach to it. Separate four layers:

```

return ::= none(reason) | some(payload),
report  ::= report(schema, route, inputs, outputs, trace, stop),
validation ::= accepted | rejected(failures) | unresolved,
certificate ::= certificate(report, checks, claim, provenance).
```

An optional return distinguishes absence from a numerical zero or empty bracket. A report organizes returned data. Validation checks a declared schema and claim. Only an accepted validation can be packaged as a certificate for that specific claim.

The conversion chain is explicitly one-way:

raw bytes or text $\longrightarrow$ parsed report $\longrightarrow$ validated report $\longrightarrow$ certified claim.

Raw input does not imply successful parsing; parsing does not imply validity; validity does not imply every possible claim; and certification of one claim does not certify another.

If a Boolean validator V is used for property P , acceptance licenses $P(r)$ only when the soundness implication

$$V(r) = \text{true} \implies P(r)$$

has been demonstrated or is explicitly trusted or posited. The actual Boolean outcome is *read*. A validator name or true bit cannot supply its own soundness argument.

The reporting pipeline is finite:

1. acquire the optional return within budget,
2. parse schema and version,
3. validate domains, units, scales, and route identity,
4. replay transitions and arithmetic,
5. evaluate the requested claim and slice checks into pass/fail/unresolved,
6. emit a certificate only when every required check passes.

Stops are NONE, BUDGET, PARSE-ERROR, VERSION-MISMATCH, SCHEMA-UNKNOWN, REQUIRED-ENTRY-MISSING, UNIT-SCALE-ERROR, DOMAIN-ERROR, ARITHMETIC-RESOURCE, OVERFLOW, PARTIAL-REPORT, ROUTE-MISMATCH,

REPLAY-FAILURE, CERTIFICATE-MISSING, CERTIFICATE-INVALID, CLAIM-FAILURE, and COMPLETE. The receipt records raw return, parsing decisions, all validation rows, budget spent, checked scope, unresolved frontier, erasures, stop reason, and provenance. A failed or partial validation does not erase the report; it limits what the report licenses.

For a bracket report, validation questions include

$$\ell < u, \quad C_t(\ell) = \text{yes}, \quad C_t(u) = \text{no},$$

together with domain and arithmetic assumptions. If a trace is supplied, replay checks every midpoint, branch, adjacent bracket, fuel count, and final endpoint. If no trace is supplied, endpoint checks may still pass, but no claim about how the endpoints were reached is certified.

Worked example 1: a replayable bisection report (computation).

A four-step report contains

$$[1, 2] \rightarrow [3/2, 2] \rightarrow [7/4, 2] \rightarrow [15/8, 2] \rightarrow [15/8, 31/16].$$

The validator recomputes midpoints $3/2, 7/4, 15/8, 31/16$, checks their crossing bits, verifies every retained half, and confirms trace length four and final width $1/16$. If all rows pass, the certificate owns this finite bracket and trace. It does not own an exact represented crossing point.

Worked example 2: a malformed bracket report (computation).

Suppose a payload reports

$$\ell = 2, \quad u = 1.$$

Both values parse as positive rationals, but the order check fails. The result is a valid parsed report and a rejected bracket certificate. Swapping the endpoints during validation would silently change the returned data; a repair

must be a new, explicitly transformed report with its own provenance.

Worked example 3: absence is not a default value (computation).

If acquisition returns `none(BUDGET)`, the parser must not synthesize $[0, 0]$, a zero endpoint, or an empty trace and call it a reading. The receipt owns only the budget stop and unresolved claim. Such a `none` result is scoped to the scanned candidates, budget, and run configuration; it is not a global nonexistence statement. A later run may produce data, but it is a separate acquisition.

A summary projection also loses evidence. Let

$$\pi_{\text{endpoints}}(\text{full report}) = (\ell, u).$$

Distinct traces may share endpoints. Questions about endpoint order can be pulled through this projection, but questions about midpoint history, cost, or route cannot be answered after those entries are erased. Equal summaries do not identify full reports.

Worked example 4: jar ownership versus endpoint diagnostics (instrument computation). The canonical count-three report owns the ordered-wall jar

$$[129.6, 137.7].$$

Two finer route diagnostics report, at scale 10^{18} ,

$$d = 137011290548979455469, \quad s = 137011290753751157155,$$

with scaled difference 204771701686. Those diagnostics support a declared-resolution comparison, but neither replaces the jar. A valid report tags the continued-fraction jar route, direct route, and stepped route separately; shared digits do not merge their traces or costs.

The integers are unequal, yet coarse formatting maps both to the dis-

played string 137.011. This is a display collision, not equality of readings. A loss-auditable decimal or scaled-floor report retains the raw integer or exact rational, scale, formatting rule, displayed quotient, and discarded residue. For exact ratio A/B at scale M ,

$$n = \left\lfloor \frac{AM}{B} \right\rfloor, \quad \rho = AM - nB.$$

The floor erases ρ/B ; retaining A, B, M, n, ρ permits replay.

Worked example 4b: flags are not a certificate (computation).

A payload containing entries `verified = true` and `complete = true` may parse under a permissive schema. If the required certificate is absent, validation returns CERTIFICATE-MISSING; if its checks fail, it returns CERTIFICATE-INVALID. Self-reported flags are data, not validation evidence.

Worked example 5: same payload, different route (computation).

Suppose two routines return the rational pair $(3, 2)$. One obtained it by reducing $18/12$; the other read it directly from an input table. The endpoint payloads agree, but the first certificate may own a Euclidean remainder trace while the second owns a table lookup. A claim about value equality can pass; a claim about equal process or cost must fail or remain unresolved.

Physical reporting requires another provenance chain. A displayed voltage interval is not automatically a calibrated measurement interval. Physics owns sensor response, units, calibration transfer, environmental conditions, sampling plan, and uncertainty model. Numerical validation can certify formatting, arithmetic, and finite acquisition records; it cannot manufacture empirical coverage or physical interpretation.

A reproducibility receipt records the checker and compiler releases, dependency lock and versions, source-snapshot hash, run and build configuration, arithmetic model, platform, protocol version, run identifier and time, deterministic inputs, and every declared nondeterministic input—including seeds,

schedules, and environmental state where relevant. Reproduction is conditional on matching that declared provenance. Agreement under one configuration establishes neither physical reproducibility nor invariance across platforms, arithmetic models, compilers, or checker releases.

The report audit is

$\mathcal{R}$ = (raw bytes/text and optional return, encoding, schema/version, parser, validator/checker identity and release, soundness status, compiler release, dependency lock/version, source-snapshot hash, run/build configuration, arithmetic model, platform, protocol, run ID/timestamp, deterministic and declared nondeterministic inputs, route identity, units/scale, exact raw values, formatting rule, quotient, and residue, outputs, trace, budget, checked scope, unresolved frontiers, domain/arithmetic/replay checks, failures, claim syntax/evaluator, pass/fail/unresolved slices, summary projections, erasures, certificate presence and validity, certified claims, provenance).

The action ledger is exact. Schemas, parsers, validators, checkers, versions, routes, claim syntax, evaluators, budgets, required checks, toolchain choices, and run configuration are *posited*; validator soundness is also posited whenever it has not been proved. Parsing and validation invariants, replay implications, arithmetic identities, certificate composition rules, and any actually established soundness implication are *demonstrated*. Raw returns, parse and Boolean outcomes, reports, validation rows, exact values, formatting collisions, floor residues, route diagnostics, budgets spent, stops, and owned jar walls are *read*. Exact point ownership, process identity from equal payloads, claims outside the certificate, cross-toolchain invariance, physical reproducibility, empirical uncertainty coverage, and deterministic equivalence while nondeterministic inputs remain uncontrolled are *reserved*. A report may be useful when validation is partial; it becomes a certificate only for the claims its checks actually support.

Chapter 6

The Bracketed Number

6.1 Dyadic data structure

A dyadically refined bracket is finite data, not an approximate spelling of an unknown point. Write its primary representation as

$$D = (L, U, Q), \quad [D] = [L/Q, U/Q],$$

where $L, U \in \mathbb{Z}$, $Q \in \mathbb{N}_{>0}$, and $L < U$. The integers are stored exactly; the interpretation map supplies the scale. A crossing question C orients the endpoints:

$$C(L/Q) = \text{yes}, \quad C(U/Q) = \text{no}.$$

Order and opposed endpoint answers are separate invariants. Neither follows merely from the other, and neither says that a unique point lies inside the bracket. If a run starts at explicit denominator Q_0 , an optional rung index k derives $Q_k = 2^k Q_0$. Unless Q_0 itself is a power of two, the represented rationals need not be dyadic rationals in lowest terms; it is the refinement operation, not the initial rational grid, that is dyadic.

Dyadic refinement is a conversion followed by a slice. Convert both end-

points to the next common denominator,

$$(L, U, Q) \mapsto (2L, 2U, 2Q), \quad M = L + U,$$

so that $M/(2Q)$ is the exact midpoint. Reading $C(M/(2Q))$ chooses one half:

$$(L, U, Q) \mapsto \begin{cases} (M, 2U, 2Q), & C(M/(2Q)) = \text{yes}, \\ (2L, M, 2Q), & C(M/(2Q)) = \text{no}. \end{cases}$$

The retained interval has width $(U - L)/(2Q)$. One rung doubles the denominator and halves the interpreted width. The conversion is exact; only the predicate answer selects a slice.

Representation and value must not be confused. The triples $(1, 3, 2)$ and $(2, 6, 4)$ denote the same interval. A canonicalization divides L, U, Q by their positive greatest common divisor. Canonicalization preserves the interpreted interval but can erase the relation $Q_k = 2^k Q_0$, hence a refinement-history clue. It belongs either after replay or beside an unreduced trace, never in place of that trace.

An adjacent exact grid pair $a < b$ with common positive denominator Q ,

$$a = L/Q, \quad b = U/Q, \quad U - L = 1,$$

converts to $D = (L, U, Q)$. The conversion certifies common denominator, adjacency, and order. It becomes an oriented crossing bracket only after the separate evidence $C(L/Q) = \text{yes}$ and $C(U/Q) = \text{no}$ is read. Grid conversion alone is not crossing evidence.

The numerical method is a finite state transition. Given initial data D_0 , rung budget N , and arithmetic budget A , it produces

$$D_0 \xrightarrow{b_0} D_1 \xrightarrow{b_1} \cdots \xrightarrow{b_{r-1}} D_r, \quad r \leq N, \quad b_i \in \{\text{yes}, \text{no}\}.$$

Stops include COMPLETE, RUNG-BUDGET, ARITHMETIC-BUDGET, DOMAIN-

ERROR, ENDPOINT-NOT-OPPOSED, ORDER-FAILURE, and PREDICATE-UNRESOLVED.

A budget stop owns the r checked rungs and unresolved next transition; it does not assert that further refinement is impossible.

A replay certificate contains the initial triple, scale convention, predicate identity, arithmetic model, budgets, the explicit Q_0 , optional rung indices, every midpoint numerator, every read branch bit, every retained triple, and the stop. Its checker verifies $Q' = 2Q$, midpoint addition, half selection, order, opposed endpoint answers, fuel use, numerator gap, and the final laws

$$Q_r = 2^r Q_0, \quad U_r - L_r = U_0 - L_0, \quad \text{width}(D_r) = \frac{U_0 - L_0}{2^r Q_0}.$$

Checks are sliced into pass, fail, and unresolved rows. Passing arithmetic replay does not certify predicate soundness; passing endpoint checks without a trace does not certify the route.

Worked example 1: two exact rungs (computation). Let $D_0 = (1, 2, 1)$, so $Q_0 = 1$, with endpoint answers yes and no. The optional rung index is $k = 0$. The first midpoint is $3/2$. If its answer is yes, retain $D_1 = (3, 4, 2)$, denoting $[3/2, 2]$, with $k = 1$. Its next midpoint is $7/4$. If that answer is no, retain $D_2 = (6, 7, 4)$, denoting $[3/2, 7/4]$, with $k = 2$. The widths $1, 1/2, 1/4$ and both slice choices replay without rounding.

Worked example 2: denominator growth is not value growth (computation). The conversion from $(3, 4, 2)$ to $(6, 8, 4)$ changes neither endpoint. Reading the larger integers as unscaled values would invent $[6, 8]$. Reading only $Q = 4$ while discarding the numerators would lose the bracket. The triple is the data structure; none of its fields is a reading by itself.

Counterexample 3: equal endpoints do not certify equal histories (computation). One run may reach $(6, 7, 4)$ by the two rungs above. Another may receive it as initial data. Their endpoint projections agree, but

their transition traces and costs do not. The projection

$$\pi(L, U, Q, \text{trace}) = (L, U, Q)$$

supports interval equality while erasing route, branch sequence, and budget use. Equal projections cannot be pulled back into equal processes.

Counterexample 4: an unordered payload is not repaired by interpretation (computation). The triple $(5, 3, 4)$ fails $L < U$. Sorting it during validation would create a different report and could reverse the required endpoint answers. The original is rejected; an explicitly transformed triple requires new provenance and fresh endpoint checks.

This dyadic route is legacy refinement data. It must not be renamed as, or substituted for, the canonical seed-free continued-fraction route. That route performs Euclidean division with remainder, reads whole partial quotients, and constructs ordered walls; for the owned count-three result its walls $17/9$ and 2 induce the jar $[129.6, 137.7]$. It is not denominator doubling, midpoint selection, or adjacent-grid conversion. A dyadic endpoint may numerically equal a wall, but equality of a rational value does not identify algorithms, traces, budgets, or ownership claims. The continued-fraction jar remains canonical; a dyadic run is a separately labelled diagnostic.

Governance fixes the initial triple, explicit Q_0 , optional rung-index convention, predicate question, orientation, scale convention, canonicalization policy, budgets, stop vocabulary, checker release, arithmetic model, and protocol version before a run. Provenance records those choices together with run identity, time, platform, deterministic inputs, declared nondeterministic inputs, raw branch answers, and every loss-causing projection. Changing a governed item starts a new run rather than silently extending the old certificate.

The action ledger is exact. Initial data, predicate, orientation, budgets, arithmetic model, canonicalization policy, and certificate grammar are

posited. Midpoint identities, the laws $Q' = 2Q$ and $Q_r = 2^r Q_0$, numerator-gap and width laws, invariant-preservation implications, canonicalization equality, and replay-checker implications are *demonstrated*. Endpoint answers, branch bits, visited triples, budget use, stops, and checker outcomes are *read*. A unique enclosed point, predicate soundness not separately established, route identity from equal endpoints, continuation beyond a stop, and ownership of the canonical continued-fraction jar remain *reserved*.

6.2 Resolution as an explicit parameter

Resolution is declared data, not a universal constant inferred from finite output. Four controls must remain distinct:

$$\theta = (\mathcal{X}, \mathcal{A}, q, s, f; G, N, M, K),$$

where $\mathcal{X}$ is the input domain, $\mathcal{A} = \{a_0, \dots, a_{m-1}\}$ a finite alphabet, $q : \mathcal{X} \rightarrow \mathcal{A}$ a quantizer, $s > 0$ its scale, and $f \geq 0$ a floor. The grid parameter G selects a finite exact grid; N bounds dyadic refinement rungs; $M > 0$ is the integer display scale; and K counts whole continued-fraction partial quotients. Each affects a different route. The value $G = 0$ is a sentinel meaning that no ordinary grid has been selected; it is not a grid of zero width and cannot enter division. Its cells are $B_j = q^{-1}(a_j)$. Boundary conventions are required: a value on a cell wall cannot be assigned by an unstated tie rule.

For a uniform scalar quantizer with origin o , width $\Delta > 0$, and code range $\{j_{\min}, \dots, j_{\max}\}$, one grammar is

$$q(x) = j \iff o + j\Delta \leq x < o + (j+1)\Delta,$$

with separate out-of-range stops. Numerical resolution is Δ , while s converts code units to reported units. A floor may stop refinement when $w_{r+1} < f$. This is a stopping policy, not evidence that nature contains no smaller

distinction.

Rungs count transitions; cells partition a domain; scale interprets codes; the floor bounds admitted refinement; and the quantizer chooses a cell. Grid selection changes available exact cells, N changes dyadic depth, M changes display quotient and residue, and K changes continued-fraction walls. Collapsing these parameters hides which operation lost information and falsely merges route ownership.

Parameter-relative equality is the equivalence relation

$$x \equiv_{\theta} y \iff q_{\theta}(x) = q_{\theta}(y).$$

Reflexivity, symmetry, and transitivity follow from equality of labels; equality of underlying values does not. A specification θ' refines θ when a map $c : \mathcal{A}_{\theta'} \rightarrow \mathcal{A}_{\theta}$ satisfies $q_{\theta} = c \circ q_{\theta'}$, equivalently every fine cell lies inside its indicated coarse cell. Refinement can separate a former equivalence class but cannot justify comparisons if route, scale, or boundary conventions are incompatible.

Given rung budget N , predicate budget P , and arithmetic budget A , computation returns a finite trace and stops with COMPLETE, RUNG-BUDGET, PREDICATE-BUDGET, ARITHMETIC-BUDGET, BELOW-FLOOR, OUT-OF-RANGE, BOUNDARY-UNRESOLVED, or INVALID-PARAMETERS. A complete stop vocabulary also includes CELLS-EXHAUSTED, VERSION-MISMATCH, OVERFLOW, and UNRESOLVED-FRONTIER. A certificate records θ , budgets, raw inputs, boundary decisions, transitions, resources spent, final cell or bracket, and stop. Replay checks parameter domains, membership, scale conversion, transitions, resource counts, and the stop inequality. Checks are sliced into pass, fail, and unresolved rows; a budget stop certifies completed work, not global indistinguishability.

Quantization is a many-to-one projection. If $x, y \in B_j$, then $q(x) = q(y)$, but not necessarily $x = y$. Retaining only a_j erases within-cell order and distance as well as quantizer version, scale, boundaries, budgets, and stop.

Equal summary symbols cannot identify equal readings across undeclared configurations.

Worked example 1: equivalence changes under refinement (computation). Let display quantization use $q_M(x) = \lfloor Mx \rfloor$, and take $x = 1.2341$, $y = 1.2358$. Then $q_{100}(x) = q_{100}(y) = 123$, so $x \equiv_{\theta_{100}} y$, whereas $q_{1000}(x) = 1234$ and $q_{1000}(y) = 1235$. On the aligned declared domain, the coarse-label map $c(j) = \lfloor j/10 \rfloor$ satisfies $q_{100} = c \circ q_{1000}$. Thus the same floor convention gives a genuine refinement: the coarse quantizer identifies the values and the fine quantizer separates them.

Counterexample 2: a wall without a tie rule (computation). On the same grid, $x = 1/2$ lies exactly on a wall. Different tie conventions can assign different symbols. Reporting only code 2 hides the governing choice. Without the convention, validation returns BOUNDARY-UNRESOLVED, not a repaired code.

Worked example 3: finite rungs versus a declared floor (computation). An initial width 1, halved per rung, has width 2^{-r} . For $f = 1/16$, a policy forbidding the next width below f permits four halvings and stops before the fifth. With $N = 3$, the run instead stops at width $1/8$ by RUNG-BUDGET. Distinct resource and floor stops prove no empirical minimum length.

Counterexample 4: output symbols do not supply a probability law (computation). For a declared distribution $p_j = \Pr(q(X) = a_j)$, Shannon entropy is

$$H(q(X)) = - \sum_{j:p_j>0} p_j \log_2 p_j.$$

Four declared equiprobable symbols give $H = 2$ bits. Four possible symbols do not establish equiprobability, hence do not establish two bits. Alphabet

size gives the ideal maximum $\log_2 m$, the entropy bound $H \leq \log_2 m$ for a random variable with a declared law, and a fixed-width label cost $\lceil \log_2 m \rceil$. For $m = 8$, fixed-width labels require 3 bits, while $H = 3$ only for a declared uniform law on all eight symbols. An occupied alphabet observed in one sample is not the possible alphabet, and neither supplies probabilities by itself.

Counterexample 5: a resource limit is not uncomputability (computation). Fix a universal prefix machine U and define

$$K_U(x) = \min\{|p| : U(p) = x\}.$$

This function is generally uncomputable. An exhibited program of length n proves only $K_U(x) \leq n$; absent a separate lower-bound demonstration it does not establish incompressibility. Likewise, a search stopping after 10^6 transitions owns only that resource stop. These facts do not make an empirical calibration constant uncomputable. The count-three jar is not thereby random, incompressible, or empirically uncomputable.

For the canonical $K = 3$ route, whole partial quotients produce walls 17/9 and 2; the owned resolution statement is the ordered-wall jar

$$[129.6, 137.7].$$

That finite jar is the floor owned by the declared route and count. Finer direct, stepped, M -formatted, G -grid, or N -dyadic diagnostics may expose narrower cells, but neither replace the jar nor change its walls. Coincident displayed values license only a declared-resolution comparison; route and process ownership remain separate.

The physics fence is strict. Numerical cell width, rung floor, or alphabet is not by itself detector resolution, noise, quantum uncertainty, repeatability, or uncertainty coverage. Those require a sensor model, units, calibra-

tion transfer, sampling procedure, environmental controls, and empirical evidence. Numerical quantization models physical acquisition only after that correspondence is posited and tested.

The numerical audit first checks route and parameter compatibility: G only for grid construction, N only for dyadic rungs, M only for display projection, and K only for whole-partial-quotient walls. It retains every raw integer or rational before projection. For each scaled display it records

$$n = \left\lfloor \frac{AM}{B} \right\rfloor, \quad \rho = AM - nB,$$

so quotient and remainder expose the loss. It records cells before and after refinement, the fine-to-coarse map, compatibility checks, pass/fail/unresolved slices, spent budgets, stop, checked scope, and unresolved frontier.

Governance fixes domain, alphabet, walls, boundary convention, quantizer, scale, floor, G, N, M, K , route assignment, budgets, stops, arithmetic model, any distribution used for entropy, universal machine used for complexity, and certificate checks. Provenance records checker and compiler releases, dependency lock and versions, source-snapshot hash, run/build configuration, protocol version, run identifier and time, arithmetic model, platform, deterministic inputs, declared nondeterministic inputs, raw branch decisions, and every projection. A changed parameter creates a new report.

The action ledger is exact. Quantizer, cells, scale, floor, G, N, M, K , route assignment, budgets, probability law, universal machine, physical correspondence, and certificate grammar are *posited*. Membership and refinement implications, equivalence laws, scale arithmetic, quotient–remainder identity, transition and width laws, stop inequalities, replay implications, entropy from a declared distribution, and exhibited complexity upper bounds are *demonstrated*. Inputs, codes, raw rationals and integers, quotients and remainders, boundary decisions, visited cells, resources, stops, jar walls, and checker outcomes are *read*. Equality within one code, a universal minimum distinc-

tion, empirical entropy without a distribution, a complexity lower bound not proved, randomness or uncomputability of the jar, replacement of the canonical jar by diagnostics, and physical resolution or uncertainty not separately established remain *reserved*.

6.3 Stop rules and budgets

A finite numerical method must say which mechanism stopped. Five mechanisms remain separate:

fuel recursion	performs at most N recursive transitions,
finite grid scan	returns the first adjacent yes–no pair or exhausts a declared grid,
continued fraction	reads exactly K whole partial quotients,
bounded live probe	returns a result, absence, or fallback within a live effort bound,
structural outgrowth	applies a deterministic reference-plus-deadband Boolean rule.

Fuel, cells visited, arithmetic operations or bit growth, checker effort, and wall time are distinct currencies. None converts to another without a separately calibrated conversion model. In particular, elapsed time cannot be silently substituted for structural fuel.

Fuel recursion carries $F \in \mathbb{N}$ and decrements it on every admitted recursive step, so that governed run terminates in at most its initial fuel. This establishes totality of the fuelled wrapper only. Turing’s result says there is no total algorithm deciding, for every arbitrary program and input, whether the unbounded computation halts; a finite wrapper neither decides that question nor proves nontermination when fuel expires.

A grid scan owns only its declared ordered finite cells. It stops at the first adjacent pair whose read labels are yes then no, or with CELLS-EXHAUSTED after the final cell. Exhaustion is scoped to that grid, ordering, predicate, and budget. Continued-fraction count K is different again: $K = 3$ owns the canonical walls $17/9$ and 2 , hence the jar $[129.6, 137.7]$. Neither grid cells nor

recursion rungs may be substituted for K .

A bounded live probe distinguishes $\text{some}(r)$, $\text{none}(\text{scope})$, and $\text{fallback}(r')$. Resource failures such as effort, memory, arithmetic growth, or timeout are not semantic failures such as invalid domain, false predicate, malformed result, or violated invariant. Structural outgrowth is not a live probe. Given candidate x , scaled reference r , deadband or noise calibration δ , and a declared orientation, its deterministic Boolean has the generic form

$$O_{r,\delta}(x) = [x \text{ exceeds the scaled reference } r \text{ under the declared deadband comparison}].$$

The receipt retains x, r, δ , the scaling rule, calibration, orientation, comparison, and Boolean. No metric or symmetric comparison is implied without separate evidence. The output cannot replace an observation, search result, or live absence.

Stop predicates are checked both before and after each transition. The stop grammar distinguishes

FUEL	$F = 0$ before another required transition,
BUDGET	a declared resource counter is exhausted,
TOLERANCE	$e(x) \leq \tau$ for a declared estimator and tolerance,
BRACKET-WIDTH	$u - \ell \leq \omega$ for a declared width target,
EXACT	an exact target condition has an executable resolver,
DOMAIN	the next operation is outside its admitted domain.

Tolerance requires an estimator and a demonstrated implication relating it to error. Bracket width is an interval fact, not point accuracy. Exact stop needs a resolved predicate. Domain stop rejects an operation, not reports an approximation.

Every rule is tagged *source-live* or *proposed*. Source-live rules govern the owned trace. Proposed rules describe an alternative experiment and cannot reinterpret the source-live result. A new width target, extra fuel, or changed

live bound yields a separate route and certificate. Structural and live rules also carry distinct tags and cannot substitute for one another.

Several predicates can fire at one state. Governance therefore declares a priority order or a combined-stop policy before the run. The receipt records every fired predicate, the selected stop, and a certificate that selection follows the declared priority. Recording only the selected reason erases a potentially material simultaneous stop.

A replay certificate contains initial state, transition relation, fuel, every live bound and counter, stopping-rule tag, structural/live tag, states and branch decisions, checked scope, unresolved frontier, and final status. Replay verifies pre- and post-checks, each transition, monotone fuel decrease, resource accounting, all fired predicates, and priority selection. Validation is sliced into transition, arithmetic, invariant, resource, stop, and claim rows, each pass, fail, or unresolved. Passing the stop row does not repair a failed invariant.

Worked example 1: exact fuel traces (computation). Starting from $[1, 2]$, read half selections give

$$N = 3 : [15/8, 2], \quad w = 1/8; \quad N = 4 : [15/8, 31/16], \quad w = 1/16.$$

These are distinct fuelled runs. With width tolerance $\varepsilon = 1/10$, the post-transition check first fires after rung four, because $1/8 > 1/10$ and $1/16 \leq 1/10$.

Counterexample 2: width is not tolerance (computation). Suppose $[\ell, u]$ has width 10^{-6} , while a claimed point estimate has no proved relation to the enclosed target. The width rule permits BRACKET-WIDTH, not a tolerance claim about that point. Renaming the stop would add an implication absent from the certificate.

Worked example 3: simultaneous stops (computation). Suppose run four both exhausts the arithmetic budget and reaches width tolerance $1/10$. The fired set is $\{\text{BUDGET}, \text{TOLERANCE}\}$. If declared priority is semantic success before resource exhaustion, the selected stop is TOLERANCE; under the reverse priority it is BUDGET. Either certificate must retain both fired predicates and verify the selected priority, rather than pretending the other event did not occur.

Counterexample 4: deterministic recipe, nondeterministic input (computation). Two reruns share a transition grammar but receive different schedules or unordered streams. They may produce different valid traces. Repeatability therefore does not mean identical results regardless of conditions. It means deterministic and declared nondeterministic inputs, versions, configurations, and rules are recorded so matching runs can be compared and their certificates replayed.

Counterexample 5: finite fuel does not solve universal halting (computation). Wrapping every run in $F = 10^6$ guarantees the wrapper stops. A fuel stop says only that the simulated process did not finish within that fuel. It neither decides whether the unbounded process eventually halts nor turns an unresolved run into nontermination.

Repeatability is a governed relation between reports. Reports are comparable when input, transition and stop grammars, arithmetic model, releases, configurations, deterministic conditions, and declared nondeterministic conditions match. Exact agreement can then be read if it occurs; disagreement is also reportable. Neither establishes physical repeatability, which requires repeated acquisition, a sensor and noise model, calibration, sampling controls, units, environmental records, and statistical analysis.

The audit tuple is

$\mathcal{S} =$ (raw input, initial state, route, source-live/proposed tag, structural/live tag, transition grammar, fuel initial/spent/remaining, grid/cells/order/scope, continued-fraction count/walls, live probe, resource versus semantic status, resource currencies, calibrated conversions, live bounds/count, pre/post checks, all fired stops, priority, selected stop/priority certificate, tolerance/width/exhaustion, states, branches, invariants, stop, checked scope, unresolved frontier, validation slices, checker/compiler releases, dependency versions, snapshot hash, configuration, protocol, run identifier, platform, deterministic inputs, declared nondeterministic inputs, projections).

Governance freezes these fields before acquisition; a changed bound, rule, or condition starts a new report.

The action ledger is exact. Transition grammar, fuel, grid scope, K , live bounds, structural rule, stop priority, tolerances, width targets, resource conversions, arithmetic model, and comparison policy are *posited*. Fuel termination, resource-accounting identities, grid-exhaustion implications, structural Boolean evaluation, transition invariants, conditional stop implications, priority selection, replay implications, and matching-provenance comparisons are *demonstrated*. States, cells, walls, counters, probe variants, branches, fired and selected stops, spent currencies, timing, validation rows, disagreements, and owned jar walls are *read*. Universal halting, global absence from bounded grid or probe scope, semantic failure from resource failure, live evidence from a structural Boolean, point accuracy from bracket width, completion beyond a budget, identical output under uncontrolled conditions, and physical repeatability remain *reserved*.

The handoff is intentional: the canonical count-three route retains its owned jar [129.6, 137.7] at its source-live stop. Later proposed refinements may diagnose it under new budgets, but do not overwrite its trace, stop, or ownership.

Part III

The Faces

Chapter 7

The Number Splits Into Faces

7.1 Computation names

A computational name is licensed by a grammar, not by resemblance to a word used elsewhere. Its record is

$\mathcal{N} = (\text{name, domain, codomain, route, version, inputs, operation, units, budget, stop, certificate, proj})$

Two records may share a spelling or a number without sharing an operation. Conversely, the same operation may admit several presentations. The record, rather than resonance, decides what a name owns.

Count. A count is $n \in \mathbb{N}$ produced by an explicit enumeration or transition rule. Its units name what is counted: cells, rungs, partial quotients, candidates, or events. Failures include invalid domain, exhausted scope, duplicate membership, overflow, and unresolved enumeration. Its certificate contains the finite domain or trace, membership rule, order when relevant, accumulator transitions, scope, and stop. A count of three partial quotients is not three grid cells merely because both are written 3.

Bit. A bit is an element of a declared two-symbol alphabet, such as $\mathbb{B} = \{\text{no}, \text{yes}\}$, obtained by a named predicate or encoding map. Its unit is one symbol; information in bits requires a separate probability law or coding convention. Failures include unresolved predicate, invalid input, unknown encoding, and missing orientation. Its certificate retains input, predicate, alphabet, orientation, output, and assumptions. A Boolean answer alone is neither a probability nor physical spin.

Cost. A cost is a nonnegative resource vector accumulated in declared currencies:

$$\mathbf{c}_{i+1} = \mathbf{c}_i + \mathbf{w}(s_i \rightarrow s_{i+1}), \quad \mathbf{c} = (\text{requested}, \text{spent}, \text{remaining}).$$

The currency may be transitions, arithmetic operations, bit growth, cells visited, checker effort, or elapsed time. Conversions between currencies require calibration; no scalar total exists without declared conversion weights. Failures include budget exhaustion, overflow, unknown weight, and incomparable currencies. A certificate replays weights and addition along the trace; it does not promote elapsed time to structural work.

Bracket. A bracket is ordered endpoint data (ℓ, u) with $\ell < u$, plus whatever opposed endpoint evidence its claim requires. Its units are endpoint units; its width has the same units. Operations include exact midpoint slicing, grid conversion, or continued-fraction wall construction, each with a distinct route. Failures include order, domain, non-opposed endpoints, arithmetic resource, and unresolved branch. A certificate owns its route, endpoints, trace, budgets, and stop, not an unnamed exact point.

Remainder. For integers $a, b > 0$, Euclidean division produces the unique pair

$$a = qb + r, \quad 0 \leq r < b.$$

The remainder is r , in the same integer unit as a and b . Division by zero, sign-convention mismatch, overflow, and an unchecked identity are failures. Its certificate retains a, b, q, r and verifies both equality and range. A discarded display residue or an empirical leftover may use similar language, but it is not this remainder without this operation.

Worked example 1: one numeral, different counts (computation).

The canonical count 3 means three whole continued-fraction partial quotients. That route produces ordered walls $17/9$ and 2, hence the owned jar $[129.6, 137.7]$. It does not mean three grid cells, dyadic rungs, branch bits, or any physical count. Those may also contain the numeral 3, but their units, transitions, stops, and certificates differ.

Worked example 2: a branch bit drives iteration and slicing (computation). For ordered bracket $[\ell, u]$, compute $m = (\ell + u)/2$. The read bit $C(m)$ selects either $[m, u]$ or $[\ell, m]$. Iteration consumes the bit; slicing discards one half. The next bracket certificate must retain the midpoint, bit, retained half, erased half, and endpoint invariant. Keeping only the bit loses the numerical state; keeping only endpoints loses the branch history.

Counterexample 3: equal remainders do not identify divisions (computation). Both $17 = 3 \cdot 5 + 2$ and $23 = 3 \cdot 7 + 2$ have remainder 2. The projection $\pi(a, b, q, r) = r$ has many-element fibers. Equality after π licenses equality of remainders only, not equality of dividends, divisors, traces, costs, or provenance.

Counterexample 4: equal brackets do not identify processes (computation). One trace may receive $[\ell, u]$ as input; another may reach it after several slices. Equal endpoint pairs certify the same bracket value under matching units, but not equal trace, route, requested resources, spent

resources, or remaining resources. Process and cost identity require their own certificate rows.

This manuscript coins *nowtrino* for one scientifically earned computation name:

$$\text{nowtrino}(h) = \pi_{\text{now}}(h) = (\text{current state, current counters, current stop}),$$

the computation-owned current-summary projection after history slicing. Its domain is a full finite history h . Its fiber over a summary contains every history with that same current state, counters, and stop. The projection therefore loses prior states, branch order, rejected slices, intermediate costs, and causal route unless separately retained. A nowtrino is not a particle, measurement, or physical hypothesis; it is this manuscript's name for a lossy computational summary.

For example, let $H_1 \neq H_2$ contain different earlier branches but end with the same current state, counters, and stop. Then $\text{nowtrino}(H_1) = \text{nowtrino}(H_2)$. Equality of summaries is exactly the projection claim; it cannot recover $H_1 = H_2$.

The spelling *naotrino*, coined from Greek *naos* (temple), is reserved for a later physical apparatus or model and earns no meaning here. The etymology records the manuscript coinage; it supplies no physical evidence. If later defined, it must state observables, units, calibration, dynamics, uncertainty, and empirical tests. It must also be distinguished explicitly from the standard neutrino of particle physics; word resonance establishes no relation. Any attempted coercion from nowtrino to naotrino or neutrino is rejected as a domain and route mismatch.

Name resolution is finite:

1. parse each name occurrence and its local declaration,
2. join on (name, domain, route, version) and require one row,
3. verify inputs, certificate, and status,
4. classify the occurrence as computation, physics, interpretive, reserved, ambiguous, or rejected

Stops include unknown name, missing key, version mismatch, multiple-row collision, domain mismatch, route mismatch, invalid certificate, ambiguous occurrence, rejected coercion, budget, and complete. The receipt retains candidates examined, checked scope, unresolved frontier, collisions, selected row, validation slices, and provenance. A lexical concordance can find equal spellings; it establishes neither equal semantics, independent evidence, nor completeness of the name ledger.

The name-provenance ledger records the full row $\mathcal{N}$, coinage or external origin, first definition, data schema, failure vocabulary, fiber description, and resolution history. Governance freezes those fields and requires a new version when a meaning changes. The numerical audit also records raw inputs, each iterative transition and slice, requested/spent/remaining resource vectors, stop, validation rows, checked scope, unresolved frontier, collisions, checker release, arithmetic model, configuration, protocol, and run identity.

The action ledger is exact. Alphabets, predicates, count domains, cost currencies and conversions, bracket routes, naming records, resolver keys and classifications, nowtrino projection, budgets, and certificate grammars are *posited*. Enumeration identities, Boolean-driven slice implications, resource-vector sums, bracket invariants, Euclidean uniqueness, projection fibers, unique-row resolution, and replay implications are *demonstrated*. Counts, bits, costs, endpoints, quotients, remainders, histories, summaries, resolver candidates, collisions, stops, and validation outcomes are *read*. Meaning from resonance, semantic independence or completeness from lexical concordance, process or cost identity from equal values, recovery of sliced history, physical interpreta-

tion of nowtrino, coercion to naotrino or neutrino, any meaning of naotrino before its physical definition, and identification with a standard neutrino remain *reserved*.

7.2 Physics names

A physical name is earned by an apparatus, model, observable, units, calibration, uncertainty statement, and data:

$\mathcal{P} = (\text{name, system, apparatus, model, observable, units, calibration, data, uncertainty, failures, certifi}$

Shared spelling with a computational object supplies none of these.

Computational projections called lower, upper, or binary may be labelled charge, mass, or phase only as an explicitly declared interpretation. The labels do not make the projections satisfy $F = ma$, $\mathbf{F} = q\mathbf{E}$, a phase model, dimensional laws, or any apparatus response. Likewise, α_c is computation-owned: it denotes the dimensionless ordered-wall jar [129.6, 137.7] under its declared route. It is neither a laboratory determination nor the fine-structure constant of physics. Saying that it was generated before it was read records provenance order only; temporal order does not validate a physical interpretation.

Mass. In an inertial experiment, $F = ma$. Calibrated force and acceleration observations estimate m in kilograms. Alignment, friction, sensor bias, timing, model inadequacy, and nonidentifiability are failures. The certificate retains calibration, observations, fit, residuals, uncertainty, and validity range.

Charge. Charge q , in coulombs, may be inferred from $\mathbf{F} = q\mathbf{E}$. Apparatus must measure force and field orientation. Background fields, leakage, sign ambiguity, saturation, and nonuniformity limit the claim. Geometry, field

map, zeroing, sign convention, data, corrections, and uncertainty belong in the certificate. The equation alone may be nonidentifying: for any admissible $\lambda \neq 0$, the pairs

$$(q, \mathbf{E}), \quad (\lambda q, \mathbf{E}/\lambda)$$

produce the same $\mathbf{F}$. The report fiber over one force therefore contains multiple charge–field pairs unless another calibrated channel fixes the scale. Dimensions are necessary consistency checks, not sufficient identification evidence.

Phase. For $y(t) = A \cos(\omega t + \phi)$, phase ϕ is in radians modulo 2π , relative to a declared clock and oscillator. Drift, aliasing, low amplitude, cycle slips, and reference changes are failures. Circular uncertainty and reference epoch are required.

Field. A field such as $\mathbf{E} : \Omega \times \mathbb{R} \rightarrow \mathbb{R}^3$, in volts per metre, is sampled only at finite locations, times, orientations, and bandwidths. Interpolation is a declared model. Drift, spatial averaging, boundaries, missing samples, and residuals limit the certificate.

Coupling. In $y_i = gx_i + \epsilon_i$, coupling g has units y/x , unless normalization makes it dimensionless. Both channels, excitation range, error covariance, backgrounds, and model selection require calibration. Raw pairs, estimator, residuals, uncertainty, and domain are retained.

Worked measurement 1: mass (physics). Apply three calibrated forces to a carriage and measure repeated accelerations. A weighted linear fit estimates m , and residuals test the model. Numerical iteration may solve the fit and slicing may form an interval, but convergence alone is not a mass certificate: friction, timing, force calibration, and the inertial model remain physical requirements.

Worked measurement 2: phase (physics). Sample a sinusoid against a traceable clock. Numerical search may iterate over ϕ and retain a likelihood slice. Changing the reference epoch shifts phase without changing the waveform, so the physical name includes its reference and modulo convention.

Counterexample 3: a slope is not yet a coupling (physics). Two digitized channels yield slope 2.0. If one channel used millivolts rather than volts, the physical slope changes by 10^3 . The numeral cannot name a coupling until units, calibrations, model, and uncertainty are supplied.

Counterexample 4: samples are not a continuous field (physics). Equal readings at two probes do not exclude variation between them. Interpolation or smoothness can fill the gap only as a model assumption. Numerical slicing bounds parameters within that model; it cannot observe unsampled points.

The manuscript spelling *naotrino* remains reserved. No concrete apparatus, model, observable, units, calibration chain, uncertainty analysis, or discriminating dataset is supplied here. It is not the computation-owned nowtrino and not the standard neutrino of particle physics. A later coinage requires a complete $\mathcal{P}$ record and evidence against alternatives.

Numerical acquisition preregisters the observable, sampling plan, blinding rule, preprocessing, estimator, protocol and version, and budgets before it parses calibrated samples. It validates units, iterates the declared estimator, slices a parameter region, and reports pass, fail, or unresolved checks. Raw and rejected samples are preserved with their rejection reasons; blinding is lifted only by the preregistered rule. Stops include acquisition budget, missing calibration, unit mismatch, saturation, domain failure, nonidentifiability, iteration budget, overflow, tolerance, complete, and model rejection. A numerical stop is not a physical failure absent an explicit apparatus relation.

Different named estimates can share data, calibration standards, preprocessing, environmental disturbances, or fitted parameters. Their covariance

and cross-covariance must therefore be estimated or bounded under the joint acquisition model. Distinct names, equations, or report columns establish no statistical independence. Treating correlated channels as independent can shrink uncertainty without new evidence.

The physics audit records apparatus, geometry, observable, units, calibration standards and dates, raw data, sampling plan, environment, equations, identifiability, estimator, iteration trace, retained and rejected slices, residuals, uncertainty construction, covariance and cross-covariance, shared data/calibration/preprocessing, systematic-error budget, preregistration, blinding, protocol/version, numerical budget, preserved rejected samples, stops, unresolved frontier, alternative models, and provenance. Governance freezes these before certification; a changed calibration, model, or apparatus starts a new report.

The action ledger is exact. Apparatus model, observables, units, calibration, sampling, estimator, uncertainty method, and validity domain are *posited*. Dimensional consequences, conditional model implications, calibration transformations, iteration identities, slice membership, uncertainty calculations, identifiability implications from additional calibrated channels, covariance propagation, and preregistered selection implications are *demonstrated* when their assumptions are checked. Sensor outputs, calibration observations, samples, fits, residuals, stops, environmental records, shared inputs, rejection reasons, protocol versions, covariance estimates, and unblinding events are *read*. Model truth beyond tested scope, unsampled values, independence from distinct names, identification from dimensions alone, physical meaning for computational projections or α_c , meaning from provenance order, physical meaning for naotrino, and identification with nowtrino or a standard neutrino remain *reserved*.

7.3 Same number or same process

Comparison begins by declaring its relation. For reduced rationals,

$$a/b =_{\text{exact}} c/d \iff ad = bc, \quad x =_{\theta} y \iff q_{\theta}(x) = q_{\theta}(y)$$

defines exact equality and equality under declared quantizer q_{θ} . Exact equality implies resolution equality only inside its domain; resolution equality does not imply exact equality.

A process report is

$$P = (\text{inputs}, \text{algorithm}, \text{transitions}, \text{trace}, \text{cost vector}, \text{budgets}, \text{stop}, \text{configuration}, \text{provenance}).$$

Process equivalence requires comparison maps for every component: compatible inputs, transition semantics, trace correspondence, cost currencies, budgets, stops, configuration, and provenance. Output equality is only one row. Shared names imply none of them.

Value and process provenance remain separate:

$$\text{raw}_i \xrightarrow{q_{\theta_i}} \text{label}_i, \quad \text{input}_i \xrightarrow{P_i} \text{output}_i.$$

A value comparison map cannot fill absent input, transition, trace, or cost maps.

Worked comparison 1: direct and stepped endpoints (computation). At scale 10^{18} ,

$$d = 137011290548979455469, \quad s = 137011290753751157155.$$

The integers differ by $|d - s| = 204771701686$, while a declared coarse display maps both to 137.011. Exact inequality, declared-resolution equivalence, and process difference are separate rows. The shared formatted string is a

formatter collision, not equality. Direct and stepped routes have different transitions and costs.

A declared inverse-report map takes continued-fraction walls $17/9$ and 2 to jar $[129.6, 137.7]$. The map and calibration belong in the certificate. Wall rationals and report endpoints inhabit different numerical domains; they are not the same numbers.

Worked comparison 2: same operation, different inputs (computation). Division of 17 by 5 and 23 by 7 uses the same operation and returns remainder 2 . Inputs and quotient identities differ. The runs share an algorithm schema but are not the same process.

Counterexample 3: same value, different algorithms (computation). One route reduces $18/12$ by remainders; another reads $3/2$ from a table. Exact outputs agree, but traces, failures, costs, and certificates differ. Value equality passes while process equivalence fails or is unresolved.

Counterexample 4: shared names provide no bridge. A nowtrino is a computation-owned summary. A naotrino is a reserved possible physical name, while a standard neutrino belongs to established physics. No comparison map, calibration, or model bridge is supplied. Resemblance cannot construct one.

Comparison is a finite join. Parse two reports; join declared relation, domain, route, version, units, and configuration; then evaluate exact-value, resolution, input, transition, trace, cost, stop, and provenance slices. Each is pass, fail, or unresolved. Stops include missing report, incompatible domain, unit or version mismatch, unknown quantizer, route or trace mismatch, cost-currency mismatch, provenance mismatch, ambiguous name, rejected bridge, budget, and complete. A budget leaves an explicit unresolved frontier.

Endpoint projection erases trace and cost; display projection erases residue; current summary erases history; name projection erases domain and route

unless keyed. A certificate states which rows passed and which erasures prevent stronger claims.

Same-name means resolution to one governed ledger row—name, domain, route, version, and status—not equal spelling. Typed value paths are

$$\text{Input}_i \rightarrow \text{Report}_i \xrightarrow{\kappa_i} C \xrightarrow{q_\theta} \text{Label}_i.$$

Each κ_i converts to common comparison domain C . Process comparison instead uses an explicit trace/state relation, not κ_i . Byte-for-byte trace equality is strong; a weaker relation must state which states, transitions, stuttering steps, and costs it identifies. Without one, process equivalence is unresolved.

Worked comparison 5: finite three-state closure (computation).

Let $S = \{\text{minus}, \text{zero}, \text{plus}\}$ and declare the deterministic rotation

$$\text{minus} \mapsto \text{zero} \mapsto \text{plus} \mapsto \text{minus}.$$

Finite case reduction certifies for every $s \in S$ that three rotations return s , while one and two rotations do not. The displayed finite table suffices under the declared transition. Its scope is exactly these three states. Closure does not make an unrelated residue a one-step invariant or fixed point. Only a separately defined residue map with a proved preservation law could do that. After the three-step return, any read defined solely from the returned state agrees with its initial state-derived read.

Worked comparison 6: reproducible tuples are not cycle causation (computation). Each of two governed runs returns the exact tuple

$$(573, 573, 552),$$

with residual $552 - 573 = -21$ and scaled inverse

$$137011290548979455469.$$

Tuple equality, residual arithmetic, and scaled-inverse equality are separate rows. The receipt records checker and compiler releases, snapshot hash, configuration, deterministic inputs, and declared nondeterministic inputs for both runs. Equality is *read*; configuration sameness is posited and validated against those records. Passing rows establish reproducibility under recorded assumptions, not that the three-state cycle caused the tuple or residual.

Charge labels, RGB labels, or a, b, c labels exhibit the same three-fold organization only when declared maps to S are supplied and their transition-preservation squares are checked. Without those maps, the resemblance is an interpretation, not process or value equality.

Governance fixes equality relations, quantizers, normalization, join keys, component maps, trace equivalence, cost conversions, budgets, stop priority, and certificate grammar. Provenance records raw reports, releases, configurations, arithmetic models, protocols, run identities, deterministic and declared nondeterministic inputs, projections, erasures, checked scope, and unresolved frontier.

The action ledger is exact. Equality relations, quantizers, process fields, comparison maps, cost conversions, join keys, budgets, and certificate grammar are *posited*. Cross-product equality, quantizer implications, map checks, trace correspondence, cost conversion, finite-join behavior, and certificate implications are *demonstrated*. Raw values, labels, inputs, transitions, traces, costs, stops, join outcomes, and provenance rows are *read*. Exact equality from coarse agreement, process identity from output equality, independence from names, completion beyond checked scope, and any nowtrino–naotrino–neutrino bridge remain *reserved*.

Chapter 8

The Electron Is Named

8.1 Partition and collision

Let the finite domain be an explicit list

$$X = [x_0, \dots, x_{n-1}],$$

and let $B = \{0, \dots, m-1\}$ be a declared set of boxes. A total box map $\beta : X \rightarrow B$ induces cells

$$C_j = \{x \in X : \beta(x) = j\}.$$

The nonempty cells form a partition of X : each listed input belongs to exactly one cell, and distinct cells are disjoint. This conclusion depends on total, single-valued evaluation of β , not on names attached afterward.

The finite pigeonhole statement is exact:

$$n > m \implies \exists i < k < n, \beta(x_i) = \beta(x_k).$$

It guarantees some colliding pair. It does not select which pair, prove that a proposed pair collides, identify the underlying inputs, or create a state

transition. In particular, a collision does not imply a cycle. Exact three-cycle closure is a different claim with a transition map and belongs to the next section.

The numerical algorithm traverses X in its declared order and maintains a visited table

$$V[j] = \text{none} \quad \text{or} \quad \text{some}(i).$$

At index i , evaluate $j = \beta(x_i)$. If $V[j]$ is empty, store i ; otherwise return the witness pair $(V[j], i)$. The table slices prior inputs by box while retaining only the first index per box. A full certificate also keeps the input list and all evaluated labels, because the table projection erases later members of each fiber.

Budgets are explicit: input fuel bounds visited indices, map effort bounds evaluation of β , table capacity bounds boxes, and arithmetic bounds label calculation. Stops include COLLISION, DOMAIN-EXHAUSTED, INPUT-BUDGET, MAP-BUDGET, INVALID-BOX, TABLE-CAPACITY, MAP-FAILURE, and OVERFLOW. Exhaustion without collision proves injectivity only on the traversed finite list and declared map. A budget stop leaves the unvisited suffix as an unresolved frontier.

A replay certificate records X, B, β 's declared graph or evaluation rule, traversal order, budgets, every label, visited-table update, collision witness or exhaustion stop, checked prefix, and unresolved suffix. Validation is sliced into domain, box-range, map, table-update, distinct-index, equal-label, count, stop, and claim rows, each pass, fail, or unresolved. The existential result licenses the collision conclusion only after finiteness, totality, input uniqueness, and $n > m$ rows pass.

Live neutral map: a specific three-to-two collision (computation).

Let

$$D = \{\text{base, first variation, second variation}\}, \quad B = \{0, 1\},$$

with $\beta(\text{base}) = 0$ and $\beta(\text{first variation}) = \beta(\text{second variation}) = 1$. Its fibers are

$$C_0 = \{\text{base}\}, \quad C_1 = \{\text{first variation}, \text{second variation}\}.$$

The two variations are therefore a specifically evaluated collision. The box count 2 is scoped to this declared domain and map. An arbitrary three-to-two map guarantees some collision but privileges no pair.

Worked example 1: a witnessed collision (computation). Let $X = [10, 11, 12, 13]$, $B = \{0, 1, 2\}$, and $\beta(x) = x \bmod 3$. Labels are $[1, 2, 0, 1]$. Traversal returns indices 0 and 3, since both map to box 1. The witness certifies equal labels and distinct inputs $10 \neq 13$; it does not certify equality of the integers.

Counterexample 2: existence does not choose the advertised pair (computation). With the same map, pigeonhole guarantees a collision because $4 > 3$. It does not imply that 11 and 12 collide: their labels are 2 and 0. A selected-pair claim requires its own two map evaluations.

Worked example 3: projection fibers (computation). For $X = [0, 1, 2, 3, 4, 5]$ and $\beta(x) = x \bmod 2$, the fibers are

$$C_0 = \{0, 2, 4\}, \quad C_1 = \{1, 3, 5\}.$$

Keeping only labels $[0, 1]$ records occupied boxes but erases multiplicity, members, order, and collision witnesses. The projected alphabet cannot reconstruct the partition.

Counterexample 4: repeated label is not a cycle (computation). The sequence of independently mapped inputs may have labels 0, 1, 0. This is a collision in a projection. Without a state transition T and checked equations $T(x_i) = x_{i+1}$, it says nothing about return, period, or dynamics.

Counterexample 5: an incomplete traversal (computation). Suppose $n = 5$, $m = 3$, but input fuel permits only two evaluations with distinct labels. The mathematical finite data still satisfy $n > m$, yet the traversal has not produced a witness because the remaining map values were not evaluated. Its report owns two rows and an unresolved suffix; it must not fabricate a pair.

Counterexample 6: distinct indices are not necessarily distinct terms (computation). For $[a, a, b]$, indices $0 \neq 1$ do not witness distinct domain terms. Without an input-uniqueness certificate, returning them proves only repeated occurrence. The traversal stops with INVALID-DOMAIN, or applies a declared provenance-preserving deduplication and restarts. Silent deduplication changes n and may invalidate $n > m$.

Finite traversal needs no axiom of choice: its declared order returns the first encountered pair. Existence itself supplies no canonical pair. A constant map on three distinct terms has several colliding pairs, and another traversal order can select another witness without changing the existential result.

Governance fixes the finite list, input-uniqueness evidence, equality, boxes, map and version, traversal order, deduplication and table policies, budgets, stop priority, and certificate grammar. Provenance records raw inputs, map configuration, labels, table states, projections, erasures, spent resources, checked prefix, unresolved suffix, stop, arithmetic model, protocol, run identity, deterministic inputs, and declared nondeterministic inputs.

The physics fence is strict. A finite computational collision is not particle identity, indistinguishability, degeneracy, recurrence, conservation, or a detector event. A physical use requires an apparatus, observable, units, calibration, sampling model, uncertainty, and a tested map from observations to boxes.

The action ledger is exact. Domain list, uniqueness evidence, equality, boxes, map, traversal, deduplication policy, budgets, and certificate gram-

mar are *posited*. Partition laws, the finite pigeonhole implication, table invariants, witness implications, exhaustion implications, and replay-checker implications are *demonstrated*. Inputs, labels, visited states, resource use, witnesses, stops, and validation rows are *read*. A canonical or privileged pair from existence, distinct terms from distinct indices, input equality from equal labels, global injectivity from scoped exhaustion, cycle dynamics from collision, reconstruction after projection, and physical meaning remain *reserved*.

8.2 A collision is not a cycle

Let S be a declared state space and $T : S \rightarrow S$ a deterministic total transition on the checked domain. A cycle certificate for initial state s_0 and period $p > 0$ requires

$$T^p(s_0) = s_0,$$

and a least-period certificate also requires $T^k(s_0) \neq s_0$ for every $0 < k < p$. These are full-state equalities. If a box projection $\beta : S \rightarrow B$ merely satisfies $\beta(T^i(s_0)) = \beta(T^j(s_0))$, it has found a label collision. Unless the projection is proved to reflect state equality and to respect transitions, that collision supplies no cycle.

The compatibility condition

$$\beta(x) = \beta(y) \implies \beta(Tx) = \beta(Ty)$$

defines a transition on labels. It can certify a quotient-label cycle, but still does not reflect full-state closure. For example, let $S = \mathbb{Z}/6\mathbb{Z}$, $T(x) = x + 1 \bmod 6$, and $\beta(x) = x \bmod 3$. Labels have period 3, whereas every full state has period 6.

The visited-table algorithm starts with s_0 , stores each full state with its first index, and repeatedly evaluates $s_{i+1} = T(s_i)$. On a previously visited state $s_j = s_{i+1}$, it reports preperiod j and period $p = i + 1 - j$. Least

period is checked by replaying the segment and testing all proper positive divisors of p , or all $1 \leq k < p$ when no divisor optimization is certified. A memory-bounded alternative advances one state by one step and another by two; a meeting proves an eventual repetition only under deterministic total transition and still requires a second pass to recover preperiod, period, and trace evidence.

Budgets distinguish transition fuel, state-comparison effort, visited-table capacity, arithmetic growth, and replay effort. Stops include CYCLE, FIXED-POINT, FUEL, COMPARISON-BUDGET, TABLE-CAPACITY, TRANSITION-FAILURE, DOMAIN, NONDETERMINISTIC-TRANSITION, REPLAY-FAILURE, and LEAST-PERIOD-UNRESOLVED. A budget stop owns its checked prefix, not absence of cycles beyond it.

If the checked state space has known cardinality $|S| = M$ and $T : S \rightarrow S$ is total and deterministic, then the $M + 1$ states $s_0, \dots, s_M$ force a repeated full state. Without that finite bound, fuel alone guarantees only termination of the bounded search, not that a repetition will be found.

A certificate records state schema, equality, transition rule and version, initial state, budgets, every full state, transition, comparison, visited-table update, repeated indices, preperiod, proposed period, least-period checks, checked prefix, unresolved frontier, and stop. Validation slices domain, determinism, transition replay, state equality, repeat index, period, minimality, resources, and claim into pass, fail, or unresolved rows.

Counterexample 1: repeated boxes without repeated states (computation). Let $S = \{a, b, c, d\}$, $T(a) = b$, $T(b) = c$, $T(c) = d$, $T(d) = d$, and let $\beta(a) = \beta(c) = 0$, $\beta(b) = \beta(d) = 1$. The label trace from a is $0, 1, 0, 1, 1, 1, \dots$, so the early 0 collision occurs at $a \neq c$, while the eventual full-state cycle is the fixed point d . Projected period two is false as a full-state claim.

Worked example 2: exact neutral three-cycle (computation). For

$$S = \{\text{minus}, \text{zero}, \text{plus}\}, \quad \text{minus} \mapsto \text{zero} \mapsto \text{plus} \mapsto \text{minus},$$

finite replay proves $T^3(s) = s$ for every state. The same table proves $T(s) \neq s$ and $T^2(s) \neq s$, so the least period is exactly 3. This is a transition certificate independent of the three-to-two pigeonhole result in the preceding section.

For any read $r : S \rightarrow R$, the return identity gives $r(T^3s) = r(s)$. It does not give one-step invariance $r(Ts) = r(s)$. In particular, no residue or α invariance, and no causal explanation of either, follows without a separately stated read map and preservation law.

Worked example 3: a tail entering a cycle (computation). Let $0 \mapsto 1 \mapsto 2 \mapsto 3 \mapsto 2$. From 0, the first repeated full state is 2, at indices 2 and 4. The preperiod is 2, the period is 2, and the cycle is $2 \leftrightarrow 3$. Reporting period four from the return time alone would confuse prefix length with cycle length.

Counterexample 4: nondeterministic compatibility failure (computation). If state u may transition to either v or w , repeated runs can diverge. A recorded path can still be replayed as a finite path, but it is not a certificate for a deterministic transition function or for all possible future paths. A schedule or choice stream must become part of the full state before deterministic cycle reasoning applies.

Counterexample 5: finite search does not decide universal halting (computation). Fuel N makes this cycle search terminate and permits FUEL when no repetition has appeared. It does not decide whether an arbitrary unbounded computation will halt or repeat. The universal halting problem remains outside the finite certificate; only the enumerated transition prefix is owned.

Governance fixes state schema and equality, transition domain and version, initial state, deterministic inputs, visited or two-speed algorithm, budgets, minimality procedure, stop priority, and certificate grammar. Provenance records raw states, configuration, transition and comparison outcomes, table states, spent resources, checked prefix, unresolved frontier, releases, snapshot hash, arithmetic model, protocol, run identity, platform, and declared nondeterministic inputs.

The action ledger is exact. State space, equality, transition, initial state, algorithms, budgets, and certificate grammar are *posited*. Replay identities, visited-table invariants, conditional meeting implications, period and least-period checks, and certificate implications are *demonstrated*. States, transitions, comparisons, table updates, repeated indices, resources, stops, and validation rows are *read*. State equality from projected collision, a cycle without transition compatibility, least period from return alone, no cycle beyond a budget, universal halting, and physical recurrence remain *reserved*.

8.3 When a physical name is licensed

Neither a collision nor a cycle licenses a physical name. A collision certifies that distinct domain terms share a box under a declared projection. A cycle certifies return under a declared full-state transition. A physical claim requires a third structure:

$\mathcal{L} = (\text{system, apparatus, observable, model, units, calibration, sampling, uncertainty, alternatives, pro}$

The bridge from a computational report to $\mathcal{L}$ must be stated, calibrated, and tested. Finiteness, repeatability, and word resemblance cannot supply it.

The four-name grammar is useful only because it prevents coercion:

$\text{nowtrino}(H)$ = current computational projection after history slicing,
 $\text{newtrino}(s)$ = $U(s)$, the next computational state under declared update U ,
 naotrino = physical candidate; manuscript coinage from Greek *naos*, temple,
 $\nu\text{-trino}$ = the standard neutrino-physics register.

After state representation and update are declared, transition supplies

$$\text{nowtrino}(H) \xrightarrow{U} \text{newtrino}(s).$$

This is a computational edge. It does not reach naotrino or $\nu\text{-trino}$. A naotrino claim needs a concrete apparatus, model, observables, units, calibration, uncertainty, and discriminating data. A standard neutrino claim must use the established physical model and observables appropriate to that register. Similar spelling is no map.

A candidate bridge has typed form

$$\text{Input} \longrightarrow \text{Report} \xrightarrow{\kappa} C \xleftarrow{\lambda} \text{Observation},$$

where κ converts a computational report into comparison domain C , and λ converts calibrated observations into the same domain. Before notation is shared, three value relations remain distinct:

exact same-number $\kappa(R) = \lambda(O)$,
 resolution equivalence $q_\theta(\kappa(R)) = q_\theta(\lambda(O))$,
 uncertainty compatibility $\mathcal{U}_R \cap \mathcal{U}_O \neq \emptyset$ under a declared region model.

Same-number is reserved for exact equality after typed conversion. None of these relations implies same-process or physical identity.

The two-pass calibration guards against a name being planted by one route. A direct acquisition and a stepped acquisition through an interme-

diate produce reports R_d, R_s . Their raw values, uncertainty regions, routes, budgets, and provenance are retained. The comparison reports three rows:

exact equality	$\kappa_d(R_d) = \kappa_s(R_s),$
resolution equality	$q_\theta(\kappa_d(R_d)) = q_\theta(\kappa_s(R_s)),$
uncertainty-region overlap	$\mathcal{U}_d \cap \mathcal{U}_s \neq \emptyset.$

Each row independently passes, fails, or remains unresolved. No combined agreement bit may erase which relation passed. Passing any row does not prove route independence, apparatus correctness, or a physical mechanism unless those hypotheses receive separate tests.

Worked example 1: a licensed computational transition. Let a full state be $s = (x, c, \sigma)$, and let U update all three fields. The current summary $\text{nowtrino}(H) = s$ and next state $\text{newtrino}(s) = U(s)$ are connected by replaying U . If history slicing erased the prior trace, the edge certifies this one update only. Calling either endpoint a physical particle would be a type and evidence error.

Counterexample 2: collision plus cycle still lacks apparatus. Three computational states may map into two boxes, and the states may also rotate with least period three. These are valid finite certificates for projection and transition. They specify no detector, physical observable, unit, calibration, noise law, or uncertainty. Their conjunction therefore licenses no naotrino or standard neutrino claim.

Worked example 3: interval compatibility at uncertainty. Suppose two calibrated channels report intervals I_1 and I_2 in the same unit, with a preregistered rule that accepts when $I_1 \cap I_2 \neq \emptyset$. An observed overlap passes the interval-overlap, or compatible-at-uncertainty, row. It is not an exact same-number result. Shared calibration or preprocessing may correlate the

channels, so covariance and cross-covariance remain in the report. Overlap does not certify independent evidence or equal acquisition processes.

Counterexample 4: same numeral in incompatible domains. A computational summary and a detector fit both display 3. One counts transitions; the other estimates an energy in declared units. Without κ, λ , unit conversion, uncertainty, and model checks, equality of printed numerals is not even a same-number claim in a common domain.

Licensing is a finite validation pipeline: parse the candidate name; resolve its domain, route, and version; validate the apparatus record; replay calibration and numerical acquisition; evaluate exact same-number, resolution equivalence, uncertainty-region compatibility, and same-process slices; test alternative models; then classify the name as computational, physically licensed, interpretive, reserved, ambiguous, or rejected. Stops include missing apparatus, missing calibration, unit mismatch, invalid model, nonidentifiability, acquisition budget, numerical budget, uncertainty failure, route mismatch, provenance mismatch, shared-data dependence unresolved, ambiguous name, rejected bridge, and complete.

The certificate retains raw and rejected samples, preregistration, blinding and unblinding events, protocol and version, apparatus configuration, environmental conditions, calibrations, observations, model equations, numerical iterations and slices, covariance, uncertainty construction, alternative-model tests, budgets, stops, checked scope, unresolved frontier, and deterministic and declared nondeterministic inputs. A physical name is conditional on this record and its validity domain.

Governance fixes the four-name ledger, computational representations and updates, apparatus and observation grammar, bridge maps, units, calibration transfer, sampling, uncertainty, two-pass comparison, process relation, budgets, stop priority, and certificate grammar. Any changed field creates a new licensing report.

The action ledger is exact. Names, representations, update, apparatus model, bridge maps, units, calibration, uncertainty, comparison relations, budgets, and certificate grammar are *posited*. Update replay, calibration transformations, dimensional checks, uncertainty calculations, exact same-number implications, resolution-label implications, uncertainty-region compatibility implications, conditional process maps, and validation implications are *demonstrated*. Computational states, observations, samples, fits, covariance, comparison outcomes, stops, and provenance rows are *read*. A physical bridge from spelling, physical identity from collision or cycle, process equality from number agreement, independence from two names or routes, a naotrino without a complete apparatus record, and any identification with the standard neutrino remain *reserved*.

Chapter 9

The Corridor Rotates

9.1 Computational rotation

A computational rotation is a permutation of a finite declared state set, not a geometric angle. Let

$$S = [s_0, \dots, s_{m-1}], \quad R : S \rightarrow S,$$

where enumeration proves the listed states unique and evaluation proves R total and one-to-one. The explicit state record contains a variation label, an order index, a sign symbol, and any additional fields that affect the transition. Omitting a field creates only a projected transition.

Permutation composition is

$$(R_2 \circ R_1)(s) = R_2(R_1(s)),$$

with identity $\text{id}(s) = s$. Iteration is $R^0 = \text{id}$ and $R^{n+1} = R \circ R^n$, hence $R^{a+b} = R^a \circ R^b$. The order of R is the least positive p satisfying $R^p = \text{id}$ on every state in the declared domain. A return of one state does not establish this global order.

Variation is computed by applying R and comparing the full before-and-

after states. Order is computed by finite iteration until identity is reached or the search budget ends. A sign is a declared map $\sigma : S \rightarrow \{\text{minus}, \text{zero}, \text{plus}\}$; it is a label, not arithmetic multiplication unless operations on signs are separately defined. Permutation parity is determined from a disjoint-cycle decomposition:

$$\text{parity}(R) = (-1)^{m-c},$$

where c counts cycles including fixed points. Alternatively, a certified transposition decomposition yields the same parity. Parity does not identify the permutation: many permutations share one parity bit.

If the disjoint-cycle lengths are $\ell_1, \dots, \ell_c$, including length 1 for fixed points, then

$$\text{ord}(R) = \text{lcm}(\ell_1, \dots, \ell_c).$$

The least common multiple returns every cycle because it is divisible by each ℓ_i . Conversely, if $R^k = \text{id}$ globally, then each cycle returns after k , so every $\ell_i \mid k$; hence their least common multiple divides every global return exponent.

The numerical procedure traverses the state list, evaluates each image, checks range and uniqueness, builds inverse and visited tables, slices the permutation into disjoint cycles, and accumulates cycle lengths and parity. Iteration then replays each cycle to verify closure and minimal length. Projection tables for order, sign, or other labels are kept separate from the full-state table.

Worked example 1: exact three-state rotation (computation). For

$$\text{minus} \mapsto \text{zero} \mapsto \text{plus} \mapsto \text{minus},$$

full-state replay gives $R^3 = \text{id}$, while $R \neq \text{id}$ and $R^2 \neq \text{id}$. Its order is exactly 3. The single three-cycle has parity $(-1)^{3-1} = +1$. Exact return follows from the transition table, not from pigeonhole collision.

Counterexample 2: label collision is not rotation order (computation). Let six full states rotate by $R(i) = i + 1 \bmod 6$, and project them by $\beta(i) = i \bmod 3$. Labels repeat after three steps, while full states return after six. The compatibility $\beta(x) = \beta(y) \Rightarrow \beta(Rx) = \beta(Ry)$ defines a quotient-label rotation of order three; it does not make the full permutation order three.

Worked example 3: composition order matters (computation). On $\{0, 1, 2\}$, let A exchange 0, 1 and let B exchange 1, 2. Then

$$B \circ A = (0\ 2\ 1), \quad A \circ B = (0\ 1\ 2).$$

Both are three-cycles and both have even parity, but they are different permutations. Equal order and parity do not prove same process or equal state-by-state transition.

Counterexample 4: same operation schema, different representations (computation). Two reports may both say rotate forward, while one enumerates states (a, b, c) and another $(0, 1, 2)$. Process comparison requires a bijection J satisfying

$$J \circ R_1 = R_2 \circ J.$$

Without this checked conjugacy map, shared wording establishes neither same-number nor same-process.

Worked example 5: order and parity are different invariants (computation). A permutation with one disjoint two-cycle and one disjoint three-cycle has

$$\text{ord}(R) = \text{lcm}(2, 3) = 6, \quad \text{parity}(R) = (-1)^{(2-1)+(3-1)} = -1.$$

Its order is six and its parity is odd. Neither invariant determines the other.

Budgets distinguish state enumeration, transition evaluations, comparison effort, table capacity, composition depth, and replay effort. Stops include COMPLETE, ENUMERATION-BUDGET, TRANSITION-BUDGET, OUT-OF-DOMAIN, DUPLICATE-STATE, NON-TOTAL, NON-INJECTIVE, TABLE-CAPACITY, ORDER-UNRESOLVED, PARITY-UNRESOLVED, and REPLAY-FAILURE. A fuel stop certifies only the checked iterations.

A certificate records state schema and enumeration, equality, full transition table, projection maps, budgets, visited and inverse tables, cycle slices, compositions, order candidates, proper-divisor checks, parity calculation, stops, checked scope, and unresolved frontier. Validation slices enumeration, totality, injectivity, transition, composition, closure, minimal order, parity, projection compatibility, resources, and claims into pass, fail, or unresolved rows.

Comparison follows the established grammar. Exact state equality, equality under a declared projection, and process equivalence are separate relations. Process equivalence requires compatible inputs, a state map, a commuting transition square, trace correspondence, cost conversion, configuration, and provenance. Equal cycle length, parity, or projected labels cannot replace those rows.

Governance fixes state representation, equality, enumeration, transition direction, composition convention, label maps, parity convention, algorithms, budgets, stop priority, and certificate grammar. Provenance records raw states, transition and comparison outcomes, table states, cycle slices, spent resources, configuration, releases, snapshot hash, arithmetic model, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

The action ledger is exact. States, equality, transition, composition convention, projections, algorithms, budgets, and certificate grammar are *posited*. Composition laws, finite permutation implications, cycle decomposition, or-

der and minimality checks, parity identities, conjugacy implications, and replay implications are *demonstrated*. States, transitions, comparisons, cycles, order, parity, resource use, stops, and validation rows are *read*. Full-state return from label collision, process identity from equal order or parity, a physical phase, geometric rotation, holonomy, angular momentum, and any physical identification remain *reserved*.

9.2 Physical phase

A physical phase is defined relative to a model and reference. A complex amplitude

$$z = Ae^{i\theta} = A(\cos \theta + i \sin \theta), \quad A \geq 0,$$

has phase θ modulo 2π when $A > 0$. At $A = 0$, phase is undefined. The complex plane requires addition and multiplication satisfying their field laws, a distinguished unit i with $i^2 = -1$, and a reference axis. Two instances, two labels, or a Boolean pair do not construct that structure.

Operationally, phase is inferred by comparing a signal with a calibrated reference. For quadrature observations

$$I = A \cos \theta + \epsilon_I, \quad Q = A \sin \theta + \epsilon_Q,$$

the principal estimate is

$$\hat{\theta} = \text{atan2}(Q, I) \in (-\pi, \pi].$$

Its apparatus record includes detector response, reference oscillator, epoch, channel gains and offsets, sampling clock, bandwidth, noise covariance, saturation limits, and environmental conditions. A numerical angle without this record is not a physical phase measurement.

Phase extraction first calibrates I, Q , rejects or flags invalid samples,

checks amplitude against a declared noise floor, evaluates the quadrant-aware angle, and propagates uncertainty. Away from the origin, first-order propagation uses

$$\nabla\theta = \frac{1}{I^2 + Q^2}(-Q, I), \quad \text{Var}(\hat{\theta}) \approx \nabla\theta^\top \Sigma_{IQ} \nabla\theta.$$

Near $I = Q = 0$, this linearization is unstable and the phase row remains unresolved or uses a declared nonlinear uncertainty construction.

For sampled phases $\phi_0, \dots, \phi_N$, unwrapping selects integers k_j and sets

$$\tilde{\phi}_j = \phi_j + 2\pi k_j$$

so adjacent changes satisfy a declared continuity bound. Unwrapping is model-based: if the true change can exceed the bound between samples, several lifts are compatible with the data. The algorithm must retain principal phases, chosen integers, rejected alternatives, continuity threshold, and unresolved branch points.

A holonomy is a round-trip quantity defined by a connection and path, not by a repeated label. In an Abelian model,

$$H(\gamma) = \exp\left(i \oint_{\gamma} A_{\mu} dx^{\mu}\right).$$

For a partition $t_0 < \dots < t_N$ of path γ , a finite approximation is

$$H_N = \prod_{j=0}^{N-1} \exp(i A_{\mu}(\gamma(t_j))[\gamma^{\mu}(t_{j+1}) - \gamma^{\mu}(t_j)]).$$

The certificate records path orientation, connection units and gauge convention, quadrature rule, samples, products, mesh width, and error model. Refinement compares H_N, H_{2N} in a declared metric; agreement at tolerance is numerical convergence evidence, not an exact integral absent a demonstrated

error bound.

For matrix-valued transport, factor order matters:

$$H_N = U_{N-1} \cdots U_1 U_0.$$

Changing path order or multiplying factors as if they commute changes the model. Numerical stabilization may renormalize factors or use structure-preserving products, but every such projection and its error must be reported.

Spinor return is another model-specific phase statement. For a spin- $\frac{1}{2}$ representation,

$$U(\varphi) = \exp(-i\varphi \mathbf{n} \cdot \boldsymbol{\sigma}/2), \quad U(2\pi) = -I, \quad U(4\pi) = I.$$

The spinor vector changes sign after 2π and returns exactly as a vector after 4π . The ray is already unchanged after 2π , because $[\psi] = [-\psi]$ for an isolated pure state. The minus sign is observable only as a relative phase against another path or reference in an interference or equivalent apparatus; a global sign by itself is not a detector reading. These statements are scoped to the declared spin- $\frac{1}{2}$ representation and apparatus. They do not make 4π a universal period of physical states, and they are not inferred from a three-state computational permutation.

Worked measurement 1: calibrated quadrature phase (physics).

After gain and offset calibration, suppose $I = 3$, $Q = 4$ in common amplitude units. Then $\hat{\theta} = \text{atan2}(4, 3)$. Repeating the arithmetic exactly verifies the numerical estimate. Physical certification additionally requires the reference epoch, calibration covariance, amplitude validity, and uncertainty interval.

Counterexample 2: two labels are not a complex plane. Calling two states real and imaginary supplies neither addition, multiplication, distributivity, nor an element whose square is -1 . A transition exchanging the labels

is a permutation of order two. It is not multiplication by i until a complex representation and a checked intertwining map are supplied.

Worked measurement 3: discretized loop refinement (physics). Compute H_N, H_{2N}, H_{4N} on nested path partitions and compare their principal phase differences. If successive differences fall below tolerance while a discretization bound decreases with mesh width, the reported uncertainty can include truncation and sampling errors. Equal formatted phases alone are only resolution equivalence.

Counterexample 4: computational period three is not physical phase. An exact permutation $\text{minus} \mapsto \text{zero} \mapsto \text{plus} \mapsto \text{minus}$ has order three. It specifies no complex amplitude, connection, path, apparatus, clock, or uncertainty. Mapping its labels to angles $0, 2\pi/3, 4\pi/3$ is a posited representation; only a checked transition map supports that numerical encoding, and no physical identification follows.

Budgets distinguish acquired samples, calibration effort, phase-extraction arithmetic, unwrap branches, path segments, matrix growth, refinement levels, and uncertainty computation. Stops include COMPLETE, MISSING-REFERENCE, MISSING-CALIBRATION, BELOW-AMPLITUDE-FLOOR, SATURATION, UNIT-MISMATCH, UNWRAP-AMBIGUOUS, SEGMENT-BUDGET, REFINEMENT-BUDGET, NONCONVERGENT, OVERFLOW, and MODEL-FAILURE. A resource stop does not become a physical zero or an absence of phase.

Comparison reports three value rows separately. Exact same-number means equality after typed conversion. Resolution equivalence means equal labels under a declared quantizer. Uncertainty compatibility means overlap of declared phase regions, with circular topology respected. Each independently passes, fails, or remains unresolved. Same-process additionally requires matching apparatus inputs, calibration, sampling, numerical transitions, traces, costs, configuration, and provenance.

The certificate records apparatus and model, complex representation,

units, references and epochs, raw and rejected samples, calibration, covariance, principal phases, unwrap integers and alternatives, path and connection samples, ordered products, refinements, convergence and error bounds, uncertainty regions, budgets, stops, checked scope, unresolved frontier, protocol, run identity, platform, and deterministic and declared nondeterministic inputs.

Governance fixes physical model, apparatus, complex and spin representations, reference, gauge convention, sampling, calibration, extraction, unwrapping, path quadrature, product order, convergence rule, uncertainty method, budgets, stop priority, and certificate grammar. A changed field starts a new report.

The action ledger is exact. Apparatus, model, representation, reference, calibration, path, connection, algorithms, uncertainty, budgets, and certificate grammar are *posited*. Complex identities, conditional extraction and propagation formulas, unwrap implications, ordered-product identities, discretization bounds, spinor representation identities, and validation implications are *demonstrated*. Samples, calibration observations, quadratures, phases, unwrap choices, products, convergence rows, uncertainties, stops, and provenance are *read*. A complex plane from two labels, physical phase from a permutation, exact holonomy from unbounded numerical refinement, independence from differently named routes, a five-quantity interpretation of one residue, and any uncalibrated physical identification remain *reserved*.

9.3 Resonance audit

Resonance is a prompt for comparison, not evidence. The computational report contains a finite state set, permutation, compositions, traces, and costs. The physical report contains complex amplitudes, references, apparatus, calibration, path transport, uncertainty, and observables. An audit asks which precise maps, if any, connect them.

A typed representation has the form

$$\rho : S_{\text{comp}} \rightarrow V_{\text{phys}},$$

where V_{phys} belongs to a declared physical model. For a computational transition R and physical transport U , one-step compatibility requires

$$\rho \circ R = U \circ \rho.$$

Same-process is stronger than this value square. It requires compatible inputs, composition laws, invariants, state and trace relations, cost or resource correspondence, configuration, apparatus, calibration, and provenance. If only a projected label commutes, the result is a projected comparison, not full process equivalence.

An invariant also needs a declared read r . Computational preservation is

$$r(Rs) = r(s) \quad \text{for all checked } s.$$

A return $R^p(s) = s$ implies $r(R^p(s)) = r(s)$, but not one-step preservation. Without r and the displayed law, no residue invariance has been established.

Audit 1: relabeling versus angular momentum. A finite relabeling is a permutation. Angular momentum is a physical observable and generator within a continuous symmetry model, with units, dynamics, apparatus, and calibration. A discrete permutation does not construct a continuous one-parameter group, differentiable action, invariant physical action, or conserved observable. Consequently relabeling is not angular momentum, and a permutation does not invoke a Noether-type conservation law without the missing physical structure and demonstrated hypotheses.

Audit 2: order-three versus phase. The exact three-state rotation has $R^3 = \text{id}$. Mapping its states to $1, e^{2\pi i/3}, e^{4\pi i/3}$ gives a numerical representation

only after ρ is posited and the intertwining square is checked. That square still supplies no oscillator, path, connection, detector, or uncertainty. Equal order therefore does not license physical phase or holonomy.

Audit 3: same number through two numerical routes. Let two independently specified refinement routes produce y_h and z_h in a common typed domain. The audit reports three rows:

$$\begin{array}{ll} \text{exact equality} & y_h = z_h, \\ \text{resolution equality} & q_\theta(y_h) = q_\theta(z_h), \\ \text{uncertainty compatibility} & \mathcal{U}(y_h) \cap \mathcal{U}(z_h) \neq \emptyset. \end{array}$$

Each passes, fails, or remains unresolved. A combined agreement flag must not erase which relation passed. Agreement of values does not establish same-process.

When two nested discretizations approximate the same smooth-model quantity with demonstrated expansion

$$A(h) = A^* + Ch^p + O(h^{p+1}),$$

Richardson extrapolation may be used:

$$A_R = \frac{2^p A(h/2) - A(h)}{2^p - 1}.$$

This is legitimate only when both values use compatible models, paths, boundary data, refinement ratio, asymptotic regime, and error order p . The order is either derived analytically or estimated independently from at least three compatible nested levels. Define

$$D_h = A(h) - A(h/2), \quad D_{h/2} = A(h/2) - A(h/4),$$

and, when both are nonzero with signs and regime appropriate to the assumed

leading term,

$$p_{\text{obs}} = \log_2 \frac{|D_h|}{|D_{h/2}|}.$$

Observed order must stabilize across further refinements from the same family, while raw differences decrease and a demonstrated error bound remains valid. Choosing p to make an extrapolate hit a target is rejected. Under the displayed expansion, extrapolation cancels the leading term and leaves a demonstrated $O(h^{p+1})$ remainder here; it does not produce an exact limit. Nearby values alone do not justify extrapolation, and a route with different physics or preprocessing is a comparison channel, not automatically a refinement level.

The finite comparison algorithm parses both reports, validates types and units, resolves representation maps, replays transitions or transport products, checks commuting squares and invariants, aligns refinement levels, computes exact differences, quantized labels, uncertainty intersections, and—when admissible—Richardson estimates and observed convergence ratios. It retains rejected alignments and alternative maps.

Worked example 4: a failed intertwining map. Suppose R cycles three computational states, while proposed U fixes all represented vectors. If ρ distinguishes the states, then $\rho(Rs) \neq U(\rho(s))$ for some s . The representation row fails even though $U^3 = I$ and $R^3 = \text{id}$. Equal return exponents alone do not make the processes correspond.

Counterexample 5: apparent Richardson convergence outside scope. Three displayed values can accidentally fit a power-law ratio. Without nested meshes, common target, stable boundary conditions, and a demonstrated truncation expansion, extrapolation manufactures precision. The audit reports the ratio as read data and leaves the extrapolated-limit claim unresolved or rejected.

Budgets distinguish report parsing, state and path replay, representation

search, equality checks, refinement levels, arithmetic growth, uncertainty propagation, alternative-model tests, and apparatus validation. Stops include COMPLETE, TYPE-MISMATCH, UNIT-MISMATCH, MISSING-REPRESENTATION, INTERTWINING-FAILURE, INVARIANT-FAILURE, TRACE-MISMATCH, APPARATUS-MISSING, CALIBRATION-MISSING, REFINEMENT-INCOMPATIBLE, ASYMPTOTIC-REGIME-UNRESOLVED, ORDER-UNRESOLVED, ZERO-DIFFERENCE, ORDER-UNSTABLE, NONCONVERGENT, BUDGET, and PROVENANCE-MISMATCH.

The certificate slices type, domain, exact value, resolution, uncertainty, composition, state, trace, invariant, resource, apparatus, calibration, refinement, convergence, extrapolation, and provenance checks into pass, fail, or unresolved rows. It records raw reports, representation candidates, commuting-square evaluations, invariant reads, path products, mesh data, differences, convergence ratios, p_{obs} , analytic or estimated order provenance, stabilization rows, error budgets, covariance, stops, checked scope, unresolved frontier, releases, snapshot hash, configurations, protocols, run identities, platforms, and deterministic and declared nondeterministic inputs.

Governance fixes report schemas, typed domains, representation and comparison maps, invariants, same-process relation, quantizer, uncertainty model, apparatus requirements, refinement family, error expansion, budgets, stop priority, and certificate grammar. A changed map, model, path, or configuration begins a new audit.

The action ledger is exact. Representations, process relations, reads, invariants, apparatus model, quantizer, uncertainty, refinement family, error order, budgets, and certificate grammar are *posited*. Intertwining and preservation implications, composition checks, exact and resolution comparisons, uncertainty intersections, analytic or independently estimated order justification, stabilized-order implications, conditional Richardson cancellation, convergence bounds, and certificate implications are *demonstrated*. States, values, labels, traces, costs, observations, products, differences, ratios, p_{obs} , uncertainties, stops, and provenance rows are *read*. Relabeling as angular mo-

mentum, continuous symmetry or conservation from a discrete permutation, residue invariance without a read-preservation law, phase or holonomy from order alone, independence from different names, an exact limit from extrapolation, choosing order to fit a target, extrapolation outside its demonstrated regime, and word resonance as evidence remain *reserved*.

Part IV

The Closing

Chapter 10

The Native Apparatus Presents the Residue

10.1 Apparatus ratio

The internal apparatus begins with integer data

$$A = (n, d), \quad n \in \mathbb{Z}, \quad d \in \mathbb{Z}_{>0}.$$

It denotes the exact rational n/d . Positivity of d is part of the domain, not a conclusion of division. If an external representation permits $d < 0$, normalization first replaces (n, d) by $(-n, -d)$. A zero denominator is rejected and produces no ratio, quotient, or remainder.

For $d > 0$, Euclidean floor division produces the unique integers q, r satisfying

$$n = qd + r, \quad 0 \leq r < d, \quad q = \left\lfloor \frac{n}{d} \right\rfloor.$$

The sign convention matters when $n < 0$. For example,

$$-7 = (-3) \cdot 3 + 2.$$

Truncation toward zero would instead report quotient -2 and signed leftover -1 ; that is a different operation and does not satisfy the displayed Euclidean remainder range.

Scaled floor at integer scale $M > 0$ applies the same operation to nM :

$$Q_M = \left\lfloor \frac{nM}{d} \right\rfloor, \quad R_M = nM - Q_M d, \quad 0 \leq R_M < d.$$

The pair (Q_M, R_M) retains the exact decomposition. Reporting only Q_M is a slicing projection: it erases the fractional residue R_M/d . Decimal punctuation or formatting is a later projection and must state its sign, padding, rounding, and boundary rules.

All arithmetic is integer arithmetic. Exactness does not mean cost-free: multiplication by M , quotient selection, greatest-common-divisor normalization, and replay can increase intermediate bit length. A resource record therefore distinguishes operation count, requested/spent/remaining arithmetic budget, maximum bit length, storage, and comparison effort. A bounded integer representation also checks overflow before multiplication; wrapping arithmetic is not an exact ratio operation.

The iterative division algorithm starts with $a_0 = n$, $b_0 = d > 0$. One floor-division step yields

$$a_k = q_k b_k + r_k, \quad 0 \leq r_k < b_k.$$

For a single apparatus ratio it stops after this step with q_0, r_0 . If normalization or a greatest common divisor is requested, iteration continues with $(a_{k+1}, b_{k+1}) = (b_k, r_k)$ while $r_k > 0$. Each step strictly decreases the positive divisor, so finite termination follows. The quotient sequence and remainder trace must not be confused with the single scaled-floor projection.

Worked example 1: exact positive division (computation). For $n = 17, d = 5$,

$$17 = 3 \cdot 5 + 2.$$

The certificate checks equality and $0 \leq 2 < 5$. Quotient 3, remainder 2, and rational $17/5$ are three related fields, not interchangeable names.

Worked example 2: scaled floor and retained loss (computation).

For $n = 17, d = 5, M = 100$,

$$1700 = 340 \cdot 5 + 0.$$

The scale happens to resolve this rational exactly. For $M = 10$, the result is also exact; for $n = 1, d = 3, M = 100$,

$$100 = 33 \cdot 3 + 1,$$

so the displayed quotient 33 omits residue $1/3$ in scaled units. Retaining n, d, M, Q_M, R_M makes the loss replayable.

Counterexample 3: denominator zero (computation). Input $(5, 0)$ has no rational interpretation and no Euclidean quotient. Returning zero, infinity, or a sentinel numeral as if it were a ratio would merge a domain failure with numerical data. The report stops ZERO-DENOMINATOR and retains the rejected input.

Counterexample 4: equal displayed quotient, unequal ratios (computation). At scale $M = 10$, both $1/3$ and $17/50$ have floor quotient 3, yet

$$\frac{1}{3} \neq \frac{17}{50}.$$

They are resolution-equivalent under that declared floor quantizer, not exactly the same number. Their remainders and denominators differ, and no uncertainty claim follows from the collision.

Stops include COMPLETE, ZERO-DENOMINATOR, SIGN-NORMALIZATION-FAILURE, INVALID-SCALE, ARITHMETIC-BUDGET, BIT-GROWTH, OVERFLOW, STORAGE-BUDGET, ITERATION-BUDGET, IDENTITY-FAILURE, and REMAINDER-RANGE-FAILURE. A resource stop preserves the checked prefix and unresolved operation; it does not return a partial quotient as complete.

A certificate records raw numerator and denominator, normalization, scale, arithmetic model, budgets, every product, quotient candidate, comparison, remainder, identity, range check, iterative state, projection, formatting rule, stop, checked scope, and unresolved frontier. Validation slices domain, sign, exact identity, remainder range, scale, iteration, resource, projection, and claim rows into pass, fail, or unresolved.

Comparison uses three separate relations. Exact same-number after typed conversion is checked by cross-products:

$$n_1 d_2 = n_2 d_1.$$

Resolution equality compares declared scaled or formatted labels. Uncertainty compatibility compares declared regions after unit and domain conversion. Each row independently passes, fails, or remains unresolved. No combined agreement bit may hide which relation passed.

Governance fixes integer domain, denominator and sign convention, normalization, scale, floor operation, arithmetic model, growth limits, iteration, projection and formatting rules, comparison relations, budgets, stop priority, and certificate grammar. Provenance records raw data, transformations, releases, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

The ratio is computation-owned. It has no physical unit unless a later apparatus supplies one, and it is not an external coupling or physical fine-

structure value. Numerical resemblance cannot create that bridge. Physical balance, calibration, force, and uncertainty belong to the next section.

The action ledger is exact. Domain, sign convention, scale, arithmetic model, algorithms, projections, comparison relations, budgets, and certificate grammar are *posited*. Euclidean existence and uniqueness under $d > 0$, identity and range implications, scaled-floor decomposition, decreasing-remainder termination, cross-product equality, and certificate implications are *demonstrated*. Inputs, products, quotients, remainders, traces, bit lengths, labels, resource use, stops, and validation rows are *read*. A ratio at zero denominator, Euclidean range under an incompatible sign rule, exact equality from a display collision, uncertainty from rounding, a physical balance, and identification with any external physical constant remain *reserved*.

10.2 The physical balance

A torsion balance measures torque through a calibrated angular response. In a symmetric idealization, two small masses m lie at lever arm L from a suspended axis, two source masses M produce approximately tangential forces, and the fiber supplies restoring torque. Static equilibrium is

$$\tau_{\text{ext}}(\theta; \mathbf{g}) = \kappa\theta,$$

where κ is the torsional spring constant in newton metres per radian and $\mathbf{g}$ records measured geometry. Under a simplified inverse-square model,

$$F = G \frac{Mm}{r^2}, \quad \tau_{\text{ext}} \approx 2LF,$$

so

$$G \approx \frac{\kappa\theta r^2}{2LMm}.$$

This estimator is conditional on geometry, force direction, point-mass approximation, equilibrium, and the absence or correction of competing torques.

The apparatus record includes masses and their calibration, center-to-center distances, lever arms, source-mass positions, fiber material and dimensions, vacuum and temperature, leveling, magnetic and electrostatic shielding, readout geometry, timing, damping, and source-position protocol. Every quantity carries units and uncertainty. Dimensional consistency is necessary:

$$\left[\frac{\kappa \theta r^2}{LMm} \right] = \text{m}^3 \text{kg}^{-1} \text{s}^{-2},$$

but dimensions alone do not identify G or validate the model.

For an optical lever of path length D , a small angular twist may produce spot displacement

$$x - x_0 \approx 2D\theta, \quad \hat{\theta}_{\text{lin}} = \frac{x - x_0}{2D}.$$

A more complete readout model retains the calibrated nonlinear relation, for example

$$x - x_0 = D \tan(2\theta), \quad \hat{\theta} = \frac{1}{2} \arctan \frac{x - x_0}{D}.$$

The linear approximation requires a demonstrated angular error bound over the admitted range. Saturation, beam clipping, detector nonlinearity, reference drift, and ambiguous spot position are readout failures.

The torsion constant can be calibrated dynamically. For moment of inertia I , lightly damped free oscillation has

$$T_0 = 2\pi \sqrt{\frac{I}{\kappa}}, \quad \kappa = \frac{4\pi^2 I}{T_0^2},$$

under the stated oscillator model. Period estimation requires timestamp calibration, transient handling, damping assessment, and uncertainty in I . A static calibration with known torque is a different route and retains different

systematics.

Numerical estimation begins with raw time, readout, source-position, and environmental series. It validates units and calibration intervals, preserves rejected samples and reasons, subtracts a preregistered baseline, estimates equilibrium or fits a damped-oscillation model, converts readout to angle, computes torque, and fits the physical parameter. Iterative nonlinear fitting records every parameter vector, residual, objective value, accepted step, rejected step, and convergence check.

Worked measurement 1: source-position reversal (physics). Alternate source masses between configurations A and B . If fitted equilibria are θ_A, θ_B , use

$$\Delta\theta = \frac{\theta_A - \theta_B}{2}$$

under a model in which the gravitational torque reverses and a constant zero offset does not. The subtraction rejects that offset but not drift synchronized with source motion. Randomization, blinding, dwell time, and a drift model belong in the certificate.

Worked measurement 2: covariance propagation (physics). For

$$\hat{G} = \frac{\hat{\kappa} \hat{\theta} \hat{r}^2}{2\hat{L} \hat{M} \hat{m}},$$

write $u = (\log \kappa, \log |\theta|, \log r, \log L, \log M, \log m)$. First-order relative variance is

$$\text{Var}(\log |\hat{G}|) \approx a^\top \Sigma_u a, \quad a = (1, 1, 2, -1, -1, -1)^\top.$$

Cross-covariances matter when geometry or calibration standards are shared. Adding marginal variances while discarding covariance is not evidence of independence.

Counterexample 3: deflection does not prove ontology (physics).

A nonzero spot shift can arise from gravitational torque, thermal drift, tilt, magnetic coupling, electrostatics, fiber creep, readout bias, or source-motion disturbance. Deflection is read data. Attribution to a parameter is conditional on calibration, controls, alternative-model tests, and uncertainty; the needle does not name its cause.

Two-pass calibration compares a direct route and a stepped route through independently recorded intermediate quantities. The direct route maps calibrated equilibrium data to $\widehat{G}_d$. The stepped route first estimates $\widehat{\kappa}$, $\widehat{\theta}$, and geometry, then composes them into $\widehat{G}_s$. Their raw data, preprocessing, shared calibrations, covariance, numerical traces, budgets, and provenance remain visible.

The comparison reports independent rows:

exact equality	$\widehat{G}_d = \widehat{G}_s$ after typed conversion,
resolution equality	$q_\eta(\widehat{G}_d) = q_\eta(\widehat{G}_s)$,
uncertainty compatibility	$\mathcal{U}_d \cap \mathcal{U}_s \neq \emptyset$.

Each passes, fails, or remains unresolved. Physical floating estimates rarely satisfy exact equality, and interval overlap is not exact same-number. Shared apparatus can correlate routes, so agreement is not automatically independent evidence or same-process.

Counterexample 4: numerical convergence is not model validation (physics). A nonlinear fit may converge tightly under an incorrect geometry or unmodeled drift. Stable parameters and small iteration steps demonstrate numerical behavior conditional on the objective; residual structure, held-out source positions, calibration checks, and alternative models test the physical claim.

Budgets distinguish acquired samples, source reversals, calibration runs, fit iterations, objective evaluations, arithmetic growth, covariance estima-

tion, systematic-model variants, and uncertainty propagation. Stops include COMPLETE, MISSING-CALIBRATION, UNIT-MISMATCH, GEOMETRY-INVALID, READOUT-SATURATION, NOT-EQUILIBRATED, NONIDENTIFIABLE, FIT-BUDGET, NONCONVERGENT, COVARIANCE-INVALID, SYSTEMATIC-UNRESOLVED, and PROVENANCE-MISMATCH. A resource stop does not become a zero force or null result.

A certificate records apparatus identity and geometry, units, calibration standards and dates, raw and rejected samples, source schedule, pre-registration, blinding, environment, readout and torque models, parameter transforms, iteration trace, residuals, convergence diagnostics, covariance, systematic-error budget, alternative models, two-pass comparison rows, budgets, stops, checked scope, unresolved frontier, protocol and version, run identity, platform, and deterministic and declared nondeterministic conditions.

Governance fixes apparatus, force and readout models, geometry convention, units, calibration transfer, sampling and source schedule, preprocessing, estimator, convergence criteria, covariance and uncertainty methods, alternative models, two-pass comparison, budgets, stop priority, and certificate grammar. A changed field starts a new report.

The internal ratio of the preceding section is separate. Its quotient and remainder neither cause a torque nor measure G . They are not an external alpha value, orbit, field, spinor, force law, or variational identity. Connecting an internal numerical report to this balance requires a typed, calibrated, tested bridge.

The action ledger is exact. Apparatus, physical and readout models, calibration, geometry, preprocessing, estimator, uncertainty, comparison relations, budgets, and certificate grammar are *posited*. Conditional equilibrium and oscillation implications, readout conversion, dimensional checks, numerical iterations, covariance propagation, uncertainty construction, and validation implications are *demonstrated*. Raw samples, calibrations, deflec-

tions, periods, fits, residuals, covariance estimates, comparison rows, stops, and provenance are *read*. A cause from deflection alone, model truth from convergence, route independence from agreement, a physical force from an internal remainder, identification with an external alpha, and untested orbit, field, spinor, force, or variational interpretations remain *reserved*.

10.3 Slicing the same ledger

One exact ledger can support several numerical views without making those views interchangeable. Use the normalized record

$$\mathcal{L} = (n, d), \quad n \in \mathbb{Z}, \quad d \in \mathbb{Z}_{>0},$$

with exact interpretation $v(\mathcal{L}) = n/d$. If raw input has negative denominator, normalization negates both entries; denominator zero stops before a ledger exists.

The quotient and remainder slice is

$$\pi_{\text{div}}(\mathcal{L}) = (q, r), \quad n = qd + r, \quad 0 \leq r < d.$$

The scaled-floor slice at $M > 0$ is

$$\pi_M(\mathcal{L}) = (Q_M, R_M), \quad nM = Q_M d + R_M, \quad 0 \leq R_M < d.$$

The exact-value slice retains (n, d) or its reduction by a positive greatest common divisor. A display slice retains only a formatted label derived from Q_M and therefore loses R_M , denominator, and exact rational unless those fields accompany it.

The reciprocal slice has a stricter domain:

$$\pi_{\text{inv}}(\mathcal{L}) = \begin{cases} (d, n), & n > 0, \\ (-d, -n), & n < 0, \end{cases}$$

followed by positive-denominator normalization. If $n = 0$, the original ratio is valid but its reciprocal is not; the inverse slice stops ZERO-NUMERATOR. This failure does not invalidate quotient or scaled-floor slices already certified.

Slicing differs from iteration. A slice is a projection of one ledger. The Euclidean algorithm evolves a state:

$$(a_k, b_k) \mapsto (b_k, r_k), \quad a_k = q_k b_k + r_k, \quad 0 \leq r_k < b_k,$$

while $b_k > 0$. The trace terminates when $r_k = 0$, yielding a greatest common divisor. Reducing (n, d) by that divisor creates a canonically normalized value record; it does not reconstruct any information discarded by an earlier display projection.

The data structure ties both methods together. Each iterative state has the same numerator/positive-denominator shape needed by the division slice, while every iteration step stores its quotient/remainder certificate. Conversely, the initial quotient slice is only the first row of the Euclidean trace when further iteration is requested. The ledger therefore supplies common exact data, but route and history fields distinguish one-shot slicing from repeated state transition.

Worked example 1: several slices, one ledger (computation). For $\mathcal{L} = (17, 5)$,

$$\pi_{\text{div}} = (3, 2), \quad \pi_{10} = (34, 0), \quad \pi_{\text{inv}} = (5, 17).$$

These outputs answer whole-part, scaled-display, and reciprocal questions. Their agreement through the original ledger does not make 3, 34, and 5/17 the same number.

Worked example 2: Euclidean iteration (computation). Starting from $(17, 5)$,

$$17 = 3 \cdot 5 + 2, \quad 5 = 2 \cdot 2 + 1, \quad 2 = 2 \cdot 1 + 0.$$

The iterative trace yields greatest common divisor 1. Keeping only first quotient 3 erases later quotients, remainders, and the termination certificate.

Counterexample 3: valid ratio, invalid reciprocal (computation).

For $(0, 7)$, division returns $q = 0, r = 0$, and every positive scale returns $Q_M = 0, R_M = 0$. Reciprocal formation fails because it would create denominator zero. A single global success bit would conceal this slice-specific failure.

Counterexample 4: sign and truncation mismatch (computation).

For $(-7, 3)$, floor division gives

$$-7 = (-3) \cdot 3 + 2.$$

Truncation toward zero gives quotient -2 with signed leftover -1 , a different convention. A parser that accepts both without a convention tag cannot replay the remainder range or compare reports.

Counterexample 5: formatted collision (computation). At $M =$

100, $1/3$ and $334/1000$ may both format as 0.33 under a two-decimal floor display. They are resolution-equivalent under that formatter, not exactly equal. Exact cross-products and retained remainders expose the difference.

Arithmetic budgets distinguish multiplication, division, comparison, greatest-common-divisor iterations, reciprocal normalization, formatting, storage, and replay. Bit-growth bounds are checked before forming nM or cross-products. Under bounded arithmetic, overflow stops rather than wraps. Stops include COMPLETE, ZERO-DENOMINATOR, ZERO-NUMERATOR, INVALID-SCALE, SIGN-CONVENTION-MISMATCH, ITERATION-BUDGET, ARITHMETIC-BUDGET, BIT-GROWTH, OVERFLOW, FORMATTING-FAILURE, IDENTITY-FAILURE, and RANGE-FAILURE.

A replay certificate records raw and normalized ledgers, domains, sign and division conventions, requested slices, scale and formatter, products, quotients, remainders, reciprocal normalization, every Euclidean state, bit lengths, budgets, projections, erasures, stops, checked scope, and unresolved frontier. Validation reports domain, quotient, remainder, scaled-floor, inverse, iteration, normalization, formatting, resource, and claim rows independently as pass, fail, or unresolved.

Comparison retains three value relations. Exact same-number is cross-product equality after typed normalization. Resolution equality is equality of labels under one declared formatter. Uncertainty compatibility is overlap of declared regions after unit and domain conversion; integer truncation alone supplies no uncertainty model. Each row is independent, and process equality additionally requires matching inputs, requested slices, Euclidean transitions, traces, resource currencies, configurations, and provenance.

Governance fixes ledger schema, domain, sign normalization, floor or truncation convention, scales, reciprocal domain, Euclidean transition, canonicalization, formatter, arithmetic model, growth limits, requested slices, budgets, stop priority, comparison relations, and certificate grammar. Provenance records raw data, versions, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

This ledger is computational. Quotient, remainder, trace length, and re-

source use are separately read fields. Cost is not the remainder; deterministic behavior is conditional on governed inputs and configuration; and no serial fraction or performance ceiling is identified with an arithmetic residue. No physical interpretation follows.

The action ledger is exact. Ledger schema, conventions, slices, scale, inverse domain, iteration, formatter, arithmetic model, budgets, and certificate grammar are *posited*. Division identities, scaled-floor identities, reciprocal implications, decreasing-remainder termination, normalization equality, cross-product comparison, and certificate implications are *demonstrated*. Inputs, quotients, remainders, traces, labels, bit lengths, resources, stops, and validation rows are *read*. A reciprocal at zero, reconstruction from a lossy display, exact equality from formatting, uncertainty from truncation, cost as residue, unconditional run identity, a serial fraction as residue, and all physical meaning remain *reserved*.

Chapter 11

The Meter

11.1 What a heartbeat counts

A heartbeat is an operational counter exposed by a particular language toolchain. In the configuration considered here it behaves as an allocation or effort counter consumed by bounded elaboration, reduction, search, and related internal work. Its exact increment policy is toolchain-defined and may change with compiler release, libraries, optimization, cache state, command form, or configuration. Therefore a reading is not merely h , but

$H = (h, \text{counter semantics, release, dependencies, configuration, input, run conditions}).$

Several currencies must remain distinct:

logical step	one transition in a declared abstract procedure,
reduction/search step	one event in a declared operational trace,
allocation counter	a toolchain-defined charged event,
wall time	elapsed clock duration,
CPU time	processor time charged by the operating environment,
energy	joules measured by calibrated physical instrumentation.

No fixed conversion among these follows from their names. A counter can change while wall time falls, or remain stable while wall time varies. Energy requires a physical meter and calibration.

The measurement protocol uses a controlled workload family W_j . For each workload it records a requested counter budget B_j , runs the operation, and obtains

$$(o_j, h_j, \sigma_j),$$

where o_j is the returned report, h_j the spent counter, and σ_j the stop. Requested, spent, and remaining budget are retained separately. A budget exhaustion result reports checked scope and unresolved frontier; it is not a completed cost.

Calibration experiment 1: baseline subtraction (computation). Measure an empty or minimal harness W_0 and a target W_1 under the same governed conditions. Define

$$\Delta h = h_1 - h_0.$$

This subtraction removes only charges common and additive under the protocol. It does not prove that every overhead is shared, that counter attribution is linear, or that Δh belongs exclusively to the target. Interleaved baseline and target runs can expose drift.

Calibration experiment 2: workload scaling (computation). Choose workloads with declared structural sizes $n_1 < \dots < n_k$, measure counter vectors, and compare candidate models such as $a + bn$ or $a + bn \log n$. Numerical fitting iterates parameter estimates and slices residuals by workload, run order, and configuration. A good fit within the calibrated range is descriptive; it is not a universal complexity law or a conversion from logical steps to counter units.

Warm and cold conditions are separate strata. A cold run may include parsing, loading, cache construction, and initial allocation that a warm run

reuses. A warm run may inherit mutable state from earlier workloads. The protocol declares process lifetime, cache policy, workload order, randomization, number of repetitions, and whether each sample is cold, warm, or mixed. Discarding a first run without preregistration is a data-dependent slice.

Counterexample 3: same input, different configuration (computation). Two runs receive identical source text but use different releases or dependency sets. Equal outputs do not make their heartbeat counts directly comparable. A process-comparison row fails or remains unresolved until counter semantics and configuration are matched or a calibrated conversion is demonstrated.

Counterexample 4: stable counter, variable wall time (computation). Repeated runs may return the same allocation count while elapsed time varies with scheduling, processor frequency, contention, or thermal state. This is consistent: the counter and clock measure different currencies. Neither one is a surrogate for energy without separate calibration.

Shannon information supplies a careful description framework. If a declared random variable X over workload descriptions has law $p(x)$, then

$$H(X) = - \sum_x p(x) \log_2 p(x).$$

For a prefix code with lengths $\ell(x)$, expected description length is $\mathbb{E}[\ell(X)]$. These are bits under a declared distribution and code. Heartbeat counts may be statistically modeled against description lengths, but a counter is not Shannon information, entropy, probability, or code length. A correlation requires data and uncertainty; it does not identify the quantities.

The numerical audit iterates over configurations, workloads, repetitions, and cold/warm strata. It slices readings by outcome, stop, configuration, run order, and resource class; retains rejected and failed runs with reasons; computes means, medians, dispersion, paired differences, covariance, and

model residuals only where the sampling design licenses them. Deterministic inputs can still meet nondeterministic scheduling, randomized search, address layout, parallel execution, or other environmental state.

Stops include COMPLETE, COUNTER-BUDGET, SEARCH-BUDGET, ALLOCATION-FAILURE, ARITHMETIC-OVERFLOW, TOOLCHAIN-MISMATCH, DEPENDENCY-MISMATCH, CONFIGURATION-MISMATCH, CACHE-STATE-UNKNOWN, NONDETERMINISTIC-INPUT-UNKNOWN, OUTPUT-FAILURE, and PROVENANCE-INCOMPLETE. A failed output and an exhausted resource are different statuses; neither is silently replaced by a numeric cost.

The certificate records counter semantics, release and compiler identity, dependency lock and versions, snapshot hash, command and build configuration, workload representation, structural size, requested/spent/remaining budgets, output, stop, cold/warm state, process lifetime, cache policy, run order, random seeds and schedules, counter readings, wall and CPU timing when collected, physical-meter provenance for energy if collected, rejected samples, checked scope, unresolved frontier, and validation slices.

Comparison follows three value relations. Exact equality compares integer counter readings only after matched counter semantics. Resolution equality compares labels under a declared binning or formatter. Uncertainty compatibility compares declared sampling or calibration regions. Each independently passes, fails, or remains unresolved. Same-process additionally requires matching workload, transitions, counter policy, releases, dependencies, configuration, cache and warmup state, deterministic inputs, declared nondeterministic inputs, and provenance.

Governance fixes workload family, counter semantics, releases, dependencies, configuration, budget, cold/warm protocol, repetition and ordering, baseline rule, statistical summaries, model family, binning, uncertainty, stop priority, and certificate grammar. A changed field creates a new measurement series.

The action ledger is exact. Counter interpretation, workloads, baselines,

configurations, sampling, models, Shannon law and code, comparisons, budgets, and certificate grammar are *posited*. Integer subtraction, conditional baseline implications, fitted-model arithmetic, statistical summaries, entropy and expected-length calculations, and certificate implications are *demonstrated*. Outputs, counters, timings, configurations, cache strata, rejected runs, residuals, stops, and provenance are *read*. A logical-step conversion without calibration, counter as information or physical time, energy without a physical meter, universal complexity from a finite fit, identical counts across uncontrolled runs, and deterministic sameness from input text alone remain *reserved*.

11.2 Warmup, read, subtract

A paired cost experiment has four phases: register the workloads, perform declared setup, execute warmup, and read a baseline–target pair. Registration fixes representations, counter semantics, budgets, and evaluation order before any reading. Setup may load dependencies, initialize caches, or allocate shared tables. Warmup is a measured and reported phase, not evidence discarded merely because its cost is inconvenient.

For pair i , let b_i be the baseline count and t_i the target count under matched conditions. The signed residual is

$$\delta_i = t_i - b_i \in \mathbb{Z}.$$

A negative residual is valid arithmetic and may indicate noise, ordering effects, cache reuse, or a baseline that is more expensive under that condition. Clamping δ_i to zero would change the data. Integer subtraction is exact; attribution of the difference is conditional.

For $N \geq 2$ completed valid pairs, the paired estimator is

$$\bar{\delta} = \frac{1}{N} \sum_{i=1}^N \delta_i = \bar{t} - \bar{b}.$$

Under a declared independent or exchangeable-pair model, its sampling variance is estimated from paired differences,

$$s_{\delta}^2 = \frac{1}{N-1} \sum_{i=1}^N (\delta_i - \bar{\delta})^2, \quad \widehat{\text{Var}}(\bar{\delta}) = \frac{s_{\delta}^2}{N}.$$

Within one pair,

$$\text{Var}(T - B) = \text{Var}(T) + \text{Var}(B) - 2 \text{Cov}(T, B).$$

This identity handles within-pair dependence only. It does not account for correlations across pairs induced by blocks, drift, cache history, or run order. With residual covariance matrix Σ_{δ} ,

$$\text{Var}(\bar{\delta}) = \frac{1}{N^2} \mathbf{1}^T \Sigma_{\delta} \mathbf{1}.$$

When full covariance is not estimable, a preregistered clustered or block method must match the acquisition design.

Interleaving controls drift. A balanced sequence such as B, T, T, B can be randomized within blocks, with run order retained. Repetitions are sliced by cold/warm state, block, outcome, configuration, and stop. A paired row is valid only when both members complete under the pairing policy; failed or censored partners remain in the audit and are not silently replaced.

Worked example 1: signed paired residuals (computation). Suppose three matched pairs give

$$(b_i, t_i) = (573, 552), (570, 575), (580, 579).$$

Then $\delta = (-21, 5, -1)$. These values support exact signed arithmetic and a paired summary. They do not prove that the first target has negative intrinsic cost; the attribution model must explain shared and order-dependent charges.

Counterexample 2: subtracting unmatched runs (computation).

A cold baseline from one configuration and a warm target from another can be subtracted as integers, but the result is not a calibrated paired residual. The configuration and warm-state validation rows fail, so interpretation of the difference remains rejected or unresolved.

Shared-overhead cancellation assumes an additive model,

$$B_i = O_i + \epsilon_{Bi}, \quad T_i = O_i + C_i + \epsilon_{Ti}.$$

Then $\delta_i = C_i + \epsilon_{Ti} - \epsilon_{Bi}$. If setup interacts with the target, if caches change C_i , or if O_i differs within a pair, subtraction does not isolate C_i . Additivity and shared overhead must be tested with controls, order reversals, and residual diagnostics.

Two numerical routes are kept separate. The direct route reads target and baseline in one paired block and subtracts. The stepped route records setup, warmup, baseline, intermediate workload, and target, then composes declared differences. For example,

$$(t - m) + (m - b) = t - b$$

is an exact arithmetic identity when all three readings belong to one compatible currency and pairing context. It does not make the direct and stepped

acquisition processes identical.

Worked example 3: direct and stepped comparison (computation).

Let $\delta_d = t - b$ and $\delta_s = (t - m) + (m - b)$. Exact equality follows algebraically if the same b, m, t are reused. If the routes acquire separate readings, equality becomes an empirical comparison with covariance and provenance. Near agreement under a formatter is only resolution equality, not same-process.

Build-to-build reproducibility is scoped to matched counter semantics, compiler and dependency releases, snapshot, configuration, workload representations, budgets, setup, warmup, cache policy, run order, deterministic inputs, and declared nondeterministic conditions. Two builds may return equal tuples under this governed relation. Equality is read; matching configuration is posited and validated from provenance. A mismatch does not identify its cause without further slicing.

The comparison reports independent rows:

exact equality	$\delta_1 = \delta_2,$
resolution equality	$q_\theta(\delta_1) = q_\theta(\delta_2),$
uncertainty compatibility	$\mathcal{U}_1 \cap \mathcal{U}_2 \neq \emptyset.$

Each passes, fails, or remains unresolved. Exact equality of residuals does not imply same-process; that relation also requires compatible inputs, transitions, traces, resource currencies, setup, warmup, configurations, and provenance.

The numerical algorithm iterates blocks and pairs, validates membership, computes signed differences, updates exact sums and covariance accumulators, and slices results by all governed strata. It retains raw pair order and can replay summaries from the raw table. Overflow and bit-growth checks precede accumulator updates.

Budgets distinguish setup, warmup, baseline and target readings, repetitions, block count, counter allowance, arithmetic growth, covariance estimation, and replay. Stops include COMPLETE, REGISTRATION-FAILURE, SETUP-

FAILURE, WARMUP-BUDGET, COUNTER-BUDGET, UNPAIRED-RUN, OUTPUT-FAILURE, CONFIGURATION-MISMATCH, CACHE-STATE-UNKNOWN, OVERFLOW, INSUFFICIENT-PAIRS, and PROVENANCE-INCOMPLETE. Resource failure is not a numeric residual.

A certificate records registered workloads, counter semantics, releases, dependency lock, snapshot hash, configuration, setup and warmup traces, cache/process policy, raw baseline/intermediate/target readings, pair and block identifiers, order and randomization, signed residuals, exact sums, covariance, rejected and failed runs, direct and stepped routes, budgets, stops, checked scope, unresolved frontier, protocol, run identity, platform, deterministic inputs, declared nondeterministic inputs, completed-valid-pair count, and dependence-model validation.

Governance fixes workload representations, pairing, registration, setup, warmup, interleaving and randomization, baseline, target and intermediate roles, subtraction orientation, additive model, summaries, uncertainty, direct and stepped comparisons, budgets, stop priority, and certificate grammar. A changed field begins a new measurement series.

The action ledger is exact. Pairing, workloads, roles, setup, warmup, additive model, routes, sampling, comparisons, budgets, and certificate grammar are *posited*. Signed subtraction, telescoping identities, paired summaries, covariance arithmetic, conditional cancellation, sufficient-pair and dependence-model validation, and certificate implications are *demonstrated*. Raw counts, pair order, residuals, covariance estimates, configuration rows, route comparisons, stops, and provenance are *read*. A fixed structural-rank conversion, isolated target cost without tested additivity, same-process from near or exact residual agreement, naive s_δ^2/N uncertainty under correlated pairs, unconditional build reproducibility, self-energy, particle identity, alpha, and physical coupling remain *reserved*.

11.3 Self-application without mystification

Self-application begins by separating five objects:

- P an object program or workload,
- $\lceil P \rceil$ a finite representation of that program,
- E an evaluator under a declared toolchain and configuration,
- M a meter that observes declared resource counters during evaluation,
- $\mathcal{R}$ the resulting value, trace, counters, stop, and provenance.

An experiment may evaluate $E(P, x)$, $E(P, \lceil P \rceil)$, or $E(E, \lceil E \rceil)$ if the domains admit those inputs. These are different workloads. Saying that a machine measures itself is incomplete until the object, representation, evaluator, meter boundary, and input are specified.

The meter is not outside all influence. Instrumentation can change allocation, search order, representation size, caching, and evaluation path. A calibration therefore compares an unmetered or minimally metered control with the metered workload under matched conditions and reports the intervention. It does not assume measurement overhead is additive or transparent.

Bounded evaluation has state

$$S_k = (e_k, \rho_k, F_k, B_k, T_k),$$

where e_k is the current expression or task, ρ_k the environment, F_k transition fuel, B_k a resource vector, and T_k the trace. Each admitted transition updates state and decreases fuel. Stops include a value, explicit failure, domain error, fuel exhaustion, resource exhaustion, or an unresolved external action. The fuelled wrapper terminates; that does not decide whether the corresponding unbounded evaluation terminates.

Self-input increases several risks. Representation can expand before evaluation, quotation and interpretation may fail to round-trip, diagonal construction can generate a workload unlike the original, and evaluation can

recurse without reaching a value. The report therefore retains representation bytes or symbols, parser and decoder decisions, round-trip checks, every self-input boundary, growth, and the precise stop. A timeout or fuel stop is not semantic evidence of divergence.

Worked example 1: representation round trip (computation). Given P , form $\ulcorner P \urcorner$, decode it to P' , and compare P' with P under a declared structural relation. Passing certifies the representation round trip. It does not certify that $E(P, \ulcorner P \urcorner)$ halts or returns the same value as $E(P, P)$, which may not even be well-typed.

Worked example 2: bounded self-input (computation). Run $E(P, \ulcorner P \urcorner)$ with fuel N . If it returns value v after $k \leq N$ transitions, replay certifies that finite trace. If fuel reaches zero first, the report owns the prefix and unresolved next state only. Increasing N creates a new run and may reveal a value; the earlier stop remains valid within its scope.

Counterexample 3: self-description is not consciousness. A text can contain or reproduce its own representation through an explicit quoting scheme. That is a relation among finite strings, decoders, and evaluations. It supplies no operational definition, apparatus, observation, or theory of consciousness and therefore licenses no such conclusion.

Turing's halting result is precise: no total algorithm decides for every arbitrary program–input pair whether its unbounded evaluation halts. A particular bounded trace, or a termination certificate for a restricted evaluator, does not solve that universal problem. Conversely, the universal impossibility does not prevent termination demonstrations or successful runs for specific restricted workloads.

Fix a universal prefix machine U . Chaitin complexity is

$$K_U(x) = \min\{|p| : U(p) = x\}.$$

It is generally uncomputable. Exhibiting a self-description of length L demonstrates only $K_U(x) \leq L$. For universal prefix machines U, V , invariance supplies constants independent of x :

$$K_U(x) \leq K_V(x) + c_{UV}, \quad K_V(x) \leq K_U(x) + c_{VU}.$$

The difference is uniformly bounded in both directions, but there is no exact shift and no machine-independent complexity value. Incompressibility, randomness, or a lower bound requires separate evidence. Metered evaluation cost is not program length or K_U , and the report names the chosen U .

Counterexample 4: a short self-description is only an upper bound.

If a description of length 500 reproduces report x , then $K_U(x) \leq 500$ for the declared U . Another description may be shorter. The invariance inequalities bound changes between universal prefix machines but do not make their values equal; the report must name the convention.

The numerical metering algorithm iterates representation, parsing, evaluation, and replay states. It slices counters by setup, representation, decoding, target evaluation, meter overhead, and reporting where the counter semantics permit. Each slice preserves requested, spent, and remaining budgets. Shared work and interaction terms are retained rather than assigned to one slice by fiat.

This evaluator–representation relation is internal to a model and supplies no name bridge to the world. If a physical meter or robot is introduced, its covariant action instead records

$$(N, s) \longmapsto (N', s', y, \text{interaction trace}),$$

including back-action, sensor loading, actuation, energy or material exchange, latency, hysteresis, drift, saturation, geometry, calibration, environment, prior state, rejected outcomes, and alternative causal models. Observation y an-

swers that intervention in changed world N' , not an untouched world.

Comparison reports exact, resolution, and uncertainty rows separately. Exact equality compares values or integer counters only after typed conversion and matched counter semantics. Resolution equality compares declared labels. Uncertainty compatibility compares sampling or calibration regions. Same-process additionally requires compatible object, representation, evaluator transitions, meter intervention, traces, resource currencies, releases, configuration, deterministic inputs, declared nondeterministic inputs, and provenance. Equal outputs or nearby costs do not establish it.

Budgets distinguish representation length and growth, parser work, evaluation fuel, recursion depth, allocation, search, trace storage, meter overhead, replay, and report construction. Stops include COMPLETE, REPRESENTATION-FAILURE, DECODE-FAILURE, ROUNDTrip-FAILURE, DOMAIN-ERROR, FUEL, RECURSION-BOUND, ALLOCATION-BOUND, SEARCH-BOUND, GROWTH-BOUND, TRACE-CAPACITY, EXTERNAL-ACTION-UNRESOLVED, and PROVENANCE-INCOMPLETE.

A certificate records object and representation schemas, exact represented input, evaluator and meter boundaries, toolchain and dependency releases, snapshot hash, configuration, quotation and decoding rules, round-trip relation, initial state, every transition, meter intervention, counter slices, growth, requested/spent/remaining budgets, value or failure, stop, checked scope, unresolved frontier, rejected runs, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Governance fixes object domain, representation, evaluator, meter semantics and boundary, self-input form, equality and round-trip relations, fuel and resource policy, slicing, comparison, budgets, stop priority, and certificate grammar. A changed field creates a new experiment.

The action ledger is exact. Objects, representations, evaluator, meter, self-input form, universal machine, resource policy, comparisons, budgets, and certificate grammar are *posited*. Round-trip implications, bounded termination, transition replay, counter arithmetic, exhibited-description up-

per bounds, and certificate implications are *demonstrated*. Representations, parser outcomes, transitions, values, failures, counters, slices, growth, stops, and provenance are *read*. Consciousness, self-energy, universal halting, incompressibility, randomness, a physical coupling, process identity from equal outputs, transparent instrumentation, and intrinsic same-on-every-machine behavior remain *reserved*.

Chapter 12

The Field Closes

12.1 Second differences

An indexed sample ledger is

$$\mathcal{F} = [(x_0, f_0), \dots, (x_{N-1}, f_{N-1})],$$

with strictly increasing coordinates and values in a declared arithmetic domain. At an interior index $1 \leq k \leq N - 2$, the unscaled second difference is

$$\Delta^2 f_k = f_{k+1} - 2f_k + f_{k-1}.$$

The lower, middle, and upper roles are positional: f_{k-1}, f_k, f_{k+1} . Reordering samples or substituting three unindexed values changes the operation.

On a uniform grid $x_k = x_0 + kh$, $h > 0$, the centered approximation is

$$D_h^{(2)} f_k = \frac{f_{k+1} - 2f_k + f_{k-1}}{h^2}.$$

If a sampled function has four continuous derivatives near x_k , Taylor expansion gives

$$D_h^{(2)} f_k = f''(x_k) + \frac{h^2}{12} f^{(4)}(\xi_k)$$

for a point ξ_k in the stencil interval. Thus second-order truncation requires smoothness and a demonstrated bound on $f^{(4)}$. Three numbers alone do not provide that bound or identify a derivative.

For a nonuniform grid, let $h_- = x_k - x_{k-1} > 0$ and $h_+ = x_{k+1} - x_k > 0$. The separate three-point stencil is

$$D_{\text{nu}}^{(2)} f_k = \frac{2}{h_- + h_+} \left(\frac{f_{k-1}}{h_-} - \left(\frac{1}{h_-} + \frac{1}{h_+} \right) f_k + \frac{f_{k+1}}{h_+} \right).$$

Using the uniform formula on unequal spacing is a model error, not a small implementation detail. Its truncation order depends on mesh regularity and smoothness and must be demonstrated for the declared grid family.

Exact integer arithmetic computes $\Delta^2 f_k$ without rounding, subject to resource growth. Exact rational arithmetic computes scaled stencils after positive-denominator normalization and cross-products. Floating arithmetic introduces rounding and possible cancellation:

$$\widehat{\Delta^2 f_k} = \text{fl}(\text{fl}(f_{k+1} - 2f_k) + f_{k-1}).$$

The certificate records evaluation order, format, rounding mode, exceptional values, and an error analysis. Algebraic equivalence over exact numbers does not guarantee bitwise equality under floating evaluation.

Boundary indices $k = 0, N - 1$ lack one centered neighbor. Valid policies include stopping with BOUNDARY, using a separately derived one-sided stencil with enough samples, supplying ghost values through a declared boundary model, or restricting output to interior indices. Silent wraparound asserts periodic boundary conditions; silent padding asserts a value outside the sampled domain.

The iteration algorithm traverses interior indices, slices the ledger into overlapping triples, validates spacing, and emits

$$\mathcal{D} = [(k, x_k, f_{k-1}, f_k, f_{k+1}, c_-, c_0, c_+, d_k)].$$

Here c_-, c_0, c_+ are the coefficients actually consumed and d_k the computed result. Stored coefficients that are never multiplied into the result are meta-data, not evidence that a stencil used them. Replay reconstructs each arithmetic expression from consumed coefficients and sampled values.

Worked example 1: exact uniform difference (computation). For $f_k = k^2$ at $k = 1$,

$$f_0 = 0, \quad f_1 = 1, \quad f_2 = 4, \quad \Delta^2 f_1 = 4 - 2 + 0 = 2.$$

With $h = 1$, the scaled value is 2. This exact arithmetic row agrees with the second derivative of the declared quadratic, but the general derivative implication still depends on the sampling model.

Worked example 2: cancellation and scale (computation). For values $10^{20}, 10^{20} + 1, 10^{20} + 4$, exact integer arithmetic returns 2. A floating format unable to distinguish unit increments near 10^{20} may first round all three samples together and return 0. Reporting the arithmetic model is therefore part of the value.

Counterexample 3: unequal spacing with uniform coefficients (computation). Let $x = (0, 1, 3)$ and $f(x) = x^2$, giving values $(0, 1, 9)$. The unscaled uniform expression gives 7, while the nonuniform formula gives 2, the exact second derivative of this quadratic. A three-point data shape does not license uniform-grid coefficients.

Counterexample 4: coefficients stored but not consumed (computation). A report may list $(1, -2, 1)$ while its result was computed as $f_{k+1} - f_k$. The metadata resembles a second difference, but arithmetic replay fails. The certificate must trace coefficient consumption, not merely coefficient presence.

Counterexample 5: boundary wraparound changes the domain (computation). At $k = 0$, using f_{N-1} as the lower sample creates a periodic stencil. Unless periodicity was declared and validated, the result belongs to a different problem. The correct status is BOUNDARY or a declared one-sided method.

Budgets distinguish samples parsed, indices traversed, coefficient generation, exact arithmetic growth, floating operations, cross-products, storage, replay, and error-bound evaluation. Stops include COMPLETE, INSUFFICIENT-SAMPLES, INDEX-DOMAIN, BOUNDARY, NONMONOTONE-GRID, NONUNIFORM-GRID, ZERO-SPACING, COEFFICIENT-FAILURE, ARITHMETIC-BUDGET, BIT-GROWTH, OVERFLOW, NONFINITE-VALUE, ERROR-BOUND-UNAVAILABLE, and REPLAY-FAILURE.

A certificate records raw ordered samples, index and boundary domain, grid classification, exact spacings, generated and consumed coefficients, arithmetic model and evaluation order, every product and sum, exact or rounded outputs, rounding and truncation bounds, iterative triples, projections and erased fields, budgets, stops, checked scope, unresolved frontier, versions, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Comparison reports independent rows. Exact same-number compares exact outputs after typed conversion. Resolution equality compares labels under a declared quantizer. Uncertainty compatibility compares declared arithmetic and sampling regions. Same-process additionally requires matching samples, indices, grid and boundary models, coefficients, evaluation order, arithmetic, iteration trace, resources, configuration, and provenance.

Governance fixes sample schema and order, index domain, grid and boundary policies, stencil and coefficient generator, arithmetic model, evaluation order, smoothness and error assumptions, iteration and slicing, budgets, stop priority, comparison relations, and certificate grammar. A changed field begins a new report.

This operator is computation-owned. It is not by itself a field equation, physical second variation, curvature, force, mass, or coupling. Those names require a world model and covariant apparatus action with calibration, interaction trace, back-action, uncertainty, and alternative causes. A three-point value alone licenses none.

The action ledger is exact. Samples, ordering, grid, boundary policy, stencil, arithmetic, smoothness assumptions, error model, algorithms, budgets, and certificate grammar are *posited*. Exact difference identities, nonuniform coefficient identities, conditional truncation bounds, iteration and coefficient-consumption implications, error propagation, and certificate implications are *demonstrated*. Samples, spacings, coefficients, arithmetic events, outputs, errors, resources, stops, and provenance are *read*. A derivative without smoothness and scale, uniform accuracy on a nonuniform grid, periodicity from wraparound, an operation from stored metadata, field or curvature meaning, force inference, and every physical name remain *reserved*.

12.2 Physics field equations

Maxwell's equations are a physical model for fields on a declared spacetime domain:

$$\begin{aligned}\nabla \cdot \mathbf{E} &= \rho/\varepsilon_0, & \nabla \cdot \mathbf{B} &= 0, \\ \nabla \times \mathbf{E} &= -\partial_t \mathbf{B}, & \nabla \times \mathbf{B} &= \mu_0 \mathbf{J} + \mu_0 \varepsilon_0 \partial_t \mathbf{E}.\end{aligned}$$

In SI units, $\mathbf{E}$ is volts per metre, $\mathbf{B}$ tesla, ρ coulombs per cubic metre, and $\mathbf{J}$ amperes per square metre. The equations require a spatial domain, time interval, sources, initial conditions, boundary conditions, and constitutive relations. A field solution is conditional on all of them.

In material media, constitutive laws such as

$$\mathbf{D} = \varepsilon \mathbf{E}, \quad \mathbf{B} = \mu \mathbf{H}, \quad \mathbf{J} = \sigma \mathbf{E}$$

introduce measured, possibly tensor-valued, frequency-, temperature-, and history-dependent parameters. Vacuum relations cannot be substituted silently for a material apparatus.

A London model for a superconducting region may be written

$$\partial_t \mathbf{J}_s = \frac{n_s q_s^2}{m_s} \mathbf{E}, \quad \nabla \times \mathbf{J}_s = -\frac{n_s q_s^2}{m_s} \mathbf{B}.$$

Combined with Maxwell equations under stated assumptions, it yields magnetic penetration with characteristic length

$$\lambda_L = \sqrt{\frac{m_s}{\mu_0 n_s q_s^2}}, \quad \nabla^2 \mathbf{B} = \lambda_L^{-2} \mathbf{B}$$

in an idealized homogeneous interior. Meissner expulsion is an empirical and model-dependent magnetic response, requiring temperature, material phase, geometry, preparation history, applied field, and boundary data. It is not the Boolean statement $\nabla \cdot \mathbf{B} = 0$.

Finite-volume discretization integrates conservation laws over cells. Electric flux through a cell boundary is balanced against enclosed charge; magnetic face fluxes are arranged so discrete divergence can be monitored or preserved. Finite differences instead approximate derivatives on a declared grid. Both require oriented geometry, metric factors, material interfaces, boundary conditions, and time-stepping rules. A three-point second difference is one local stencil, not Maxwell's system.

For a linearized discrete problem,

$$A_h u_h = b_h,$$

the solver iterates residuals $r_k = b_h - A_h u_k$ and stops under a declared norm and tolerance. Nonlinear constitutive or material models produce $F_h(u_h) = 0$, requiring a declared linearization, step acceptance, and globalization policy. Algebraic convergence of r_k is solver verification; it does not

validate the physical model.

Divergence constraints receive their own rows. A computed magnetic field reports

$$\eta_{\text{div}} = \|D_h \mathbf{B}_h\|$$

in a declared norm and scale. Small η_{div} is a numerical constraint residual. It is not exact zero unless exact arithmetic and a structural preservation identity demonstrate that result, and it does not establish the other Maxwell equations.

Mesh refinement compares $u_h, u_{h/2}, u_{h/4}$ after interpolation into a common typed space. Exact equality, resolution equality, and uncertainty compatibility remain separate. Observed convergence order is estimated only for compatible nested families and must stabilize with decreasing raw differences and a demonstrated error bound. Refinement without such evidence does not produce a continuum solution.

Worked example 1: one-dimensional penetration model (physics).

For a half-space idealization,

$$B''(x) = \lambda_L^{-2} B(x), \quad B(0) = B_0, \quad B(x) \rightarrow 0 \text{ as } x \rightarrow \infty,$$

the model solution is $B(x) = B_0 e^{-x/\lambda_L}$. A finite-domain numerical experiment replaces infinity with a declared truncation boundary and reports truncation, mesh, solver, material-parameter, and measurement uncertainties. Agreement with exponential decay tests this model within the apparatus range.

Counterexample 2: labels do not establish a PDE. Three finite labels, Boolean flags, or a three-rung trace provide no spatial coordinates, derivatives, sources, boundary data, units, or constitutive law. They cannot demonstrate Maxwell equations, superconductivity, Meissner response, field

closure, or a coupling.

Worked example 3: verification versus validation (physics). Choose a smooth manufactured continuum field, derive source and boundary data from the differential equations, then sample or project that field onto each mesh. Measure discrete residuals, numerical error against the known continuum reference, and observed convergence under refinement. A discrete target may be called exact only when it is separately constructed to satisfy the discrete equations. These tests verify stencil and solver behavior. Separately compare calibrated apparatus observations with predictions, including uncertainty and alternatives. That validates a physical model only within the tested domain; the two activities cannot substitute for one another.

The covariant apparatus action is

$$(N, s) \longmapsto (N', s', y, \Gamma),$$

where N is the prior world, s robot or apparatus state, y the observation, and Γ the interaction trace. The record retains applied fields, actuation, sensor loading, back-action, energy and material exchange, latency, hysteresis, drift, saturation, geometry, calibration, environment, prior state, rejected outcomes, and alternative causal models. The world answers the intervention in N' , not an untouched N .

Counterexample 4: probe loading (physics). A magnetic probe can perturb a small sample or average over a finite sensing volume. Treating y as the pre-intervention point field ignores back-action and geometry. Calibration and a forward sensor model are required before comparing observation with $\mathbf{B}_h$.

The three-point Gauss diagnostic and count-to-three resolution are unrelated. A three-point quadrature rule is a marked numerical side diagnostic with weights and an error model. Count-to-three means three whole

continued-fraction partial quotients and owns the open internal jar [129.6, 137.7]. Neither is a source for the other, and neither proves a field equation.

Budgets distinguish mesh cells, time steps, stencil evaluations, linear and nonlinear iterations, memory, arithmetic growth, refinement levels, divergence checks, apparatus samples, calibration runs, alternative models, and uncertainty propagation. Stops include COMPLETE, DOMAIN-INVALID, BOUNDARY-MISSING, INITIAL-DATA-MISSING, UNIT-MISMATCH, CONSTITUTIVE-MODEL-MISSING, MESH-INVALID, SOLVER-BUDGET, NONCONVERGENT, DIVERGENCE-CONSTRAINT-FAILURE, REFINEMENT-UNSTABLE, CALIBRATION-MISSING, SENSOR-SATURATED, BACK-ACTION-UNRESOLVED, and MODEL-REJECTED.

A certificate records domains, meshes, orientations, units, sources, initial and boundary data, constitutive parameters, discrete operators, consumed coefficients, solver and refinement traces, residual and divergence norms, error bounds, apparatus and robot states, actuation and interaction trace, raw and rejected observations, calibration, covariance, systematics, alternative models, budgets, stops, checked scope, unresolved frontier, releases, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Comparison reports exact same-number only after typed conversion, resolution equality under one declared quantizer, and uncertainty-region compatibility as independent rows. Same-process additionally requires compatible models, domains, boundary data, operators, meshes, solver traces, apparatus actions, calibration, resources, configuration, and provenance.

Governance fixes the physical domain, equations, sources, constitutive laws, units, initial and boundary data, discretization, constraints, solvers, refinement and error methods, apparatus intervention, calibration, uncertainty, budgets, stop priority, comparison relations, and certificate grammar. A changed field begins a new report.

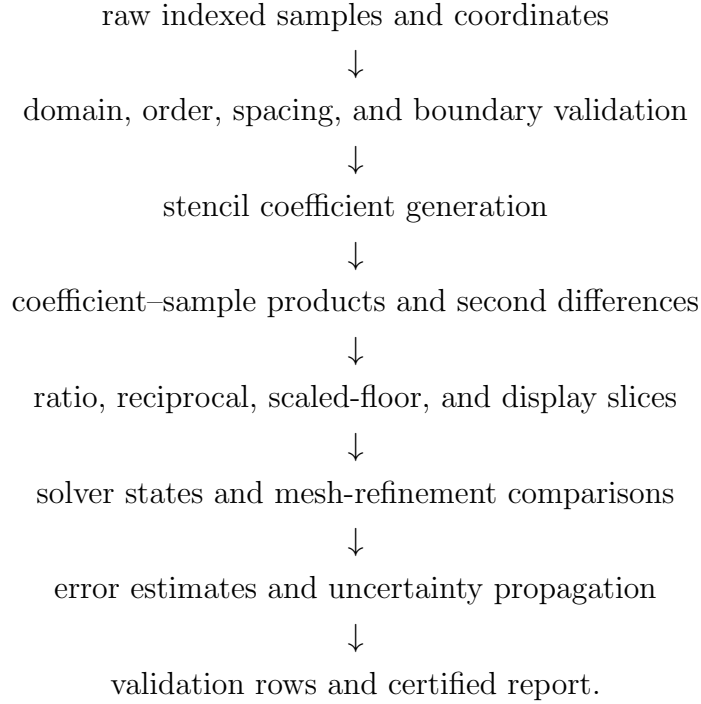
Dimensionless quantities are not thereby internally derivable. This section neither derives an external alpha nor identifies the internal α_c jar with

the laboratory number 137.036; the numbers and provenance remain separate, with no nearness or containment claim.

The action ledger is exact. Equations, domains, sources, material laws, boundary and initial data, discretizations, solvers, apparatus model, calibration, uncertainty, budgets, and certificate grammar are *posited*. Conditional continuum identities, discrete conservation, solver and constraint implications, refinement and error bounds, calibration transforms, and validation implications are *demonstrated*. Samples, fields, residuals, divergence rows, convergence estimates, apparatus interactions, observations, stops, and provenance are *read*. A PDE from finite labels, superconductivity or closure from a Boolean, continuum truth from a finite mesh, validation from solver convergence, an unperturbed world from intervention, internal derivability from dimensionlessness, external alpha, and every uncalibrated physical identification remain *reserved*.

12.3 Iteration and slicing in the field report

A field-shaped numerical report is a dependency ledger, not a physical field. Its data flow is



Every arrow is typed, budgeted, and replayable. Skipping an arrow does not inherit its guarantees.

For stencil index k , the consumed row is

$$S_k = (i_-, i_0, i_+, x_-, x_0, x_+, f_-, f_0, f_+, c_-, c_0, c_+, d_k),$$

with

$$d_k = c_- f_- + c_0 f_0 + c_+ f_+.$$

The report records where each coordinate, value, and coefficient came from. Coefficients stored elsewhere but not used in these three products are not

part of this result. Conversely, an unrecorded multiplier makes arithmetic replay incomplete even if the final numeral looks plausible.

Exact rational outputs use normalized ledgers (n_k, d_k^*) with $d_k^* > 0$; the symbol d_k^* here denotes a denominator and is distinct from the stencil result above. Ratio slices compute Euclidean quotient and remainder, scaled-floor slices compute

$$n_k M = Q_{k,M} d_k^* + R_{k,M}, \quad 0 \leq R_{k,M} < d_k^*,$$

and reciprocal slices require $n_k \neq 0$. Denominator zero rejects the ratio record; numerator zero rejects only the reciprocal slice. A display retaining $Q_{k,M}$ but dropping $R_{k,M}$ is explicitly lossy.

Iteration and slicing play different roles. Index traversal iterates overlapping stencils. A linear or nonlinear solver iterates state vectors and residuals. Mesh refinement iterates discretizations and comparison maps. Slicing projects one state or report into endpoint, quotient, residual, display, constraint, or uncertainty views. A slice cannot replace the iterative trace that produced its source.

The dependency graph has nodes for raw samples, calibration and coordinate transforms, boundary data, coefficients, products, sums, normalized ratios, projections, solver states, refinement transfers, error estimates, and validation claims. An edge records operation, units, arithmetic model, version, and consumed inputs. Claims are licensed only by reachable, validated ancestors. Orphan metadata and cyclic provenance are rejected.

The replay algorithm topologically orders the graph, validates node schemas and edge domains, recomputes each deterministic operation, and compares stored outputs. It then replays solver and refinement sequences in their declared order, verifies stops and budgets, and evaluates claim slices. External or nondeterministic observations are read from the receipt rather than recomputed and carry their acquisition provenance.

Worked example 1: replayed stencil to display (computation). For uniform samples $(0, 1, 4)$, consumed coefficients $(1, -2, 1)$ give second difference 2. Store this as exact ratio $2/1$, scale by $M = 100$, and obtain $Q = 200, R = 0$. The path from samples to display is replayable. Keeping only label 2.00 erases indices, coefficients, exact arithmetic, scale residue, and route.

Counterexample 2: stored stencil, different consumed inputs (computation). A row advertises coefficients $(1, -2, 1)$, but its trace multiplies $(1, -1, 0)$ by the samples. Coefficient metadata passes a presence check while consumption replay fails. The result cannot be certified as a second difference.

Counterexample 3: slice-specific domain failure (computation). A stencil output exactly equal to zero forms valid ratio $0/1$, quotient, remainder, and scaled-floor slices. Its reciprocal is undefined. The report marks only the inverse row failed; it does not discard the valid source ledger or invent infinity.

Worked example 4: refinement iteration and uncertainty (computation). Let compatible meshes produce $u_h, u_{h/2}, u_{h/4}$. Transfer them into a common typed space, retain raw differences, estimate observed order only under a declared same-family error model, and combine demonstrated discretization and arithmetic bounds with input uncertainty. A decreasing residual and stable observed order support the declared numerical claim; they do not validate a physical model.

Bit growth is tracked at every exact multiplication, cross-product, common-denominator conversion, and accumulation. Floating paths record format, rounding, exceptional values, evaluation order, and cancellation estimates. Bounded arithmetic stops before overflow instead of wrapping. Mixed exact and floating paths require an explicit conversion node with a loss bound.

Boundary treatment remains a dependency. A centered stencil at an endpoint stops unless a one-sided rule, ghost value model, or periodic wrap was declared. Missing boundary or initial data blocks downstream solver claims. Likewise, a nonuniform grid cannot consume uniform coefficients without an explicit approximation decision and error analysis.

Budgets distinguish parsed samples, graph nodes and edges, stencil evaluations, arithmetic and bit growth, ratio slices, solver iterations, residual evaluations, mesh levels, transfers, uncertainty propagation, storage, and replay. Stops include COMPLETE, SCHEMA-FAILURE, INDEX-DOMAIN, BOUNDARY-MISSING, NONMONOTONE-GRID, COEFFICIENT-GENERATION-FAILURE, NONUNIFORM-GRID, STENCIL-GRID-MISMATCH, COEFFICIENT-CONSUMPTION-MISMATCH, ZERO-DENOMINATOR, ZERO-NUMERATOR, ARITHMETIC-BUDGET, BIT-GROWTH, OVERFLOW, SOLVER-BUDGET, NONCONVERGENT, REFINEMENT-INCOMPATIBLE, UNCERTAINTY-UNRESOLVED, CONVERSION-FAILURE, MIXED-ARITHMETIC-LOSS-UNBOUNDED, ORPHAN-PROVENANCE, CYCLIC-PROVENANCE, and REPLAY-FAILURE.

A certificate records raw samples and rejected inputs, coordinates, domains, boundary and initial data, coefficient generation and consumption, every arithmetic node, normalized ratio ledgers, requested slices, projections and erasures, solver states, residuals, mesh transfers, convergence and error rows, uncertainty construction, graph versions, grid-stencil compatibility, mixed-arithmetic conversions and loss bounds, provenance acyclicity, domain/coefficient/conversion/provenance validation rows, budgets, stops, checked scope, unresolved frontier, releases, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared non-deterministic inputs.

Comparison reports exact same-number after typed conversion, resolution equality under a declared quantizer, and uncertainty-region compatibility as independent rows. Same-process additionally requires compatible raw data, dependency graphs, consumed inputs, arithmetic, projections, solver and

refinement traces, resources, configuration, and provenance. No combined agreement flag erases which relation passed.

Governance fixes data and graph schemas, domains, boundary policies, coefficient generators, arithmetic and conversion models, ratio and display slices, solvers, refinement and error models, uncertainty, budgets, stop priority, comparison relations, and certificate grammar. A changed field begins a new report.

The computational ledger remains distinct from a physical field and apparatus. Physical naming requires the full covariant action

$$(N, s) \longmapsto (N', s', y, \Gamma)$$

with interaction trace, back-action, sensor loading, actuation, energy and material exchange, latency, hysteresis, drift, saturation, geometry, calibration, environment, prior state, rejected outcomes, and alternative causal models. Projecting these fields away destroys the basis for the physical claim; a compact numerical slice cannot restore it.

The action ledger is exact. Schemas, domains, stencils, graph edges, arithmetic, slices, solvers, refinement and error models, uncertainty, budgets, and certificate grammar are *posited*. Dependency reachability, coefficient-consumption identities, ratio and scaled-floor identities, replay implications, conditional convergence and uncertainty implications, and certificate composition are *demonstrated*. Samples, coefficients actually consumed, arithmetic events, ratios, projections, solver states, residuals, errors, stops, and provenance are *read*. An operation from stored-but-unused coefficients, recovery after lossy projection, reciprocal at zero, convergence as physical validation, a physical field from a field-shaped ledger, apparatus identity after baggage is sliced away, and any claim that a true coupling lies below a numerical floor remain *reserved*.

Part V

The Bound

Chapter 13

Alpha As A Bracket

13.1 The internal formula

The internal constant α_c is a computation-owned ratio in the model of one electron reading. Its consumed inputs are a target integer $t > 0$, a natural-unit integer $R > 0$, and a positive rational distance

$$d = p/q, \quad p, q \in \mathbb{N}_{>0}, \quad p > q.$$

The dependency graph is

$$(p, q) \longrightarrow \frac{p - q}{p}, \quad t \longrightarrow \frac{t}{R}, \quad \left(\frac{p - q}{p}, \frac{t}{R}, R \right) \longrightarrow \alpha_c = \frac{t(p - q)}{pR^2}.$$

Thus

$$\alpha_c^{-1} = \frac{pR^2}{t(p - q)}.$$

Every denominator condition is part of the domain. The formula uses the distance numerator and denominator, target, and natural-unit scale. Other report fields—labels, flags, calibration-tape lengths, field-shaped annotations, or stored coefficients not multiplied into this expression—are unused meta-data for this value.

The second-variation ledger supplies the integers $C = 18$ and $t = 5$ for a crossing condition

$$d_*^2 = C/t = 18/5.$$

This condition determines a model-owned distance question. Numerical routes approximate its positive root and then evaluate the exact rational formula above at their rational distance reports. A stored second-difference value earns no role unless a dependency edge shows that it supplies C or t .

For a rational report p/q , the scaled inverse at $M > 0$ is computed by exact floor division:

$$I_M = \left\lfloor \frac{pR^2M}{t(p-q)} \right\rfloor, \quad pR^2M = I_M t(p-q) + \rho_M, \quad 0 \leq \rho_M < t(p-q).$$

Retaining numerator, denominator, M, I_M, ρ_M makes the display loss replayable. A zero target, $p = q$, or zero scale denominator rejects the inverse slice rather than producing a sentinel number.

Two numerical routes are reported separately. The direct route evaluates the positive crossing in one declared approximation and then slices the inverse ratio. The stepped route iterates rational distances

$$d_0 \rightarrow d_1 \rightarrow \cdots \rightarrow d_k$$

through declared intermediate updates, retaining every residual, accepted and rejected step, denominator growth, and stop before applying the same inverse-ratio slice.

At scale 10^{18} , the direct and stepped reports are exactly

$$\begin{aligned} I_d &= 137011290548979455469, \\ I_s &= 137011290753751157155, \\ I_d &< I_s, \\ I_s - I_d &= 204771701686. \end{aligned}$$

These are unequal integers from different processes. Their difference is read exactly. No claim of nearness, equality, containment, or physical agreement follows.

Worked example 1: replaying one rational distance (computation).

Given $p/q > 1$, compute $p - q$, form numerator $t(p - q)$, denominator pR^2 , normalize by a positive greatest common divisor if requested, and perform scaled inverse floor division. Replay verifies every multiplication, denominator condition, quotient, and remainder. The distance q cancels from the simplified ratio only after the displayed exact algebra; it remains provenance for the distance report.

Counterexample 2: stored fields are not dependencies (computation).

A report can contain a Boolean called closure and a coefficient called coupling while the arithmetic consumes neither. Their presence does not affect α_c . Altering an unused field while the dependency graph and consumed values remain fixed leaves this numerical formula unchanged; any semantic claim attached to that field requires a separate certificate.

Worked example 3: stepped iteration versus final slice (computation).

The stepped route produces a trace of rational states and resource costs. Its last state is then projected to (I_s, ρ_s) . Keeping only I_s erases the iterative route, rejected steps, exact rational, and floor residue. The slice can be compared numerically with the direct slice, but it cannot establish process equivalence.

Richardson extrapolation is not licensed merely because direct and stepped routes yield two values. They are not declared nested mesh levels of one asymptotic family, and no common error expansion or independently stabilized order is supplied. Choosing an order from these two reports to manufacture another value is rejected.

The dependency audit topologically replays raw inputs, crossing data, rational distance construction, exact ratio products, normalization, scaled-floor quotient and remainder, route iterations, projections, and comparison rows. It records generated versus consumed fields and rejects orphan, cyclic, or type-incompatible edges.

Budgets distinguish crossing evaluation, direct approximation, stepped iterations, rational normalization, multiplication and division, denominator and bit growth, scale conversion, storage, comparison, and replay. Stops include COMPLETE, INVALID-TARGET, INVALID-UNIT-SCALE, DISTANCE-DOMAIN, ZERO-INVERSE-DENOMINATOR, ITERATION-BUDGET, ARITHMETIC-BUDGET, BIT-GROWTH, OVERFLOW, NONCONVERGENT, DEPENDENCY-MISSING, UNUSED-FIELD-CLAIM, PROJECTION-LOSS-UNBOUNDED, and REPLAY-FAILURE.

A certificate records C, t, R, p, q , crossing and distance domains, route identity, every direct or stepped state, accepted and rejected updates, consumed inputs, unused stored fields, exact products, normalized ratio, scale, quotient and remainder, bit lengths, projections and erasures, budgets, stops, checked scope, unresolved frontier, releases, dependency lock, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Comparison reports exact equality, resolution equality under a declared quantizer, and uncertainty-region compatibility as independent pass/fail/unresolved rows. Here the exact scaled-integer row fails because $I_d \neq I_s$. Any resolution row states only its quantizer result; no uncertainty row exists without a declared uncertainty model. Same-process additionally requires matching inputs, update law, states, traces, costs, stops, configuration, and provenance; direct and stepped routes do not satisfy that identity.

Governance fixes the internal formula, domains, consumed dependency graph, crossing data, distance representation, direct and stepped algorithms, arithmetic and normalization, scale and projection, budgets, stop priority, comparison relations, and certificate grammar. A changed field begins a new

report.

α_c is the compiler structure constant and model-owned electron reading. It is not the laboratory fine-structure value 137.036, and it is not a physical coupling, self-energy, field strength, or apparatus observation. The model cannot bridge its name to the world's answer. A physical comparison would require a covariant apparatus action, calibrated observation model, uncertainty, interaction trace, back-action, and provenance.

The action ledger is exact. Formula, domains, crossing inputs, distance representation, routes, arithmetic, scale, dependency graph, comparisons, budgets, and certificate grammar are *posited*. Exact ratio identities, floor-division identities, dependency reachability, iteration replay, cross-route arithmetic comparisons, and certificate implications are *demonstrated*. Inputs, route states, consumed fields, unused fields, products, scaled integers, difference, remainders, resources, stops, and provenance are *read*. Equality or nearness of the unequal reports, process identity, Richardson extrapolation without an asymptotic family, physical meaning, external alpha, and every bridge from name resemblance remain *reserved*.

13.2 The count-to-three bracket

The crossing is the positive quadratic surd

$$d_* = \sqrt{\frac{18}{5}} = \frac{\sqrt{90}}{5}.$$

Its simple continued fraction is

$$d_* = [1; \overline{1, 8, 1, 2}],$$

whose convergents begin

$$\frac{1}{1}, \quad \frac{2}{1}, \quad \frac{17}{9}, \quad \frac{19}{10}, \quad \frac{55}{29}, \quad \frac{74}{39}, \dots$$

Each partial quotient is obtained by floor division followed by reciprocal of the positive fractional remainder. The repeating block describes this surd; it is not inferred from decimal formatting.

The recurrence

$$p_k = a_k p_{k-1} + p_{k-2}, \quad q_k = a_k q_{k-1} + q_{k-2}$$

with standard initial pairs generates exact integer convergents. Replay retains every partial quotient, product, sum, numerator, denominator, determinant

$$p_k q_{k-1} - p_{k-1} q_k = (-1)^{k-1},$$

and comparison with the quadratic equation.

The parameter $K = 3$ means three whole partial quotients:

$$[1; 1, 8] = 17/9.$$

The adjacent previous convergent is $2/1$. Exact comparisons give

$$\left(\frac{17}{9}\right)^2 = \frac{289}{81} < \frac{18}{5} < 4 = \left(\frac{2}{1}\right)^2,$$

since $1445 < 1458$ and $18 < 20$. Positivity therefore gives

$$\frac{17}{9} < d_* < 2.$$

The internal inverse-ratio map from the preceding section is

$$I(d) = \frac{R^2}{t} \frac{d}{d-1}, \quad R = 18, \quad t = 5, \quad d > 1.$$

It is strictly decreasing on this domain. Evaluating the two rational walls

exactly yields

$$I(2) = \frac{18^2}{5} \frac{2}{1} = \frac{648}{5} = 129.6,$$

$$I(17/9) = \frac{18^2}{5} \frac{17}{8} = \frac{5508}{40} = \frac{1377}{10} = 137.7.$$

Thus the $K = 3$ owned jar is the open interval

$$\boxed{(129.6, 137.7)}.$$

Open endpoints mean the exact surd crossing is strictly between the rational distance walls and its mapped internal value is strictly between their inverse-ratio images. The endpoints are owned diagnostics computed from adjacent convergents; neither endpoint is reported as the internal point value.

Worked example 1: exact wall ordering (computation). The lower inverse wall is $648/5$, the upper $1377/10$, and

$$\frac{648}{5} < \frac{1377}{10} \iff 6480 < 6885.$$

This orders the report without floating arithmetic. Formatting the rationals as 129.6 and 137.7 is a lossless decimal representation here because both denominators divide 10.

Worked example 2: one more partial quotient is a new report (computation). At $K = 4$, the next convergent is $19/10$ and the adjacent pair changes. The resulting wall computation belongs to a different resolution-depth report. It cannot be inserted into the $K = 3$ certificate while retaining the same stop and ownership.

The finite algorithm iterates Euclidean floor and reciprocal steps, slices the convergent stream at K , selects the adjacent pair, checks straddling against $5d^2 - 18$, applies the exact inverse-ratio map to each wall, reverses

the wall order because the map decreases, and emits an open interval. Every domain, orientation, and ordering check is recorded.

The three-point Gauss–Legendre rule is a separate marked diagnostic. It samples an integrand at three nodes with weights

$$5/9, \quad 8/9, \quad 5/9.$$

Its three counts quadrature nodes, not continued-fraction partial quotients. It neither generates the wall pair nor supplies the interval width, and the jar is not a quadrature remainder. Results from that diagnostic require their own integrand, interval transform, node approximation, arithmetic, and error record.

The older dyadic-grid interval is superseded and is not an alternative owned bracket. No grid seed, denominator doubling, or midpoint bisection participates in this continued-fraction certificate.

Counterexample 3: two threes do not identify algorithms (computation). A $K = 3$ continued-fraction report and a three-node quadrature report both contain the integer 3. Their inputs, iterations, data structures, outputs, error models, and stops differ. Equal counts do not establish same-number or same-process.

Counterexample 4: a wall is not a selected value (computation). Returning 129.6, 137.7, their midpoint, or a formatted internal diagnostic as the jar’s point would add a selection rule absent from the report. The owned object is the ordered open interval and its construction trace.

Budgets distinguish partial quotients, Euclidean divisions, numerator and denominator growth, quadratic comparisons, wall-map arithmetic, scale formatting, storage, and replay. Stops include COMPLETE, INVALID-RADICAND, NONPOSITIVE-REMAINDER, PARTIAL-QUOTIENT-BUDGET, INSUFFICIENT-CONVERGENTS, BIT-GROWTH, OVERFLOW, STRADDLING-FAILURE, WALL-DOMAIN, ORDERING-

FAILURE, MAPPING-FAILURE, and REPLAY-FAILURE.

A certificate records the exact radicand, continued-fraction convention, partial quotients, convergent recurrence and states, determinant checks, K , adjacent-wall selection, exact quadratic comparisons, inverse-ratio map and domain, wall products and reductions, open-endpoint status, projections and formatting, budgets, stops, checked scope, unresolved frontier, releases, dependency lock, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Comparison reports exact equality, resolution equality under a declared quantizer, and uncertainty compatibility as separate rows. The jar itself is an owned interval report, not an uncertainty region unless an uncertainty model explicitly makes it one. Same-process requires identical continued-fraction convention, K , recurrence, trace, wall map, arithmetic, budgets, configuration, and provenance.

Governance fixes the crossing equation, positive-root domain, continued-fraction convention, K , adjacent-wall policy, straddling test, internal inverse map, arithmetic and formatting, budgets, stop priority, comparison relations, and certificate grammar. A changed field begins a new report.

The jar belongs to internal α_c . The laboratory number 137.036 is not inserted, compared by proximity, or claimed to lie inside it. No statement that a true coupling lies below a floor is made. A physical comparison would require the world-facing apparatus, observation model, calibration, uncertainty, interaction trace, back-action, and provenance that the computation cannot supply from its own names.

The action ledger is exact. Crossing, continued-fraction convention, K , wall policy, inverse map, algorithms, arithmetic, budgets, and certificate grammar are *posited*. Surd identities, recurrence and determinant identities, exact straddling, monotonic wall mapping, endpoint derivations, ordering, and certificate implications are *demonstrated*. Partial quotients, convergents, wall values, comparisons, resources, stops, and provenance are *read*.

A selected point from the jar, quadrature as bracket source, dyadic ownership, process identity from equal count, laboratory containment or proximity, physical coupling, and every world answer absent apparatus remain *reserved*.

13.3 Nodes, quadrature, and root finding

Quadrature, root finding, and continued-fraction enclosure answer different questions. A quadrature rule approximates an integral. A root algorithm seeks or encloses a zero of a declared function. A continued-fraction report brackets the internal ratio through rational walls. Shared nodes, counts, or numerals do not merge these reports.

The three-point Gauss–Legendre rule on $[-1, 1]$ uses

$$\xi_1 = -\sqrt{3/5}, \quad \xi_2 = 0, \quad \xi_3 = \sqrt{3/5},$$

and weights

$$w_1 = 5/9, \quad w_2 = 8/9, \quad w_3 = 5/9.$$

For an interval $[a, b]$, map

$$x_j = \frac{a+b}{2} + \frac{b-a}{2}\xi_j$$

and compute the finite weighted sum

$$Q_3(f) = \frac{b-a}{2} \sum_{j=1}^3 w_j f(x_j).$$

This is a marked diagnostic. It is exact for polynomials through degree five under exact nodes, weights, and arithmetic. For general f , an error claim requires smoothness and a demonstrated derivative or analytic bound.

The irrational nodes must be approximated or enclosed. One robust

method brackets $z = \sqrt{3/5}$ by positive rationals $\ell < u$ satisfying

$$5\ell^2 < 3 < 5u^2,$$

then refines by bisection or a safeguarded rational update. The node certificate retains the exact inequalities, iteration trace, stop width, and propagation of node uncertainty through f and the weighted sum. A decimal node without an enclosure supplies only a formatted approximation.

Worked example 1: polynomial quadrature (computation). For $f(x) = x^4$ on $[-1, 1]$,

$$Q_3(f) = \frac{5}{9} \left(\frac{3}{5}\right)^2 + \frac{8}{9}(0) + \frac{5}{9} \left(\frac{3}{5}\right)^2 = \frac{2}{5},$$

which equals the exact integral. This validates the declared rule on this input. It does not make every three-point sum exact.

Root finding begins with a function $g : D \rightarrow \mathbb{R}$ or an exact rational predicate derived from it. A sign-change bracket $[\ell, u] \subset D$ requires

$$g(\ell)g(u) < 0.$$

Existence of an interior root additionally uses continuity on $[\ell, u]$. Uniqueness requires a separate condition, such as a derivative of fixed nonzero sign. Endpoint signs, continuity, and uniqueness are distinct certificate rows.

A safeguarded iteration proposes a Newton, secant, or interpolation step only when it lies in the current bracket and passes domain checks; otherwise it takes the midpoint. After evaluating the new point, it retains the subinterval with opposed endpoint signs. The trace records proposals, fallbacks, residuals, widths, accepted slices, rejected slices, and resource use. A small residual without conditioning information is not a root-error bound.

Worked example 2: exact crossing enclosure (computation). For

$$g(d) = 5d^2 - 18,$$

the rational points $17/9$ and 2 satisfy

$$g(17/9) = -13/81 < 0, \quad g(2) = 2 > 0.$$

Continuity gives a root in the open distance interval $(17/9, 2)$; positivity and strict increase for $d > 0$ identify the positive root as $\sqrt{18/5}$. Mapping these walls through the declared internal inverse formula is the continued-fraction jar construction, not a quadrature result.

Counterexample 3: residual alone is not enclosure (computation).

A point x with small $|g(x)|$ may be far from a root when g is flat, badly scaled, or contaminated by rounding. Without a sign-change bracket or a demonstrated inverse-slope bound, the report owns a residual only.

Refinement studies require one compatible asymptotic family. For nested quadrature partitions or root discretizations with

$$A(h) = A^* + Ch^p + O(h^{p+1}),$$

Richardson extrapolation may cancel the leading term when p is derived or independently estimated from at least three compatible levels, raw differences decrease, observed order stabilizes, and a demonstrated remainder bound holds. Direct and stepped internal routes are not automatically such levels, and choosing p to hit a target is rejected.

Counterexample 4: three nodes are not resolution-depth three (computation). The quadrature rule counts nodes. The internal jar's $K = 3$ counts whole continued-fraction partial quotients. Quadrature nodes and weights neither generate the convergents nor define the jar width. The

jar is not a quadrature remainder.

The external laboratory inverse alpha may be reported as 137.036 only as a separately sourced physical observation summary. It belongs to an apparatus, physical model, units and conventions, calibration chain, uncertainty, data reduction, and provenance. The covariant acquisition is

$$(N, s) \longmapsto (N', s', y, \Gamma),$$

and retains sensor loading, actuation, energy and material exchange, latency, hysteresis, drift, saturation, geometry, environment, prior state, rejected outcomes, back-action, and alternative causal models. The world answers this intervention; the number is not produced by the internal model.

Internal α_c , its direct and stepped diagnostics, its count-to-three open jar, and the laboratory report are different typed objects. This section makes no nearness, containment, prediction, capture, or real-versus-model staging. A shared symbol is forbidden until typed conversion, apparatus calibration, uncertainty, and provenance justify a particular comparison.

The comparison pipeline reports:

exact equality	equality after typed conversion,
resolution equality	equal labels under one declared quantizer,
uncertainty compatibility	overlap of declared regions.

Each independently passes, fails, or remains unresolved. Same-process requires matching inputs, numerical transitions, traces, costs, models, apparatus action, configuration, and provenance. A combined agreement bit may not erase which relation was tested.

Budgets distinguish node-enclosure iterations, function evaluations, weighted arithmetic, root proposals and fallbacks, bracket refinements, exact and floating growth, derivative or continuity checks, asymptotic levels, uncertainty propagation, apparatus samples, and replay. Stops include COM-

plete, node-domain, node-enclosure-budget, function-domain, nonfinite-evaluation, quadrature-budget, error-bound-unavailable, no-sign-change, continuity-unestablished, root-budget, uniqueness-unresolved, refinement-incompatible, order-unstable, uncertainty-unresolved, calibration-missing, and provenance-incomplete.

A certificate records diagnostic type, function and domain, exact or enclosed nodes, weights, interval transform, consumed evaluations, weighted products and sum, quadrature error assumptions, root endpoints and signs, continuity and uniqueness conditions, proposals and fallbacks, residuals, accepted and rejected slices, refinement family and observed order, arithmetic and uncertainty bounds, budgets, stops, checked scope, unresolved frontier, releases, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, declared nondeterministic inputs, and—for physical comparison—the complete apparatus interaction record.

Governance fixes diagnostic type, domains, node and weight conventions, approximation and enclosure methods, quadrature transform, root predicate, continuity and uniqueness evidence, iteration and fallback rules, refinement and error model, physical comparison source, apparatus requirements, uncertainty, budgets, stop priority, comparison relations, and certificate grammar. A changed field begins a new report.

The action ledger is exact. Diagnostic type, function, domains, nodes, weights, algorithms, smoothness and error models, physical apparatus model, comparisons, budgets, and certificate grammar are *posited*. Polynomial exactness, node-enclosure implications, weighted-sum identities, conditional root existence and uniqueness, safeguarded-bracket invariants, conditional Richardson cancellation, apparatus calibration implications, and certificate implications are *demonstrated*. Evaluations, node bounds, sums, signs, residuals, brackets, iterations, errors, physical observations, stops, and provenance are *read*. General exactness from three nodes, a root from residual alone, quadrature as continued-fraction source, target-fitted extrapolation, labora-

tory capture or containment, identity of α_c with laboratory alpha, and every physical bridge without apparatus remain *reserved*.

Chapter 14

The Machine Measures Its Own Measurement

14.1 The two propositions

Let u and d be two finite records, let a be a declared orientation label, and define atomic propositions

$$U \equiv (u.\text{orientation} = a), \quad D \equiv (d.\text{orientation} = a).$$

The two descriptions are the proposition-shaped expressions

$$P \equiv U \wedge D, \quad Q \equiv D \wedge U.$$

Their content differs only in conjunction order. No particle, orbit, pair, or physical system is introduced by these formulas.

There are explicit implication maps

$$P \rightarrow Q, \quad (u_0, d_0) \mapsto (d_0, u_0), \quad Q \rightarrow P, \quad (d_0, u_0) \mapsto (u_0, d_0).$$

Thus $P \leftrightarrow Q$. Propositional extensionality is the declared principle

$$(P \leftrightarrow Q) \implies (P = Q),$$

which converts logical equivalence into equality of propositions. The equality concerns proposition values in the logical universe. It does not identify their syntax, conjunction order, evaluation traces, resource costs, names, input records, or any objects those names may suggest.

A difference predicate is likewise only a proposition. For records x, y with declared equality,

$$\text{Different}(x, y) \equiv x \neq y.$$

It classifies a relation between two records. It is not a particle, residual substance, force, or physical event. A decision procedure returning yes or no for this proposition returns a Boolean classification together with its declared semantics; word resonance adds nothing.

Decidability and truth evidence remain separate. Decidability supplies a terminating procedure which, for a given proposition A , returns either evidence for A or evidence for $A \rightarrow \perp$. It does not by itself supply the positive branch. A bare Boolean true bit licenses A only when a demonstrated soundness relation connects that bit to A . A procedure name or successful termination is not a truth witness.

Worked example 1: finite evaluation of the atoms (computation).

Read the two orientation fields, compare each with a , and obtain Boolean rows b_U, b_D . Under validated comparison soundness, $b_U = b_D = \text{true}$ supplies evidence for both P and Q . Swapping the two evidence fields demonstrates equivalence. The swap changes order even when proposition equality is accepted.

Counterexample 2: equal propositions, different computations (computation). One evaluator checks U before D ; another checks D before U . If the first check can fail early, their traces and costs can differ even though $P = Q$ under extensionality. Proposition equality is therefore not same-process evidence.

Counterexample 3: decision is not the positive branch (computation). If $u.\text{orientation} \neq a$, the atomic decision returns the negative alternative. The procedure still terminates correctly, but there is no evidence for U , P , or Q . Treating decidable as true erases the returned branch.

The finite decision algorithm parses the records, validates the orientation domain and equality version, evaluates the two atomic comparisons, constructs conjunction rows in each order, verifies the swap maps, and—only if proposition equality is requested—checks that propositional extensionality is among the declared logical dependencies. It slices syntax, atomic outcomes, implication maps, equivalence, proposition equality, trace, cost, and status into separate report fields.

Stops include COMPLETE, RECORD-MISSING, FIELD-MISSING, DOMAIN-MISMATCH, EQUALITY-VERSION-MISMATCH, ATOMIC-DECISION-FAILURE, NEGATIVE-BRANCH, IMPLICATION-FAILURE, EQUIVALENCE-UNRESOLVED, EXTENSIONALITY-NOT-DECLARED, BUDGET, and PROVENANCE-INCOMPLETE. The negative branch is a semantic outcome, not a resource failure. A budget stop leaves the untested proposition unresolved.

A certificate records raw records and labels, proposition syntax, equality and decision semantics, atomic comparison inputs and outputs, soundness evidence, positive or negative branches, conjunction order, implication maps, equivalence row, declared extensionality dependency, proposition-equality row, evaluator traces, resource counters, budgets, stops, checked scope, unresolved frontier, releases, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Comparison follows separate relations. Exact proposition equality is the declared extensional result after typed conversion. Syntax equality compares expression trees and fails when conjunction order differs. Resolution equality compares labels under a declared projection. Same-process requires compatible records, comparison order, short-circuit policy, traces, costs, configuration, and provenance. No combined equality flag may conceal which row passed.

These propositions belong to contravariant naming: the model chooses predicates and pulls records into its own classifications. They do not supply a covariant answer from the world. Physical naming would require an apparatus or robot action

$$(N, s) \longmapsto (N', s', y, \Gamma),$$

with calibration, geometry, environment, prior state, interaction trace, back-action, sensor loading, actuation, energy and material exchange, latency, hysteresis, drift, saturation, rejected outcomes, uncertainty, and alternative causal models. The proposition equality neither constructs this action nor interprets its observation.

Governance fixes record and orientation schemas, equality and decision semantics, proposition syntax, evaluation order, short-circuit policy, implication maps, logical dependencies, comparison relations, budgets, stop priority, and certificate grammar. A changed field begins a new report.

The action ledger is exact. Records, label, predicates, equality, decision semantics, evaluation order, extensionality dependency, comparisons, budgets, and certificate grammar are *posited*. Atomic comparison soundness, implication maps, logical equivalence, conditional proposition equality, decision termination, and certificate implications are *demonstrated*. Atomic outcomes, branches, conjunction rows, traces, costs, stops, and provenance are *read*. Positive truth from decidability alone, syntax identity from proposition equality, same-process or equal cost from equivalence, object identity, parti-

cle meaning for difference, electron descriptions, self-energy, alpha, physical identity, and every world answer absent apparatus remain *reserved*.

14.2 The cost readings

Three counter reports are kept distinct:

H_{orbit} the governed counter reading for the first proposition-shaped workload,
 H_{pair} the governed counter reading for the reordered proposition-shaped workload,
 H_{echo} the governed counter reading for the equivalence/equality workload.

The names orbit, pair, and echo are manuscript labels for computational workload roles. They do not introduce an orbit, a pair of particles, or an acoustic or physical process.

Under one recorded configuration, the exact tuple is

$$(H_{\text{orbit}}, H_{\text{pair}}, H_{\text{echo}}) = (573, 573, 552).$$

The signed self-residual is defined by subtraction orientation:

$$R_{\text{self}} = H_{\text{echo}} - H_{\text{orbit}} = 552 - 573 = -21.$$

This is an exact integer identity in the declared counter currency. Reversing subtraction gives 21, a different reported field. Clamping the negative result or taking an absolute value would erase orientation.

Proposition equality and counter equality are separate. The extensional relation between the two propositions does not imply equal syntax, evaluation order, traces, or costs. Here $H_{\text{orbit}} = H_{\text{pair}}$ is an independently read counter row under this configuration; it is not a consequence of logical equivalence. Likewise, the unequal echo count does not weaken proposition equality.

Worked example 1: exact tuple replay (computation). Replay validates the three workload registrations, counter semantics, outputs, and stops, then reads 573, 573, 552. Arithmetic replay computes $573 - 573 = 0$ for pair minus orbit and $552 - 573 = -21$ for echo minus orbit. These are different slices of one tuple, not competing interpretations of one residual.

Counterexample 2: value equality does not force cost equality (computation). Two evaluators can return the same proposition value while checking conjunction fields in opposite orders. If one field fails early, traces and costs differ. Same value licenses no same-cost or same-process conclusion.

For repeated paired runs $i = 1, \dots, N$, $N \geq 2$, define

$$\delta_i = H_{\text{echo},i} - H_{\text{orbit},i}.$$

Under a declared independent or exchangeable-pair model,

$$\bar{\delta} = \frac{1}{N} \sum_i \delta_i, \quad s_{\bar{\delta}}^2 = \frac{1}{N-1} \sum_i (\delta_i - \bar{\delta})^2, \quad \widehat{\text{Var}}(\bar{\delta}) = s_{\bar{\delta}}^2/N.$$

This last estimate is invalid when blocks, drift, cache history, or run order correlate pairs. With residual covariance matrix Σ_{δ} ,

$$\text{Var}(\bar{\delta}) = N^{-2} \mathbf{1}^T \Sigma_{\delta} \mathbf{1},$$

or a preregistered clustered/block method must match the acquisition design.

The direct route measures the three workloads in a declared paired block. The stepped route records setup, warmup, orbit, pair, and echo stages with their intermediate counters before subtracting. Telescoping differences may satisfy an exact arithmetic identity when they reuse the same readings, but separately acquired routes remain different processes with separate covariance and provenance.

Worked example 3: build-to-build reproducibility (computation).

Suppose two governed builds each return (573, 573, 552). Tuple equality and residual equality are read. Matching counter semantics, compiler release, dependencies, snapshot, configuration, budgets, warmup, cache policy, workload representations, deterministic inputs, and declared nondeterministic inputs are posited and validated from their receipts. The result is reproducibility under that relation, not intrinsic sameness on every machine.

Counterexample 4: near counts are not same process (computation). Two routes may return residuals -21 and -20 . A coarse quantizer may give them one label, but the exact values differ. Even exact residual equality would not establish matching traces, cache interactions, or workload transitions.

The measurement algorithm registers workload roles, validates common counter semantics, executes randomized or balanced blocks, records setup and warmup, reads each output and counter, constructs the tuple, computes signed slices, and updates paired covariance summaries. Failed and rejected runs remain with reasons. Iteration is over blocks and pairs; slicing produces orbit-pair, echo-orbit, absolute, formatted, or statistical views only when requested, with every erasure recorded.

Budgets distinguish registration, setup, warmup, each workload, repetitions, blocks, counter allowance, allocation, search, arithmetic accumulation, covariance estimation, storage, and replay. Stops include COMPLETE, REGISTRATION-FAILURE, SETUP-FAILURE, WARMUP-BUDGET, COUNTER-BUDGET, OUTPUT-FAILURE, UNPAIRED-RUN, INSUFFICIENT-PAIRS, DEPENDENCE-MODEL-INVALID, CONFIGURATION-MISMATCH, CACHE-STATE-UNKNOWN, OVERFLOW, and PROVENANCE-INCOMPLETE.

A certificate records workload definitions and representations, proposition outputs, proposition comparison rows, counter semantics, compiler and dependency releases, snapshot hash, configuration, setup and warmup, pro-

cess and cache policy, raw tuples, block and pair identifiers, run order and randomization, signed subtraction orientation, residual slices, exact sums, covariance and dependence model, direct and stepped routes, rejected and failed runs, budgets, stops, checked scope, unresolved frontier, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Comparison reports exact equality, resolution equality under a declared quantizer, and uncertainty compatibility as independent pass/fail/unresolved rows. Same-process additionally requires matching workload inputs, evaluation transitions, traces, counter policy, setup, warmup, cache state, resource currencies, configuration, and provenance. No combined agreement bit can replace these rows.

Governance fixes workload roles, representations, counter semantics, subtraction orientation, pairing, block order, randomization, setup, warmup, cache policy, dependence model, direct and stepped routes, slicing, comparisons, budgets, stop priority, and certificate grammar. A changed field begins a new series.

Counter cost is neither a witness value nor a physical measurement. It is not Landauer energy, heat, self-energy, alpha, coupling, or evidence of electron identity. A physical energy claim would require a calibrated world-facing meter or robot action with interaction trace, loading, back-action, energy exchange, timing, environment, uncertainty, and alternative causal models.

The action ledger is exact. Workloads, counter semantics, roles, pairing, subtraction orientation, routes, sampling, dependence model, comparisons, budgets, and certificate grammar are *posited*. Integer subtraction, tuple identities, paired summaries, covariance arithmetic, conditional reproducibility comparisons, and certificate implications are *demonstrated*. Outputs, counter tuples, residuals, blocks, pair order, covariance estimates, configuration rows, stops, and provenance are *read*. Cost equality from proposition equality,

proposition identity from equal cost, same-process from near or exact counts, naive variance under correlated pairs, intrinsic build independence, Landauer energy or heat, witness value, self-energy, alpha, coupling, electron identity, and all physical measure remain *reserved*.

14.3 The proposed second pass

The proposed calibration has three typed roles a, b, c . At any pass, two roles are supplied by direct returned or verified values, while the remaining role is unresolved and reconstructed through a preregistered relation. The unresolved marker is part of state:

$$S = (a, b, c; u), \quad u \in \{a, b, c\}.$$

It is not a zero, missing-value default, or physical absence.

Let A, B, C be the three value domains and let Z be a common composite domain. A declared composite is

$$\Sigma : A \times B \times C \rightarrow Z.$$

When c is unresolved, the forward reconstruction map is

$$R_c : A \times B \times Z \rightarrow C,$$

and when a is unresolved after role exchange, the reverse reconstruction map is

$$R_a : B \times C \times Z \rightarrow A.$$

Both maps require domains, units, versions, ambiguity policies, and failure results. A single map in one direction does not establish round-trip calibration.

Pass one returns or verifies a_0, b_0 , reads a composite report z_1 , and pro-

poses

$$\hat{c}_1 = R_c(a_0, b_0, z_1).$$

Pass two changes roles: b and c are directly returned or verified under the second protocol, a new composite z_2 is read, and

$$\hat{a}_2 = R_a(b_2, c_2, z_2).$$

The overlap rows compare b_0 with b_2 , $\hat{c}_1$ with c_2 , and $\hat{a}_2$ with a_0 under their declared relations. Agreement is not one Boolean. Each row separately reports exact equality, resolution equality, and uncertainty-region compatibility as pass, fail, or unresolved.

Round-trip closure requires both reconstruction directions and comparison maps:

$$a_0 \xrightarrow{R_c \text{ with } b_0, z_1} \hat{c}_1 \xrightarrow{R_a \text{ with } b_2, z_2} \hat{a}_2,$$

followed by the typed comparison $\hat{a}_2$ against a_0 . It also requires the overlap comparison for the carried role b . Closure is conditional on the reconstruction model, composite measurements, calibration, and uncertainty. It does not make the three roles one object.

The role scheduler is the exact finite rotation

$$a \mapsto b, \quad b \mapsto c, \quad c \mapsto a.$$

Three scheduler applications return every role, while one or two do not. This certifies role order only. It does not certify value equality, reconstruction correctness, a physical cycle, phase, particle identity, or causal mechanism.

The calibration itself has two measurement passes and one role exchange between them. Three scheduler applications returning every role is an algebraic property of the role scheduler, not a requirement for three calibration passes.

Target independence is procedural. Before observing pass-two outcomes,

governance fixes roles, interventions, reconstruction maps, composite Σ , comparison domains, quantizers, uncertainty model, budgets, exclusions, and stop priority. Choosing or tuning these after reading the target response invalidates the calibration row. Preregistration does not guarantee model truth; it makes target-dependent selection auditable.

Worked example 1: exact algebraic round trip (computation). Let $A = B = C = Z = \mathbb{Q}$, define $\Sigma(a, b, c) = a + b + c$,

$$R_c(a, b, z) = z - a - b, \quad R_a(b, c, z) = z - b - c.$$

If both passes use the same exact $z = \Sigma(a_0, b_0, c_0)$ and return $b_2 = b_0, c_2 = c_0$, then $\hat{c}_1 = c_0$ and $\hat{a}_2 = a_0$. This demonstrates the algebraic maps under exact shared inputs. It does not license a physical interpretation.

Counterexample 2: one-pass planting (calibration). If R_c is chosen after seeing c_2 , it can be made to return that value by construction. The resulting agreement is target-dependent and fails preregistration. The second pass earns information only when the world-facing answer was not selected by the model.

The covariant pass acts through an apparatus or robot:

$$(N, s) \mapsto (N', s', y, \Gamma).$$

The model supplies the contravariant question—which observable, role, and comparison to ask. It may not supply y . The interaction record Γ retains back-action, sensor loading, actuation, energy and material exchange, latency, hysteresis, drift, saturation, geometry, calibration, environment, prior state, rejected outcomes, and alternative causal models. The world answers the intervention in N' , not an untouched N .

Worked example 3: role reversal with apparatus data (calibration).

In pass one, channels a, b are calibrated directly and channel c is reconstructed from a composite observation. In pass two, the apparatus directly calibrates b, c after a preregistered intervention and reconstructs a . Shared sensor standards and environment create covariance across overlap rows. The report propagates this covariance rather than calling the routes independent because their role names differ.

Counterexample 4: internal closure is not physical licensing. A finite table may satisfy both reconstruction identities and exact role rotation using generated data. That is internal consistency. Without a covariant apparatus action, calibration, uncertainty, and competing-cause tests, it does not license physical names or claims about the world.

The numerical algorithm iterates passes and role rotations, validates which two roles are direct and which is unresolved, parses composite reports, applies the appropriate reconstruction, and appends overlap rows. It slices raw returns, reconstructed values, composite values, exact/resolution/uncertainty comparisons, interaction trace, and round-trip status without erasing their provenance.

Budgets distinguish apparatus interventions, samples, role rotations, direct validations, composite evaluations, reconstruction branches, arithmetic growth, covariance and uncertainty propagation, comparison rows, storage, and replay. Stops include COMPLETE, ROLE-INVALID, UNRESOLVED-ROLE-AMBIGUOUS, DIRECT-VALUE-MISSING, COMPOSITE-DOMAIN, RECONSTRUCTION-FAILURE, NONIDENTIFIABLE, OVERLAP-FAILURE, ROUNDTrip-FAILURE, PREREGISTRATION-VIOLATION, INTERVENTION-BUDGET, SENSOR-SATURATION, BACK-ACTION-UNRESOLVED, UNCERTAINTY-UNRESOLVED, and PROVENANCE-INCOMPLETE.

A certificate records role and value schemas, scheduler state, direct and unresolved status, Σ, R_c, R_a , domains and versions, preregistration timestamp and contents, raw and rejected values, both apparatus interventions,

world and robot before/after states, interaction traces, calibration, environment, reconstructed values, overlap rows, covariance, exact/resolution/uncertainty results, round-trip maps, role-rotation checks, budgets, stops, checked scope, unresolved frontier, releases, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Same-number and same-process remain separate. Exact same-number requires equality after typed conversion; resolution equality and uncertainty compatibility are weaker independent relations. Same-process requires matching interventions, inputs, state transitions, reconstruction maps, traces, costs, apparatus configuration, and provenance. Round-trip value agreement alone does not establish it.

Governance fixes roles, scheduler, direct/unresolved policy, composite and reconstruction maps, preregistered interventions, apparatus and observation model, comparison relations, uncertainty and covariance, budgets, stop priority, and certificate grammar. A changed field begins a new calibration.

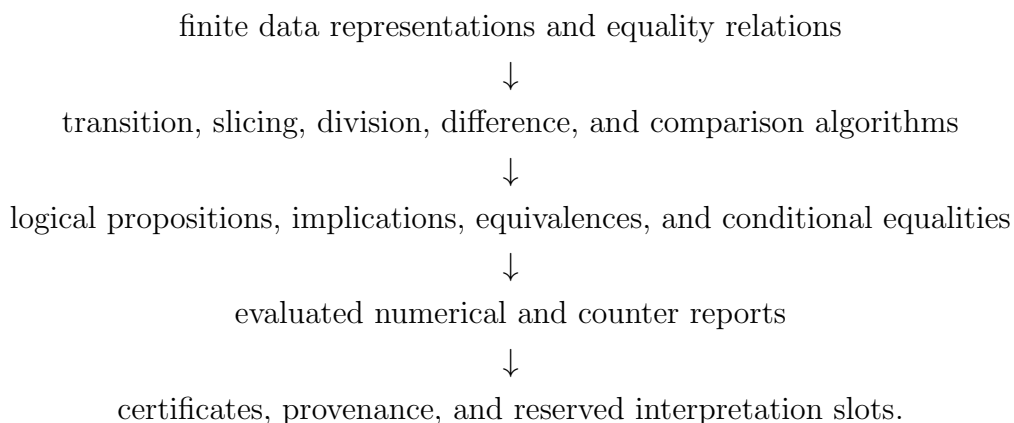
The action ledger is exact. Roles, scheduler, composite, reconstructions, interventions, apparatus model, comparisons, uncertainty, budgets, and certificate grammar are *posited*. Rotation closure, reconstruction identities, conditional overlap and round-trip implications, covariance propagation, and certificate implications are *demonstrated*. Direct returns, composite observations, reconstructed values, overlap rows, apparatus state changes, interaction traces, stops, and provenance are *read*. An answer chosen by the model, one-pass calibration, one object from role rotation, proposition equality as object or electron identity, physical licensing from internal consistency, alpha, coupling, self-energy, and every name bridge absent a world answer remain *reserved*.

Chapter 15

What The Machine Can And Cannot Claim

15.1 What is built

The root construction is a finite dependency graph. Its neutral layers are



An edge means that a downstream node actually consumes the upstream value or declaration. Importing, storing, or naming a field without consuming it creates no dependency for the reported result.

The root surface selects a small set of reports: the two reordered proposition descriptions and their equivalence, the continued-fraction count-to-three jar, the internal ratio reading, the direct and stepped numerical diagnostics, the three workload counter tuple, and the self-input metering report. Each report has its own domain, algorithm, budget, stop, and dependency footprint. Their coexistence does not make them one result, one number, or one process.

The representation layer contains finite records, ordered lists, positive-denominator ratios, indexed samples, three-state roles, proposition syntax, workload descriptions, and typed report schemas. Equality is declared per domain. A shared numeral or spelling across schemas is not a conversion.

The algorithm layer contains finite enumeration and pigeonhole search, deterministic permutation and cycle replay, Euclidean quotient/remainder iteration, scaled-floor projection, second-difference stencils, continued-fraction recurrence and wall selection, bounded root and node enclosure, paired counter subtraction, dependency-graph replay, and two-pass reconstruction. Each algorithm exposes inputs, transitions, projections, erased fields, budgets, stops, and validation rows.

The logical ledger distinguishes:

proposition content	the two conjunction orders and their implication maps,
logical dependencies	declared principles used to convert equivalence into proposition equality,
decision results	positive or negative finite branches,
numerical implications	domain-conditional identities and interval ordering.

A dependency footprint is the transitive set of logical principles and declarations actually used by a claim. An empty or small footprint reports logical dependence only; it does not establish physical truth, empirical independence, or absence of implementation assumptions.

The evaluation ledger contains read outcomes rather than logical consequences: exact integers, rational walls, formatted labels, counter tuples,

stops, and traces. In particular, it retains the open internal jar

(129.6, 137.7)

from three whole continued-fraction partial quotients and the exact direct/stepped scaled integers as distinct diagnostics. The superseded dyadic route owns no current bracket.

The two-pass grammar is built as a typed proposal: roles a, b, c , one unresolved marker, composite Σ , both reconstruction maps, one role exchange between two measurement passes, separately reported overlap rows, and a period-three role scheduler. Its internal algebra and scheduler can be checked on finite generated data. Physical calibration remains conditional on a co-variant apparatus action and world response.

Worked audit 1: graph reachability (computation). To audit a jar endpoint, start at the endpoint node and traverse parents: inverse-ratio map, rational wall, continued-fraction recurrence, partial quotients, and crossing equation. A stored quadrature weight is not reachable and therefore is not a dependency of the endpoint.

Counterexample 2: an imported name is not a built result. A report schema may include a field called field closure or particle identity. Unless an algorithm computes and a certificate validates that claim under a typed domain, the field is a name or input, not a demonstrated result.

The action ledger has four statuses:

POSITED	schemas, domains, transition laws, maps, budgets, and logical dependencies,
DEMONSTRATED	finite identities and implications derived under those declarations,
READ	evaluated or observed report values and branches,
RESERVED	claims not licensed by the preceding three.

Every node carries exactly the statuses relevant to its claims; a read nu-

meral does not become demonstrated physical meaning, and a demonstrated identity does not become a world observation.

Built, measured, and interpreted are likewise distinct. Built means the finite representation, algorithm, relation, or certificate grammar exists and passes its internal checks. Measured means a governed run or apparatus returns a value with provenance. Interpreted means a declared model assigns meaning to that value. A physical interpretation requires the no-name bridge: typed comparison map, calibrated apparatus, observational model, uncertainty, interaction trace, back-action, and provenance.

The graph-audit algorithm parses nodes and typed edges, validates unique identifiers and versions, checks that every consumed input is declared, topologically sorts the graph, rejects cycles where acyclicity is required, replays each finite claim, computes transitive dependency footprints, and slices the ledger by status, report, route, and claim. Projection nodes record their fibers and erasures.

Budgets distinguish graph nodes and edges, dependency traversal, logical normalization, numerical replay, certificate validation, storage, and report slicing. Stops include COMPLETE, SCHEMA-FAILURE, MISSING-NODE, TYPE-MISMATCH, VERSION-MISMATCH, ORPHAN-DEPENDENCY, CYCLIC-DEPENDENCY, UNUSED-FIELD-CLAIM, LOGICAL-DEPENDENCY-MISSING, EVALUATION-MISSING, BUDGET, REPLAY-FAILURE, and PROVENANCE-INCOMPLETE.

A root certificate records graph schema, nodes and typed edges, consumed and unused fields, representations, algorithms, proposition and implication rows, logical dependency footprints, evaluations, action statuses, projections and erasures, budgets, stops, checked scope, unresolved frontier, releases, dependency lock, snapshot hash, configuration, protocol, run identity, platform, deterministic inputs, and declared nondeterministic inputs.

Comparison reports exact equality after typed conversion, resolution equality under a declared quantizer, uncertainty compatibility under a declared region model, and same-process under compatible inputs, transitions, traces,

resources, configuration, and provenance. These are separate rows. Graph isomorphism additionally requires a typed node-and-edge bijection preserving consumption and status.

Governance fixes the root report set, graph and status schemas, logical dependencies, algorithm versions, ownership routes, certificate checks, comparison relations, budgets, stop priority, and release provenance. A changed root or dependency begins a new audit.

The construction is a model of one electron reading only. It is not the world, a nucleus, a catalogue of particles, or a universal theory. Contravariant naming produces the model's own classifications. A covariant world answer requires

$$(N, s) \longmapsto (N', s', y, \Gamma)$$

with apparatus state, prior state and history, intervention, actuation, interaction trace, geometry, calibration, environment, sensor loading, back-action, energy and material exchange, latency, hysteresis, drift, saturation, rejected outcomes, uncertainty, and alternative causal models.

The action ledger is exact. Root selection, schemas, domains, maps, algorithms, logical dependencies, status rules, comparisons, budgets, and certificate grammar are *posited*. Finite representation invariants, algorithm identities, logical implications, dependency reachability, replay and footprint implications, and certificate composition are *demonstrated*. Evaluated integers, rationals, labels, branches, traces, counters, stops, and provenance are *read*. Built as measured, measured as interpreted, a name as world answer, physical truth from a logical footprint, universal scope, other-particle scope, and every claim absent its typed dependency remain *reserved*.

15.2 What is measured

A measurement report begins with governed conditions, not a bare numeral.

The computational inventory is

$\mathcal{M}_{\text{comp}} = (\text{release, dependencies, snapshot, configuration, inputs, algorithm, budgets, raw readings, t})$

Changing any identifying field creates a new measurement context.

The governed counter readings include the exact tuple

$$(573, 573, 552)$$

and the oriented residual

$$552 - 573 = -21.$$

These are allocation or effort-counter values under the recorded counter semantics, not logical steps, wall time, CPU time, energy, or physical cost. Their reproducibility is evaluated only across matched releases, dependencies, snapshots, configurations, workloads, setup, warmup, cache policy, budgets, deterministic inputs, and declared nondeterministic conditions.

The governed numerical readings include the direct and stepped scaled integers

$$I_d = 137011290548979455469,$$

$$I_s = 137011290753751157155,$$

$$I_d < I_s, \quad I_s - I_d = 204771701686,$$

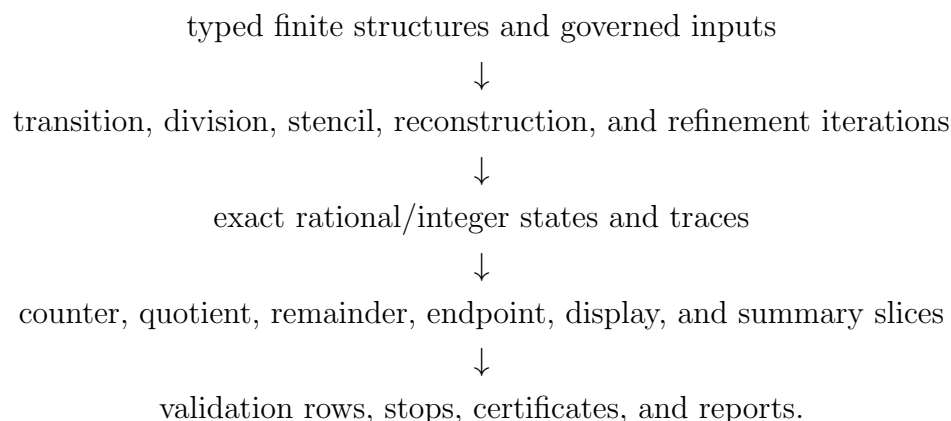
at scale 10^{18} . They are exact integer projections from distinct route reports. Equality, nearness, physical agreement, and process identity are not inferred.

The owned interval reading is the open internal jar

$$(129.6, 137.7),$$

derived from $K = 3$ whole continued-fraction partial quotients and the exact internal inverse map. The endpoints, open status, partial quotients, wall trace, and ownership route are measured report fields. The interval is not a sampling uncertainty region unless a separate uncertainty model says so.

The complete computational data flow is



Every slice records its source and erased fields. A display or current summary cannot reconstruct an iterative trace.

Worked audit 1: counter build reproduction (computation). Two governed builds return $(573, 573, 552)$. Exact tuple equality is read. The audit separately validates releases, dependency locks, snapshot hashes, configurations, workload representations, budgets, cache strata, and declared nondeterministic inputs. Only after those rows pass is the result called reproducible under the declared relation.

Counterexample 2: equal output under a changed configuration (computation). If two configurations happen to return 552, the integer row passes while same-process and matched-build rows fail or remain unresolved. Output equality cannot erase the configuration difference.

Parameter sensitivity is measured by governed perturbations. For param-

eter vector θ , report

$$y(\theta), \quad S_j(h) = \frac{y(\theta + he_j) - y(\theta - he_j)}{2h}$$

when both perturbed runs are in domain. This centered sensitivity is a numerical estimate whose interpretation requires a declared step h , arithmetic model, smoothness or local-fit assumption, and error study. One-sided differences are separate methods at boundaries.

Sensitivity iteration evaluates preregistered perturbation levels $h, h/2, h/4$, records raw changes, checks cancellation and arithmetic resolution, and estimates stability only within a compatible family. Richardson extrapolation requires an analytically justified or independently stabilized order, decreasing raw differences, and a demonstrated remainder bound. Selecting h or an order after seeing a desired target is rejected.

Worked audit 3: discrete parameter sensitivity (computation). For integer parameter K , changing $K = 3$ to $K = 4$ changes the continued-fraction report and ownership depth. The finite difference $J(4) - J(3)$ compares two reports; it is not a derivative unless a continuous embedding and model justify that name. The exact report difference remains useful as a discrete sensitivity.

Build-to-build analysis slices measurements by release, dependency set, snapshot, configuration, platform, cold/warm state, cache policy, run order, outcome, stop, and deterministic or nondeterministic input. It retains failed and rejected runs. Paired or blocked statistics require the declared sampling and dependence model; naive independent variance is not used for correlated builds.

Budgets distinguish structure parsing, iterations, counter allowance, arithmetic and bit growth, route evaluations, parameter perturbations, sensitivity levels, repetitions, covariance estimation, storage, comparison, and replay. Stops include COMPLETE, SCHEMA-FAILURE, CONFIGURATION-MISMATCH,

DEPENDENCY-MISMATCH, SNAPSHOT-MISMATCH, COUNTER-BUDGET, ITERATION-BUDGET, ARITHMETIC-BUDGET, OVERFLOW, PARAMETER-DOMAIN, SENSITIVITY-STEP-UNRESOLVED, CANCELLATION-UNRESOLVED, ORDER-UNSTABLE, INSUFFICIENT-REPETITIONS, DEPENDENCE-MODEL-INVALID, OUTPUT-FAILURE, and PROVENANCE-INCOMPLETE.

A certificate records toolchain and counter semantics, releases, dependency lock, snapshot hash, configuration, platform, input structures, algorithms, setup and warmup, cache and process policy, all iterative states, raw counter and numerical readings, exact route integers, jar fields, projections and erasures, parameter perturbations, sensitivity estimates and error rows, paired or blocked statistics, rejected and failed runs, budgets, stops, checked scope, unresolved frontier, protocol, run identity, deterministic inputs, and declared nondeterministic inputs.

Comparison reports exact equality after typed conversion, resolution equality under a declared quantizer, uncertainty compatibility under a declared region model, and same-process under compatible inputs, algorithms, traces, resources, configuration, and provenance. Each is a separately reported pass/fail/unresolved row. Reproducibility is a comparison claim over governed reports, not an intrinsic property of a numeral.

Physical measurement is a different relation:

$$(N, s) \longmapsto (N', s', y, \Gamma).$$

It includes prior world and robot state, intervention, actuation, interaction trace, sensor loading, back-action, energy and material exchange, latency, hysteresis, drift, saturation, geometry, calibration, environment, rejected outcomes, uncertainty, and alternative causal models. The observation y answers the intervention in N' . Computational reports do not become physical measurements by sharing a name or number.

Governance fixes measurement schemas, counter semantics, releases, configurations, workload and numerical routes, ownership, slicing, parameter

perturbations, sensitivity and error methods, repetition and dependence models, comparison relations, budgets, stop priority, and certificate grammar. A changed field begins a new series.

The measured scope is the model's one electron reading and its computational diagnostics. It is not the world, a nucleus, other particles, or a universal computational limit. The laboratory value 137.036 is neither contained, approached, nor compared by proximity here.

The action ledger is exact. Measurement contexts, routes, parameters, perturbations, sampling, sensitivity models, comparisons, budgets, and certificate grammar are *posited*. Exact arithmetic, route replay, conditional sensitivity and error implications, paired/block summaries, matched-build comparison, and certificate implications are *demonstrated*. Counter tuples, residuals, route integers, jar fields, traces, perturbation outcomes, sensitivity estimates, stops, and provenance are *read*. Physical measurement from computation alone, reproducibility across unmatched builds, process identity from equal outputs, derivative meaning for arbitrary discrete differences, target-fitted sensitivity, hidden magnitude, uncomputability of alpha, universal impossibility, laboratory containment or proximity, and every world claim absent intervention remain *reserved*.

15.3 What is interpreted

The representational grammar is finite because every map has a checked domain, and local because no successful map is promoted beyond that domain, route, resolution, and provenance. Interpretation begins after construction and reading, not before. The closing ledger has three columns:

BUILT	finite representations, algorithms, relations, and certificate grammars,
MEASURED	governed computational reports or calibrated apparatus observations,
INTERPRETED	names and models assigned under declared bridges and scope.

A fourth status, `RESERVED`, marks a claim for which no licensed bridge is present.

The name-resonance row is explicit:

<code>nowtrino(<i>H</i>)</code>	current computational projection after history slicing,
<code>newtrino(<i>s</i>)</code>	next computational state under a declared update,
<code>naotrino</code>	manuscript physical candidate requiring an apparatus record,
<code>ν-trino</code>	the conventional neutrino-physics register.

The declared update may connect `nowtrino` to `newtrino` after their representations are typed. No spelling, sound, or three-fold pattern connects either computational name to `naotrino` or `$\nu$ -trino`. A physical `naotrino` remains reserved until system, observable, units, apparatus, calibration, uncertainty, interaction, and discriminating data are supplied. The manuscript coinage derives from Greek *naos*, meaning temple. This etymology supplies provenance for the name, not evidence for a physical entity. It is not the conventional neutrino.

The internal name α_c belongs first to the compiler and model ledger. It names the model-owned electron reading generated by the internal formula and routes. It does not name a laboratory coupling or a property of the world. The computation owns the open jar

$$(129.6, 137.7),$$

whose endpoints are exact diagnostics obtained from adjacent continued-fraction walls at $K = 3$. The jar is an internal interval report; it does not assert a true value caught inside.

The reader can replay the built two-pass grammar on governed computational records. Its pass-one and pass-two roles exchange once, apply the two reconstruction directions, and report overlap and round-trip rows. Physical covariant apparatus passes are auditable only when two corresponding

intervention receipts exist. The scheduler separately has period three:

$$a \mapsto b \mapsto c \mapsto a.$$

That algebraic period is not three measurement passes. Two passes with one role exchange are the calibration protocol; three scheduler steps are a property of role ordering.

The numerical route ledger also remains split. At scale 10^{18} ,

$$\begin{aligned} I_d &= 137011290548979455469, \\ I_s &= 137011290753751157155, \\ I_d &< I_s, \quad I_s - I_d = 204771701686. \end{aligned}$$

These are unequal exact integers from direct and stepped processes. A formatter or quantizer may create a declared resolution relation, but it cannot change exact inequality or establish process identity.

Two threes remain separate. Continued-fraction $K = 3$ counts three whole partial quotients and sources the internal jar. A three-point Gauss–Legendre diagnostic counts quadrature nodes and produces a weighted integral approximation under its own function and error model. It is not the jar source, interval width, or remainder.

The laboratory inverse alpha summary 137.036 is a different, separately sourced physical report. It belongs to its apparatus, model, units and conventions, calibration chain, data reduction, uncertainty, and provenance. This volume makes no proximity, containment, prediction, capture, or shared-value claim between it and α_c .

The no-name bridge is the governing rule. A model supplies contravariant naming: it chooses the question, representation, predicate, and comparison. The covariant answer requires a robot or apparatus action

$$(N, s) \longmapsto (N', s', y, \Gamma).$$

The record includes prior world and robot state and history, intervention, actuation, interaction trace, geometry, sensor loading, back-action, energy and material exchange, latency, hysteresis, drift, saturation, calibration, environment, rejected outcomes, uncertainty, and alternative causal models. The model may choose what to ask; it may not choose y . The world answers the intervention in N' , not an untouched N .

Final audit 1: a licensed computational interpretation. A full history H is sliced to its current state and counters. The ledger labels that projection nowtrino, records its fiber and erased history, and applies a declared update to obtain newtrino. This interpretation is licensed because the domains and transition are explicit. It remains computational.

Final audit 2: a rejected physical coercion. A report replaces the label nowtrino with naotrino but supplies no apparatus action or observation. The spelling row passes while domain, model, calibration, uncertainty, and world-answer rows fail. The physical name is rejected or reserved, regardless of numerical agreement elsewhere.

Final audit 3: what the reader owns. After replaying the continued-fraction trace and the two-pass grammar on computational records, the reader owns the internal open jar, its endpoint diagnostics, route reports, exact comparison rows, computational reconstruction rows, stops, and provenance. If two covariant apparatus receipts are supplied, the reader may additionally audit those receipt-backed passes and their world-answer rows. The reader does not thereby own a selected point, a physical entity, a laboratory constant, or a universal claim.

The final audit algorithm parses every name occurrence, joins it to one governed ledger row by name, domain, route, and version, traverses dependencies, replays finite computations, validates measurement provenance, and

evaluates candidate bridges. It slices results into built, measured, interpreted, reserved, ambiguous, and rejected statuses. It retains collisions and unresolved frontiers rather than choosing a convenient meaning.

Comparison remains typed:

exact same-number	equality after typed conversion,
resolution equality	equal labels under a declared quantizer,
uncertainty compatibility	overlap of declared regions,
same-process	compatible inputs, transitions, traces, costs, configuration, and provenance

Each row is separately reported as pass, fail, or unresolved. Physical licensing adds apparatus, observation model, calibration, interaction, uncertainty, and competing-cause rows.

Budgets distinguish dependency traversal, numerical replay, name resolution, route comparison, calibration passes, apparatus validation, uncertainty propagation, alternative-model tests, storage, and report generation. Stops include COMPLETE, NAME-UNKNOWN, LEDGER-COLLISION, DOMAIN-MISMATCH, ROUTE-MISMATCH, VERSION-MISMATCH, DEPENDENCY-MISSING, REPLAY-FAILURE, COMPARISON-UNRESOLVED, PREREGISTRATION-VIOLATION, APPARATUS-MISSING, CALIBRATION-MISSING, BACK-ACTION-UNRESOLVED, UNCERTAINTY-UNRESOLVED, BUDGET, and PROVENANCE-INCOMPLETE.

The final certificate records all schemas and name rows, built constructions, consumed dependency graph, direct and stepped traces, continued-fraction partial quotients and walls, open jar fields, quadrature diagnostics, two-pass roles and reconstructions, scheduler checks, counter and numerical readings, exact/resolution/uncertainty/process rows, projections and erasures, apparatus records when present, rejected bridges, budgets, stops, checked scope, unresolved frontier, releases, dependency locks, snapshot hashes, configurations, protocols, run identities, platforms, deterministic inputs, and declared nondeterministic inputs.

Governance fixes the root ledger, name grammar, ownership routes, com-

parison and bridge requirements, two-pass protocol, scheduler, internal jar, apparatus grammar, uncertainty, budgets, stop priority, and certificate rules. A changed field begins a new closing audit.

The scope remains one model-owned electron reading. There is no electron identity claim, no extension to a nucleus or other particles, and no universal theory. There is no self-energy or physical coupling claim. The closing achievement is narrower: the model states its own names, the computation reports what it read, the apparatus boundary states what would be required for the world to answer, and the ledger refuses every unsupported crossing.

The action ledger is exact. Names, ownership, bridges, models, routes, comparisons, apparatus requirements, budgets, and certificate grammar are *posited*. Dependency and replay implications, exact arithmetic, role and scheduler identities, conditional reconstruction, calibrated comparison, and certificate composition are *demonstrated*. Internal integers, jar endpoints, route differences, counter tuples, calibration rows, observations when present, stops, and provenance are *read*. A selected true value in the jar, computation as world answer, physical identity from resonance, naotrino or neutrino from computational names, equality of unequal routes, quadrature as bracket source, scheduler period as pass count, laboratory proximity or containment, uncomputability, universal limits, electron identity, self-energy, coupling, and every bridge absent covariant apparatus remain *reserved*.

Bibliography

- [1] Niels Bohr. “The Quantum Postulate and the Recent Development of Atomic Theory”. In: *Nature* 121.3050 (1928), pp. 580–590.
- [2] Gregory J. Chaitin. “Information-Theoretic Limitations of Formal Systems”. In: *Journal of the ACM* 21.3 (1974), pp. 403–424. DOI: 10.1145/321832.321839.
- [3] Paul J. Cohen. “The Independence of the Continuum Hypothesis”. In: *Proceedings of the National Academy of Sciences* 50.6 (1963), pp. 1143–1148.
- [4] Kurt Gödel. “The Consistency of the Axiom of Choice and of the Generalized Continuum-Hypothesis”. In: *Proceedings of the National Academy of Sciences* 24.12 (1938). Choice is consistent with the other axioms, pp. 556–557.
- [5] David Hume. *An Enquiry Concerning Human Understanding*. London: A. Millar, 1748.
- [6] Immanuel Kant. *Kritik der reinen Vernunft*. Riga: Johann Friedrich Hartknoch, 1781.
- [7] Per Martin-Löf. *Intuitionistic Type Theory*. Ed. by Giovanni Sambin. Notes of lectures given in Padua, June 1980. Naples: Bibliopolis, 1984.
- [8] John von Neumann. *Mathematical Foundations of Quantum Mechanics*. Princeton: Princeton University Press, 1932.

- [9] Claude E. Shannon. “A Mathematical Theory of Communication”. In: *Bell System Technical Journal* 27 (1948), pp. 379–423, 623–656.
- [10] Alan M. Turing. “On Computable Numbers, with an Application to the Entscheidungsproblem”. In: *Proceedings of the London Mathematical Society*. 2nd ser. 42 (1936), pp. 230–265.
- [11] Ernst Zermelo. “Beweis, dass jede Menge wohlgeordnet werden kann”. In: *Mathematische Annalen* 59.4 (1904). The axiom of choice stated, to prove every set well-orderable, pp. 514–516.